SPM622 Software Engineering 2: Project & Dissertation

# TrackWIL

Weekly Submission 4: Working Application, Chapters 4 and 5, Testing and Presentation Readiness

Group 3: Willem (Gedion) Basson, Moegammad Green, Dylan Shepard, Sihle Kayise

Supervisor: Ms Amenda Raganya | Submitted: 23 October 2026

Table 1: Team details and links

| Item | Details |
|---|---|
| Project title | TrackWIL: Work-Integrated Learning Logbook Tracking |
| Group | Group 3, Cape Town Campus |
| Group lead | Willem (Gedion) Basson |
| GitHub repository | https://github.com/Noobey32/TrackWIL |
| Application access | Run locally from the repository: 05_Source_Code/trackwil (npm install, npm run dev); demonstration accounts are listed in the README |

## Contents of this Submission

Table 2: Week 4 requirements and where they are met

| Week 4 requirement | Where to find it |
|---|---|
| 1. Working application with the database integrated; walkthrough video | GitHub repository (05_Source_Code/trackwil, SQLite database); video link above |
| 2. Chapter 4 complete (4.6 to 4.8): development environment, modules and features with screenshots, version control | Chapter 4 (also in the dissertation draft) |
| 3. Chapter 5 draft: test strategy, test cases and results, defects, performance and usability evaluation, requirements evaluation, feedback | Chapter 5 (also in the dissertation draft) |
| 4. Testing workbooks | Test Plan; Test Cases and Results (81 test cases); Defect Log (7 defects); Requirements Traceability Matrix (25 requirements) |
| 5. Full dissertation draft: front matter and Chapters 1 to 5 with an outline of Chapter 6 | Dissertation draft (abstract, contents, lists of figures and tables, abbreviations, Chapters 1 to 5, Chapter 6 outline, references, appendices) |
| 6. Presentation plan and Individual Contribution Forms | Demo Script and Slide Outline; Individual Contribution Forms for all four members |
| Supporting evidence | Software Requirements Specification; Database Design document; Weekly Progress Report; Sprint Backlog; Risk, Issue and Change Register |

## Changes Since Week 3

Table 3: Changes made since the Week 3 submission

| Area | What changed | Where |
|---|---|---|
| Application | JSON file storage replaced with an SQLite relational database (keys, constraints, transactions); session cookie now signed; expired links can be resubmitted; mobile layout fixed; 36 automated system tests added | Pull request #5; branch gedion/fix-defects |
| Chapters 1 to 3 and SRS | Updated to match the system that was built (mentors verify by link without an account; coordinators and lecturers share one dashboard); scope split into included and excluded; measurable non-functional requirements; requirements-gathering evidence; Agile justified against Waterfall | Chapters 1 to 3; SRS |
| Chapter 2 | Technology review names the chosen stack; comparison table of similar systems; citations matched to the reference list | Chapter 2 |
| Chapter 4 | Completed (4.1 to 4.9); new class and activity diagrams; updated architecture, use case, sequence and ER diagrams; screenshots; version-control evidence | Chapter 4 |
| Testing | Test Plan rewritten; 55 test cases with actual results; 5 defects logged (3 fixed and retested); traceability matrix completed | Folder 06 workbooks; Chapter 5 |
| Formatting | Template covers, headers and footers removed; headings kept with their text, paragraphs not split across pages, figures and tables kept with their captions; one consistent heading structure | All documents |
| Project management | Sprint backlog, risk register and weekly progress report updated | Folder 09 |

CTU Training Solutions, Cape Town Campus

SPM622 Software Engineering 2: Project & Dissertation

# TrackWIL

A web application for Work-Integrated Learning logbooks with account-free mentor verification and progress monitoring

Dissertation draft (Chapters 1 to 5 and an outline of Chapter 6)

Group 3: Willem (Gedion) Basson, Moegammad Green, Dylan Shepard, Sihle Kayise

Supervisor: Ms Amenda Raganya

Week 4 submission: 23 October 2026

## Team Details

Table 1: Team details

| Item | Details |
|---|---|
| Project title | TrackWIL: Work-Integrated Learning Logbook Tracking |
| Group | Group 3, Cape Town Campus |
| Group members | Willem (Gedion) Basson (Group Lead), Moegammad Green (Development Lead), Dylan Shepard (Testing / QA), Sihle Kayise (Group Member) |
| Module | SPM622 (Software Engineering 2) |
| Supervisor | Ms Amenda Raganya |
| Stage | Week 4: Build, Test and Presentation |
| Submission date | 23 October 2026 |
| GitHub repository | https://github.com/Noobey32/TrackWIL |

## Declaration

We, the members of Group 3, declare that this dissertation and the TrackWIL application are the work of our group, that every source used has been acknowledged and referenced in the Harvard style, and that each member's contribution is recorded accurately in the Individual Contribution Forms and the Git history.

Table 2: Signatures

| Member | Signature | Date |
|---|---|---|
| Willem (Gedion) Basson | GedionBasson | 29/10/26 |
| Moegammad Green | M.A.Green | 29/10/26 |
| Dylan Shepard | D.Shepard | 29/10/26 |
| Sihle Kayise | S.Kayise | 28/10/2026 |

## Abstract

Work-Integrated Learning (WIL) students must record their daily workplace activities and have them signed by a workplace mentor. When this is done with paper, PDF forms and email, documents are lost, signatures are delayed and coordinators cannot see which students are falling behind. This project aimed to design, develop and evaluate TrackWIL, a web application in which students record daily logbook entries with evidence for each WIL placement and upload their placement documents, workplace mentors verify each entry through a secure one-time link without needing an account, and coordinators and lecturers monitor every student's progress on one dashboard. A hybrid Agile (Scrum) approach with one-week sprints was used. The system was built with Next.js and TypeScript and an SQLite relational database, and its design is documented with use case, class, activity and sequence diagrams and an entity-relationship diagram. TrackWIL was tested with 81 test cases, combining manual tests, 62 automated system tests and browser checks: 80 passed and one usability test is waiting for ethical permission. Testing found and fixed seven defects, including two critical access-control vulnerabilities (a forgeable session cookie and page data sent with redirect responses), an expired-link problem and a mobile layout defect. All eighteen functional requirements and six of the seven non-functional requirements were met, and pages responded in under 10 milliseconds on the test computer. The results show that account-free, link-based verification combined with automatic status monitoring can replace the manual logbook process; the next step is to measure its usability with real students, mentors and staff.

# Table of Contents


Team Details ..... 2
Declaration ..... 3
Abstract ..... 4
List of Abbreviations ..... 9
Key Terms ..... 10
CHAPTER 1 - INTRODUCTION ..... 11
1.1 Background of the Study ..... 11
1.2 Problem Statement ..... 11
1.3 Research Aim ..... 12
1.4 Research Objectives ..... 12
1.5 Research Questions ..... 12
1.6 Proposed Software Solution ..... 12
1.7 Scope ..... 13
1.8 Significance of the Study ..... 14
1.9 Chapter Summary ..... 15
CHAPTER 2 - LITERATURE REVIEW ..... 16
2.1 Introduction ..... 16
2.2 Existing Research ..... 16
2.3 Existing or Similar Systems ..... 17
2.4 Technology Review ..... 18
2.5 Research Gap ..... 19
2.6 Conceptual / Technical Direction ..... 19
2.7 Chapter Summary ..... 20
CHAPTER 3 - REQUIREMENTS & METHODOLOGY ..... 21
3.1 Introduction ..... 21
3.2 Stakeholders and Users ..... 21
3.3 Functional Requirements ..... 22
3.4 Non-Functional Requirements ..... 24
3.5 Requirements Gathering ..... 24
3.6 Development Methodology ..... 25
3.7 Use Cases / User Stories ..... 26
3.8 System Architecture ..... 27
3.9 Ethical, Privacy and Security Considerations ..... 28

3.10 Chapter Summary ..... 29
CHAPTER 4 - SYSTEM DESIGN & DEVELOPMENT ..... 30
4.1 Introduction ..... 30
4.2 System Architecture ..... 30
4.3 UML Design ..... 32
4.4 Database Design ..... 36
4.5 User Interface Design ..... 38
4.6 Development Environment ..... 45
4.7 Implementation ..... 46
4.8 Version Control ..... 48
4.9 Chapter Summary ..... 50
CHAPTER 5 - TESTING, RESULTS & EVALUATION ..... 51
5.1 Introduction ..... 51
5.2 Test Strategy ..... 51
5.3 Test Cases and Results ..... 51
5.4 Defects and Corrections ..... 54
5.5 Performance / Usability Evaluation ..... 55
5.6 Requirements Evaluation ..... 56
5.7 User / Stakeholder Feedback ..... 57
5.8 Chapter Summary ..... 57
CHAPTER 6 - CONCLUSION & RECOMMENDATIONS (OUTLINE) ..... 58
6.1 Introduction ..... 58
6.2 Summary of Findings ..... 58
6.3 Achievement of Objectives ..... 58
6.4 Answers to Research Questions ..... 58
6.5 Limitations ..... 59
6.6 Recommendations ..... 59
6.7 Future Work ..... 59
6.8 Final Conclusion ..... 59
References ..... 60
Appendices ..... 62
Appendix A: Source Code and How to Run TrackWIL ..... 62
Appendix B: Supporting Documents ..... 62
Appendix C: Mobile Test Evidence ..... 63

# List of Figures

Figure 1: TrackWIL system architecture ..... 30
Figure 2: Use case diagram..... 33
Figure 3: Class diagram of the main types and modules..... 34
Figure 4: Activity diagram of the logbook and verification workflow ..... 35
Figure 5: Sequence diagram of mentor verification ..... 36
Figure 6: Entity-relationship diagram (SQLite) ..... 37
Figure 7: Sign-in page with an error for a wrong password ..... 39
Figure 8: Student's WIL placements with hours and document progress (WIL 1 completed) ..... 40
Figure 9: Placement logbook with verified hours, WIL documents and entry statuses ..... 40
Figure 10: New logbook entry form ..... 41
Figure 11: Entry details with the mentor verification link ..... 41
Figure 12: Mentor verification page (no sign-in required) ..... 42
Figure 13: Coordinator dashboard with each student's status, hours and missing documents..... 43
Figure 14: Student profile for coordinators and lecturers: placement hours, documents (view only) and logbook ..... 44
Figure 15: A completed placement: the student is asked to add the next WIL placement..... 44
Figure 16: Collaborators on the GitHub repository ..... 50
Figure 17: Manual test TC-011: an invalid sign-in is refused with a clear message (Sihle Kayise, 22 October 2026)..... 53
Figure 18: Manual test TC-009: coordinator dashboard during manual testing (Sihle Kayise, 22 October 2026) ..... 54
Figure 19: Mentor verification page after the DEF-03 fix (TC-145) ..... 63
Figure 20: Coordinator dashboard after the DEF-03 fix: the table scrolls inside its box (TC-145) ..... 64

# List of Tables

Table 1: Team details ..... 2
Table 2: Signatures ..... 3
Table 3: Abbreviations ..... 9
Table 4: Key terms ..... 10
Table 5: Comparison of similar systems against TrackWIL's needs ..... 17
Table 6: Stakeholders, their needs and how they use TrackWIL ..... 21
Table 7: Functional requirements ..... 22
Table 8: Requirements deferred to future versions ..... 23
Table 9: Non-functional requirements ..... 24
Table 10: Requirements-gathering methods and evidence ..... 25
Table 11: Sprint plan ..... 26
Table 12: User stories and acceptance criteria ..... 26
Table 13: Main components and their responsibilities ..... 31
Table 14: Technology stack and justification ..... 32
Table 15: Usability decisions ..... 45
Table 16: Development environment ..... 45
Table 17: Source code structure (05_Source_Code/trackwil) ..... 46
Table 18: Branches in the repository ..... 48
Table 19: Commits and main contributions by member (at the time of writing) ..... 49
Table 20: Summary of test results ..... 51
Table 21: Defects found, fixes and retest results ..... 54
Table 22: Measured response times (TC-144, 26 October 2026) ..... 55
Table 23: Evaluation against the requirements ..... 56
Table 24: Achievement of objectives (to be expanded) ..... 58
Table 25: Demonstration accounts (synthetic data) ..... 62
Table 26: Supporting documents in the repository ..... 62

# List of Abbreviations

Table 3: Abbreviations

| Abbreviation | Meaning |
|---|---|
| CTU | CTU Training Solutions |
| DEF | Defect (Defect Log identifier) |
| ERD | Entity-relationship diagram |
| FR / NFR | Functional requirement / non-functional requirement |
| HMAC | Hash-based message authentication code |
| HTTP(S) | Hypertext Transfer Protocol (Secure) |
| JSON | JavaScript Object Notation |
| MOU | Memorandum of Understanding (agreement between the institution, the host company and the student) |
| MVP | Minimum viable product |
| OWASP | Open Worldwide Application Security Project |
| POPIA | Protection of Personal Information Act 4 of 2013 |
| PR | Pull request |
| QA | Quality assurance |
| RTM | Requirements Traceability Matrix |
| SQL | Structured Query Language |
| SRS | Software Requirements Specification |
| TC | Test case |
| UI | User interface |
| UML | Unified Modeling Language |
| WIL | Work-Integrated Learning |

## Key Terms

Table 4: Key terms

| Term | Definition |
|---|---|
| Logbook entry | One day of WIL work recorded by a student: date, description, hours and the mentor's email address |
| Workplace mentor | The person at the host organisation who supervises the student and verifies their entries |
| Verification link (token) | A one-time web address containing a 48-character random token that lets a mentor approve or reject one entry without signing in; it expires after 7 days |
| Verified hours | The total hours of a placement's entries that a mentor has approved |
| Overdue | A student whose current placement has no verified entry in the last 14 days (counted from the placement start if nothing is verified yet) and no entry awaiting verification |
| WIL placement | One period of WIL at a host company, with its own required hours, logbook and documents |
| WIL completed | The status of a placement whose verified hours have reached its required hours; it is then locked |
| Server Action | A Next.js function that runs on the server when a form is submitted |
| Transaction | A group of database changes that are saved together or not at all |
| Regression testing | Rerunning all tests after a change to make sure nothing that worked before has broken |

# CHAPTER 1 - INTRODUCTION

## 1.1 Background of the Study

Work-Integrated Learning (WIL) is an important part of many occupational and technical qualifications because it gives students an opportunity to gain practical workplace experience and apply the knowledge and skills they have learned during their studies. During a WIL placement, students are required to complete workplace activities, record the work they have done, complete daily documentation and provide evidence of their activities.

Currently, the management of WIL activities can involve students completing documents daily and sending them to their workplace mentors or supervisors for verification and signing. Students then must wait for the signed documents to be returned before they can continue with the submission or verification process. These documents may be exchanged through email or other communication methods, which can make it difficult for students and academic staff to keep track of all required documents.

When documents are stored in different locations or sent between students and mentors, there is also a risk that documents may be misplaced, accidentally deleted or not returned on time. Coordinators and lecturers may also have difficulty determining which students have completed their daily requirements and which students are still waiting for documentation to be completed.

TrackWIL is proposed as a digital solution to improve this process. It gives students one central place to record their daily WIL activities, attach evidence and request verification. Workplace mentors confirm the work through a secure link without needing an account, and coordinators and lecturers monitor student progress from a shared dashboard.

## 1.2 Problem Statement

The current WIL documentation process can be time-consuming because students are required to complete daily documents and send them to their workplace mentors for verification and signing. Students then must wait for the signed documents to be returned. When this process is managed through emails, separate files or other manual methods, documents can be misplaced, accidentally deleted or returned late.

This affects students, because they need to keep track of their daily documents and ensure that they are completed and signed by their mentors. Delays in receiving signed documents can also make it difficult for students to keep their WIL records up to date.

It also affects WIL coordinators, who need to monitor whether students are completing their required activities and receiving the necessary verification. Checking documents from multiple students can become difficult and time-consuming.

Lecturers are also affected because they need to identify which students are up to date and which students are falling behind. Without a centralised system, it can be difficult to obtain an accurate and current view of each student's WIL progress.

Workplace mentors are affected as well. They are busy staff at host organisations who are not employees of the institution, so a process that requires them to sign paper or PDF forms, or to create and remember another account, adds work and causes further delays.

### 1.3 Research Aim

The aim of this project is to design, develop and evaluate TrackWIL, a centralised web application that allows students to record their daily workplace activities and evidence, allows workplace mentors to verify those records through a secure link, and enables coordinators and lecturers to monitor student progress.

### 1.4 Research Objectives

1. Analyse the current WIL documentation and monitoring process to identify the challenges experienced by students, workplace mentors, coordinators and lecturers.

2. Design a WIL logbook system in which students submit daily entries and evidence, and workplace mentors verify each entry through a one-time link without needing an account.

3. Develop the TrackWIL web application with a student logbook, a mentor verification page and a monitoring dashboard shared by coordinators and lecturers.

4. Implement secure storage and status tracking so that every entry is recorded in a database with its status (Draft, Pending verification, Verified or Rejected) and timestamps.

5. Test and evaluate TrackWIL against its requirements to determine whether it makes WIL documentation, verification and progress monitoring more efficient.

### 1.5 Research Questions

1. What challenges are experienced by students, workplace mentors, coordinators and lecturers when managing WIL documentation and activities using the current process? (Objective 1)

2. How can a digital workflow, in which mentors verify entries through a one-time link, improve the submission and verification of students' daily WIL records? (Objective 2)

3. How can TrackWIL support each user group through role-specific views: a student logbook, a mentor verification page and a coordinator and lecturer dashboard? (Objective 3)

4. How can storing entries, evidence and verification status in a central database improve the management and monitoring of student WIL activities? (Objective 4)

5. To what extent does TrackWIL meet its requirements and improve the efficiency of WIL documentation, verification and progress monitoring? (Objective 5)

### 1.6 Proposed Software Solution

The proposed software solution is TrackWIL, a web-based application designed to improve the management of Work-Integrated Learning activities and documentation. Each user group sees a view that matches its role.

Students sign up with their CTU student email address and add each of their WIL placements, with the host company, the start date and the number of hours the placement requires; each placement has its own logbook. They record each day's work (date, description and hours), name their workplace mentor's email address and may attach a PDF or image as evidence. When an entry is ready, the student submits it for verification. The system then creates a secure link that is valid for seven days and can be used only once, and the student sends this link to the mentor. The logbook shows the status of every entry and the verified hours completed against the placement's required hours. For each placement the student also uploads the WIL documents: the Memorandum of Understanding (MOU), the Commencement Letter and the Code of Conduct once, and an Attendance Register every semester. When the verified hours reach the required hours, the placement is marked WIL completed and locked, and the student adds the next placement to continue.

Workplace mentors do not need an account. They open the link, review the entry and approve or reject it. If the link has already been used or has expired, the system tells the mentor and does not change the entry. A rejected entry can be corrected by the student and submitted again.

WIL coordinators and lecturers sign in to a shared monitoring dashboard. It lists every student with their verified hours, number of entries, number of entries awaiting verification and number of missing documents, and labels each student by their current placement as On track, Awaiting verification, Overdue (no verified work in the last 14 days), WIL completed or No placement. Selecting a student opens their profile with each placement's hours, documents and full logbook. This allows staff to identify students who need follow-up.

### 1.7 Scope

The scope was set so that the core workflow could be built, tested and demonstrated within the four-week project period.

#### Included in the project

- Student sign-up with a CTU student email address and password rules, sign-in and sign-out.
- Role-based access for students, coordinators and lecturers.
- WIL placements: a student can add several placements, each with its own company, start date and required hours.
- Daily logbook entries with date, description, hours and the mentor's email address.
- Optional evidence upload (PDF, PNG, JPG or WEBP) attached to an entry.
- Upload of the WIL documents for each placement: the MOU, Commencement Letter and Code of Conduct once, and an Attendance Register every semester (PDF, PNG or JPG, up to 10 MB).
- Submission for verification through a one-time link that expires after seven days.
- A mentor verification page that needs no account, where the mentor approves or rejects the entry.
- Entry statuses (Draft, Pending verification, Verified, Rejected) and resubmission of rejected entries.
- Verified hours shown against the required hours of each placement.
- A placement is marked WIL completed when its required hours are verified, and is then locked.

- A coordinator and lecturer dashboard showing each student's status, verified hours, entries, pending verifications and missing documents, with a profile page for each student.
- A relational database (SQLite) for users, WIL placements, entries, verification links and document records, with timestamps for each action.

#### Excluded from the project

- Mentor accounts and a mentor dashboard (mentors verify through the link only).
- Written feedback comments from mentors (the mentor approves or rejects only).
- A separate lecturer dashboard (lecturers use the coordinator dashboard).
- Automatic email or SMS notifications; students send the link themselves.
- Editing or deleting entries after they are saved.
- Reports, data export and analytics.
- Integration with CTU student or learning management systems.
- A native mobile application.
- Use of real student or staff data; all testing uses synthetic data until written consent is obtained (see the Ethical Clearance Confirmation).

### 1.8 Significance of the Study

The TrackWIL project is expected to provide benefits to several groups involved in the WIL process.

#### Students

TrackWIL makes it easier for students to manage their daily WIL documentation. Instead of sending documents through email and waiting for them to be returned, students have one place to record their work, request verification and check the status of each entry. They can also see how many verified hours they have completed against the required total.

#### Workplace Mentors

Mentors can verify a student's work in a few clicks from a link, without creating or remembering an account. Each decision is recorded with a date and time, which provides a clear record of completed mentor approvals.

#### WIL Coordinators

Coordinators have a central view of student submissions and verification statuses. Instead of manually checking documents or asking students for updates, they can use the dashboard to identify students with outstanding verifications or no recent verified work. This can reduce the administrative workload and make monitoring more efficient.

#### Lecturers

Lecturers can monitor student progress from the same dashboard. They can identify students who are up to date and students who are falling behind, which allows them to identify problems earlier and provide support where necessary.

#### Educational Institutions

TrackWIL can help educational institutions improve the organisation and management of WIL documentation. Centralising information reduces dependence on emails, spreadsheets and separate documents and provides a clearer record of student WIL progress.

### Software Engineering Field

The project demonstrates how software engineering can be used to solve a real-world problem in education and workplace training. It applies requirements analysis, database design, authentication, role-based access, secure file handling, token-based verification, version control and software testing.

## 1.9 Chapter Summary

This chapter introduced the TrackWIL project and the challenges of managing Work-Integrated Learning documentation through email, paper and separate files. Students must complete daily documents, have them signed by workplace mentors and wait for them to be returned, which makes it difficult to track submissions and identify outstanding work.

The chapter presented the research aim, five objectives and five research questions, each linked to an objective, that guide the development of TrackWIL. The proposed solution is a web application in which students record daily entries and evidence, mentors verify entries through a one-time link without an account, and coordinators and lecturers monitor progress from a shared dashboard.

The scope defined what is included in the project and what is excluded, and the significance of the study explained how TrackWIL can benefit students, workplace mentors, coordinators, lecturers and educational institutions. Chapter 2 reviews related research, similar systems and technologies.

# CHAPTER 2 - LITERATURE REVIEW

## 2.1 Introduction

This chapter reviews research, existing software and relevant technologies related to Work-Integrated Learning (WIL) documentation and progress monitoring. The review considers how digital systems support students, workplace mentors, coordinators and lecturers, and identifies features that can inform the design of TrackWIL.

The chapter compares three existing systems: InPlace, Sonia Online and PebblePad/Mahara-style ePortfolio tools, against the project's needs. It also considers web applications, relational databases, authentication and role-based access, and secure document upload. The findings are used to define a practical research gap and to justify TrackWIL's technical direction.

## 2.2 Existing Research

Work-Integrated Learning connects academic learning with practical workplace experience. Technology can support WIL in different ways: it may deliver learning activities, or it may provide administrative support for placements, communication, evidence collection and assessment. Schuster and Glavas (2017) explain that electronic WIL covers a range of technology uses rather than one single model. For TrackWIL, the administrative role is especially relevant because the project focuses on daily activity records, evidence submission, mentor verification and progress monitoring.

Effective placement supervision depends on communication and coordination between students and supervisors. Hains-Wesson et al. (2026), in a systematic literature review of placement supervision, examine student and supervisor perspectives and identify patterns and gaps in the literature. This supports the need to consider both sides of the submission and verification process rather than designing a system only for students or academic staff.

Industry systems show that placement management commonly involves centralised records, user portals, communication, compliance and reporting. InPlace describes tools for central placement visibility, student portals, time sheets, communications, compliance and analytics (InPlace Software, n.d.). Sonia Online describes itself as a placement management system containing placement information and paperwork for students and host organisations (Sonia Online, n.d.). These examples show that digital coordination is an established approach, although their exact configuration and available functions depend on the institution using them.

Portfolio platforms also provide relevant ideas for collecting and reviewing evidence. Mahara allows learners to upload evidence, reflect on experiences, share portfolios with selected people and receive feedback (Mahara Project, n.d.). PebblePad's ATLAS assessment area supports assessors in accessing and reviewing learners' work (PebblePad, 2026). These features are relevant to TrackWIL's evidence and mentor-review workflow, but a portfolio or assessment platform is not necessarily designed around a specific institution's daily WIL forms and status-tracking requirements.

Overall, the literature and industry examples support a centralised digital workflow, clear communication, evidence management and timely verification. However, TrackWIL must still be evaluated against its specific project requirements, including daily submissions, mentor approval or rejection, role-specific views and a visible list of outstanding work. Product websites describe vendor-provided capabilities; they should not be treated as independent proof that a system will improve outcomes in every setting.

### 2.3 Existing or Similar Systems

Three relevant systems were reviewed: InPlace, Sonia Online and Mahara/PebblePad. The comparison below is based on publicly available product information, not on hands-on testing. Features may vary by institution, subscription or local configuration.

**System 1: InPlace.** InPlace is designed for higher-education placement management. Its stated features include centralised placement management, student portals, timesheets and schedules, communications, compliance management, integrations and reporting (InPlace Software, n.d.). Its strength is broad administration and visibility across placements. A limitation for this comparison is that its public description emphasises placement allocation, compliance and programme-level management; it does not establish that every TrackWIL-specific daily logbook or mentor-signing workflow is available in a particular deployment. Lesson for TrackWIL: centralise records and provide role-appropriate views, but prioritise the daily documentation workflow required by the project.

**System 2: Sonia Online.** Sonia Online is presented as a student placement management system for students and host organisations, with placement information and paperwork available through the system (Sonia Online, n.d.). Its strength is that it directly targets placement coordination and provides access for relevant placement participants. Its limitations cannot be fully assessed from the public landing page alone, and access or features may differ between institutions. Lesson for TrackWIL: make student and mentor access straightforward and keep placement paperwork in one organised location.

**System 3: Mahara / PebblePad.** Mahara supports evidence-based portfolios, reflection, sharing and feedback (Mahara Project, n.d.). PebblePad's ATLAS area supports access to learner submissions and assessment (PebblePad, 2026). Their strengths are evidence collection, sharing and review. A limitation relative to TrackWIL is that portfolio and assessment workflows may not, by themselves, provide the project's daily WIL status controls, outstanding-work monitoring and coordinator view. Lesson for TrackWIL: connect each uploaded evidence item to a dated activity, a verification status and a progress overview.

Table 5: Comparison of similar systems against TrackWIL's needs

| Criterion | InPlace | Sonia Online | Mahara / PebblePad | TrackWIL (this project) |
|---|---|---|---|---|
| Main purpose | Placement management | Placement information and paperwork | Evidence portfolios and assessment | Daily WIL logbook and verification |
| Daily activity records | Timesheets (stated) | Not clear from public information | Portfolio pages | Dated entries with hours |
| Evidence upload | Not clear from public information | Paperwork in the system | Yes | Optional PDF or image per entry |
| Workplace verification | Not clear from public information | Host organisation access (stated) | Assessor review | One-time link, no mentor account |
| Staff monitoring | Reporting and analytics | Placement information | Assessor reporting | Dashboard: On track, Awaiting, Overdue |
| Main limitation for this project | Broad and institution-configured | Features vary by institution | Not built around daily status tracking | Prototype scope; no notifications |

**Comparative summary.** InPlace is strongest as a broad placement-management platform; Sonia Online is a directly relevant placement-information and paperwork system; Mahara and PebblePad provide useful portfolio and assessment patterns. TrackWIL draws on these ideas but focuses its scope on daily WIL activity records, supporting evidence, mentor verification and monitoring by coordinators and lecturers. This is a project-specific design rationale, not a claim that the existing products lack all such features.

### 2.4 Technology Review

**Web-based application.** A browser-based application is suitable because students, mentors, coordinators and lecturers may access the system from different locations and devices, and a mentor can open a verification link on any device without installing software. A shared application can present role-specific views while keeping records in one place. Its disadvantages include dependence on network availability, the need to support different screen sizes and the need for secure deployment and maintenance.

**Framework.** Next.js is a React framework in which pages and server-side logic live in one project, and Server Actions let forms call server functions directly without a separate API layer (Vercel, n.d.). This suits a small team building a working prototype in a short time, because one language (TypeScript) is used for the user interface, the business rules and data access. A separate front end and back end (for example, React with an Express API) would give more flexibility for very large systems, but would add a second project to build, test and deploy.

**Database.** A relational database is appropriate for the structured relationships between users, logbook entries and verification links. Primary and foreign keys and CHECK constraints keep the data consistent and make it simple to count verified hours and outstanding verifications. SQLite was chosen for the prototype because it is a self-contained, serverless SQL database engine that stores the whole database in a single file (SQLite, n.d.), and Node.js includes a built-in SQLite module, so no database server or extra package has to be installed (OpenJS Foundation, n.d.). This makes the application easy to run on every team member's computer and for the live demonstration. A client-server database such as PostgreSQL would be preferred for a multi-user production deployment; because the schema uses standard SQL, it can be migrated later.

**Authentication and role-based access.** TrackWIL handles student records and workplace evidence, so users should authenticate before accessing protected information. OWASP recommends storing passwords only as salted hashes produced by a slow, purpose-built function and never in plain text (OWASP Foundation, n.d.a). Authorisation should restrict users to the records and actions needed for their roles, and checks should be enforced on the server, not only by hiding interface elements (OWASP Foundation, n.d.b). For workplace mentors, who are external and verify only occasionally, a single-use, time-limited link with a long random token is a simpler alternative to creating accounts, provided the token cannot be guessed and stops working after use or expiry.

**Document upload and storage.** Evidence upload is a core feature, but uploaded files create security and storage risks. OWASP recommends allowing only necessary file types, limiting file size, generating safe stored file names that users cannot control, and ensuring only authorised users can download uploads (OWASP Foundation, n.d.c). TrackWIL therefore accepts only PDF and image files, stores them under random names outside the public web folder, and serves them only to the owning student and to staff.

**Cloud hosting.** Cloud hosting can make a web application accessible over the internet without the project team maintaining physical infrastructure. NIST describes cloud computing as on-demand network access to a shared pool of configurable computing resources (Mell and Grance, 2011). For this project the prototype runs locally on synthetic data; cloud hosting is a future option that would require attention to cost, availability, backups, privacy and the location where student information is stored.

**Usability and testing.** The application should make submission status easy to understand and minimise unnecessary steps for students and mentors. Testing should cover successful and unsuccessful sign-in, role permissions, input and upload validation, submission and verification status changes, link expiry and reuse, and the monitoring dashboard. Claims of improved efficiency should be based on testing or user feedback rather than assumed in advance.

### 2.5 Research Gap

Existing placement-management systems and portfolio platforms already provide useful functions such as centralised placement information, paperwork, evidence sharing, assessment, communication and reporting. Therefore, the research gap is not that no digital WIL or placement tools exist.

The project-specific gap addressed by TrackWIL is the need for a focused, lightweight workflow aligned with the requirements described in Chapter 1: students record daily WIL activities and upload evidence; workplace mentors, who are outside the institution, verify each entry without having to create an account; and coordinators and lecturers see each student's verification status and outstanding work in one view.

Public product descriptions do not establish whether one existing platform, in the configuration available to the intended users, matches all these requirements in a simple workflow. The project therefore designs and tests a solution against these requirements rather than claiming that existing products cannot support them. The proposed contribution is a working prototype tailored to the stated use case, with a shared record of entries and their verification status.

The evaluation investigates whether students can submit and have entries verified more simply and whether staff can identify outstanding work more easily. This gap and evaluation focus align with the aim and objectives in Chapter 1.

### 2.6 Conceptual / Technical Direction

TrackWIL follows a centralised, role-based workflow. A student creates a dated logbook entry, records the hours worked, names the workplace mentor and may attach evidence. The student submits the entry, and the system issues a one-time verification link. The mentor opens the link and approves or rejects the entry. The system records the status and timestamps so the student can see the outcome, while coordinators and lecturers use the dashboard to identify students who are on track, awaiting verification or overdue.

The main conceptual entities are User (student, coordinator or lecturer), Logbook Entry and Verification Token. Each entry belongs to one student, and each verification token belongs to one entry, so that every approval or rejection can be traced to the link that produced it. Storing the status on the entry itself avoids multiple conflicting copies of the same submission.

Technical priorities are usability, data integrity, access control, safe file handling, maintainability and testability. Based on the technology review, the team selected Next.js with TypeScript for the application, Tailwind CSS for styling and SQLite for the database. Regardless of the stack, the system validates input and enforces permissions on the server, restricts file uploads and is tested for each role's workflow.

### 2.7 Chapter Summary

This chapter reviewed research on electronic WIL and placement supervision, and compared InPlace, Sonia Online and portfolio and assessment tools such as Mahara and PebblePad. The review found useful patterns in centralised placement information, evidence collection, submission review, compliance and reporting. It also reviewed the technologies used to build TrackWIL: a Next.js web application, an SQLite relational database, salted password hashing with role-based access, link-based mentor verification and restricted file uploads.

The research gap is a focused workflow that matches TrackWIL's needs for daily activity logging, evidence submission, account-free mentor verification and progress monitoring. These findings inform the requirements in Chapter 3 and the design in Chapter 4. TrackWIL's effectiveness must be established through testing and evaluation rather than assumed.

# CHAPTER 3 - REQUIREMENTS & METHODOLOGY

## 3.1 Introduction

This chapter explains what TrackWIL must do, how those requirements were identified and how the system was developed. TrackWIL is a web-based system that replaces the paper, PDF and email process for Work-Integrated Learning (WIL) logbooks.

In the current process students complete and sign daily WIL documents, send them to workplace mentors for verification and wait for the signed documents to be returned. This manual process can lead to delays, lost documents and difficulty tracking student progress. TrackWIL provides a central platform where students record daily work, workplace mentors verify entries through a secure link, and coordinators and lecturers monitor student progress.

The chapter describes the stakeholders, the functional and non-functional requirements, the requirements-gathering process, the development methodology, the use cases and user stories, the system architecture, and the ethical, privacy and security considerations. The complete requirements are specified in the Software Requirements Specification (SRS), and Chapter 5 traces every requirement to its test cases.

## 3.2 Stakeholders and Users

Stakeholders are people or organisations affected by TrackWIL, while users are the people who interact directly with the system.

Table 6: Stakeholders, their needs and how they use TrackWIL

| Stakeholder | Main need | How they use TrackWIL | Access |
|---|---|---|---|
| Student (primary user) | Record daily work and get it verified without chasing paper or email | Signs up, adds WIL placements, creates logbook entries, uploads evidence and WIL documents, submits entries for verification, tracks verified hours | Account (student role) |
| Workplace mentor (primary user) | Confirm a student's work quickly, without another account to manage | Opens the link sent by the student and approves or rejects the entry | No account; one-time link valid for 7 days |
| WIL coordinator (primary user) | Know which students are behind or waiting on a mentor | Monitors every student's status, verified hours, pending verifications and documents, and opens each student's profile | Account (coordinator role) |
| Lecturer (primary user) | Identify students falling behind so that support can be given | Uses the same monitoring dashboard and student profiles as the coordinator | Account (lecturer role) |
| Training institution (CTU) | Reliable WIL records and protection of personal information | Benefits from central records; sets data and consent rules | None |
| Development team | Clear, testable requirements | Builds, tests and maintains the system | Repository and development data |
| Project supervisor (client) | A project that meets its objectives and the assessment rubric | Reviews weekly submissions and gives feedback | Repository and demonstration |

### 3.3 Functional Requirements

Each functional requirement has an ID, a description and a priority. **High** means the requirement is part of the core product that must work for the demonstration; **Medium** means it adds important value but the core workflow can be shown without it. Each requirement is testable and traced to test cases in Chapter 5.

Table 7: Functional requirements

| ID | Requirement | Priority |
|---|---|---|
| FR-01 | The system must allow a registered user to sign in with an email address and password and must send them to the dashboard for their role. | High |
| FR-02 | The system must allow a student to create an account with a CTU student email address (eight-digit student number @ctucareer.co.za) and a password of at least 8 characters with an uppercase letter, a lowercase letter, a number and a special character. | Medium |
| FR-03 | The system must restrict each page and action to the roles allowed to use it: the student area to students, the monitoring dashboard to coordinators and lecturers, and the verification page to holders of a valid link. | High |
| FR-04 | A student must be able to create a daily logbook entry with a work date, a description, the hours worked (more than 0 and at most 24) and the workplace mentor's email address. | High |
| FR-05 | A student must be able to attach one optional evidence file (PDF, PNG, JPG or WEBP) to an entry. | Medium |
| FR-06 | A student must be able to submit a Draft or Rejected entry for verification; the system must create a unique link that expires after 7 days and set the entry to Pending verification. | High |
| FR-07 | A workplace mentor must be able to open the link without an account, see the student's name, date, hours and description, and approve or reject the entry. | High |
| FR-08 | A verification link must work only once and only before it expires; an invalid, used or expired link must show a message and must not change the entry. | High |
| FR-09 | The system must show each entry's status as Draft, Pending verification, Verified or Rejected. | High |
| FR-10 | A student must be able to view the entries of each of their WIL placements, newest first, and each placement's verified hours against its required hours. | High |
| FR-11 | Coordinators and lecturers must be able to see every student with their status for their current placement (On track, Awaiting verification, Overdue, WIL completed or No placement), verified hours, number of entries, number of pending verifications and number of missing documents. A student is Overdue when their current placement has no verified entry in the last 14 days (counted from the placement start if nothing is verified yet). | High |
| FR-12 | The system must automatically record when each entry is created, updated and verified, and when each link is used and with what decision. | Medium |
| FR-13 | Evidence files must be downloadable only by the student who uploaded them, coordinators and lecturers. | High |
| FR-14 | A signed-in user must be able to sign out, which ends their session. | High |

| ID | Requirement | Priority |
|---|---|---|
| FR-15 | A student must be able to add one or more WIL placements, each with a title, the host company, a start date and the number of required hours (more than 0 and at most 5 000), and to edit a placement until it is completed; every logbook entry must belong to one of the student's own placements. | High |
| FR-16 | A student must be able to upload the WIL documents for each placement as PDF, PNG or JPG files of up to 10 MB: the Memorandum of Understanding (MOU), Commencement Letter and Code of Conduct once per placement and an Attendance Register once per semester. Uploading again must replace the previous file, and documents must be downloadable only by the student, coordinators and lecturers. | High |
| FR-17 | Coordinators and lecturers must be able to open a student's profile from the dashboard, showing each placement with its verified hours, its documents (view only) and its full logbook, including mentor emails and evidence. | High |
| FR-18 | A placement must be marked WIL completed automatically when its verified hours reach its required hours. A completed placement must be locked: no new entries, submissions, document changes or edits are allowed, and the student must add a new placement to continue. | High |

FR-15 to FR-18 were added after the Week 4 submission, when the team reviewed the whole WIL process: students complete more than one WIL placement, each with its own required hours, and hand in a Memorandum of Understanding, a Commencement Letter, a Code of Conduct and an attendance register for every semester, which coordinators need to check alongside the logbook.

During development the team reviewed the original requirements against the time available. The following requirements from the Week 3 draft were deferred and are recorded as future work, so that the core workflow could be completed and tested to a good standard.

Table 8: Requirements deferred to future versions

| Deferred requirement | Reason | Current behaviour |
|---|---|---|
| Mentor accounts and a mentor dashboard | Mentors are external, verify occasionally and should not need another account | Mentors verify through the one-time link |
| Written mentor feedback on rejected entries | Time; the core decision (approve or reject) was prioritised | The mentor approves or rejects; the student sees the Rejected status |
| Mentor access to evidence files | Evidence is protected by sign-in; a link-based download needs its own security design | Evidence is visible to the student and to staff |
| Separate lecturer dashboard | Coordinators and lecturers need the same information | Lecturers use the coordinator dashboard |
| Automatic email notifications | Requires an email service and consent for contact details | The student copies the link and sends it to the mentor |
| Reports and data export | Stretch goal in the proposal | Not implemented |
| Coordinator review of uploaded documents (accept, or ask for a new upload) | Time; uploading and viewing the documents were prioritised | Coordinators and lecturers can view every document |

### 3.4 Non-Functional Requirements

Each non-functional requirement states how it will be measured, so that it can be tested rather than judged by opinion.

Table 9: Non-functional requirements

| ID | Category | Requirement | How it is measured |
|---|---|---|---|
| NFR-01 | Usability | A first-time student can create and submit an entry for verification in under 3 minutes without help. | Timed task with test users (Chapter 5) |
| NFR-02 | Performance | Each page loads and each action completes within 2 seconds on the test computer with the demonstration data. | Browser timing during system tests |
| NFR-03 | Security | Passwords are stored only as salted scrypt hashes; the session cookie is HttpOnly; verification tokens contain at least 128 bits of randomness; all role checks run on the server. | Code review and security test cases |
| NFR-04 | Reliability and data integrity | Database constraints reject invalid data (unknown status, hours outside 0–24, entries without a placement), and multi-step updates run in a transaction so that no partial update is saved. | Database constraint tests |
| NFR-05 | Maintainability | The code is written in TypeScript in separate layers (pages, actions, repository, database), passes the linter, and the README explains how to run it in no more than five commands. | Lint run and README walkthrough |
| NFR-06 | Compatibility | The application works in current versions of Google Chrome and Microsoft Edge and is usable on a 375-pixel-wide mobile screen and a 1280-pixel desktop screen. | Cross-browser and responsive tests |
| NFR-07 | Privacy | Only the data needed for WIL tracking is stored (name, email, placements, entries, evidence and WIL documents); testing and demonstrations use synthetic data only. | Data review against the Ethical Clearance Confirmation |

### 3.5 Requirements Gathering

The requirements were gathered and refined through the methods below. Interviews, questionnaires and usability tests with staff and students are planned, but, as stated in the Ethical Clearance Confirmation, no personal information may be collected before written campus permission is granted. Until then, the team relied on document analysis, comparison with existing systems, supervisor feedback and review of the working prototype, and all testing uses synthetic data.

Table 10: Requirements-gathering methods and evidence

| Method | What was done | Evidence | Effect on requirements |
|---|---|---|---|
| Scenario and document analysis | Analysed the current paper, PDF and email logbook process and the module brief | Project proposal; Chapter 1 | Defined the problem, the four user groups and the core workflow |
| Review of similar systems | Compared InPlace, Sonia Online and Mahara/PebblePad | Chapter 2; Literature Review Matrix | Mentor verification without an account; status-based monitoring |
| Supervisor (client) feedback | The proposal was reviewed by the supervisor and revised on 7 October 2026 | Topic Acceptance form; revised proposal in Git | Narrowed the scope to three core features and moved analytics, exports and audit logs to stretch goals |
| Team workshops | Weekly group meetings to agree and prioritise requirements | Attendance register; Weekly Progress Report | High and Medium priorities; deferred requirements |
| Prototype review | The Sprint 1 prototype was demonstrated to the team and checked against the requirements | Git history (12–15 October 2026) | Added sign-up rules, the 24-hour limit and lecturer access |
| Review of the WIL paperwork | Reviewed the documents a student hands in during WIL and the fact that students complete more than one WIL placement | Pull requests #11 to #15 in Git (26 October 2026) | Added FR-15 to FR-18: placements, WIL documents, a student profile for staff and WIL completion |
| Planned: questionnaire and usability test | Short task-based test and questionnaire with volunteer students and staff once permission is granted | Ethical Clearance Confirmation | Will evaluate NFR-01 and overall usefulness (Chapter 5) |

The requirements were then reviewed to make sure that each one is clear, testable and linked to a research objective, and were recorded in the SRS.

### 3.6 Development Methodology

TrackWIL was developed with a hybrid Agile approach based on Scrum, using one-week sprints that end on each Friday submission date (Schwaber and Sutherland, 2020). Each sprint followed the same cycle: plan the sprint backlog, design, build, test, and review the result with the team and the supervisor's feedback.

Table 11: Sprint plan

| Sprint | Dates | Goal | Main deliverables |
|---|---|---|---|
| Sprint 0 | 2–8 October 2026 | Proposal, scope and requirements | Approved topic; core and stretch scope; ethics form |
| Sprint 1 | 9–15 October 2026 | Logbook core | Chapters 1–2; Next.js prototype with sign-in, logbook entries, evidence upload and hour progress |
| Sprint 2 | 16–22 October 2026 | Verification and monitoring | Chapters 3–4; SRS; token verification; coordinator dashboard; sign-up; SQLite database |
| Sprint 3 | 23–29 October 2026 | Test, evaluate and submit | Chapters 5–6; testing workbooks; defect fixes; WIL placements, documents, student profiles and WIL completion; presentation and live demonstration |

### Justification for Agile

Agile suits TrackWIL for four reasons. First, the system has three features that can be built and tested separately (logbook, verification and dashboard), so each sprint can deliver something that works. Second, the requirements changed during the project: supervisor feedback narrowed the scope, and the prototype review added sign-up rules and lecturer access. An iterative approach accepts such changes, as described in the Agile Manifesto's preference for "responding to change over following a plan" (Beck et al., 2001). Third, the weekly submissions act as natural sprint reviews with the supervisor. Fourth, the four team members can work on different features in parallel on separate Git branches.

### Alternative considered: Waterfall

The Waterfall model completes each phase (requirements, design, implementation, testing) before the next starts (Sommerville, 2016). It works well when requirements are stable and well understood. It was rejected because TrackWIL's requirements were still changing, the project had only four weeks, and Waterfall would have left all testing until the end, when there would be no time to fix defects. With Agile, each feature was tested in the sprint in which it was built.

## 3.7 Use Cases / User Stories

The use cases are shown in the use case diagram in Chapter 4. Each user story has acceptance criteria that are used as the basis for the test cases in Chapter 5.

Table 12: User stories and acceptance criteria

| ID | User story | Acceptance criteria | FRs |
|---|---|---|---|
| US-01 | As a student, I want to sign up and sign in securely so that I can access my own logbook. | Valid CTU email and strong password create an account; wrong password shows "Invalid email or password." | FR-01, FR-02, FR-14 |
| US-02 | As a student, I want to record a day's work so that my workplace activities are documented in one place. | Date, description and hours (0–24) are required; the saved entry appears as Draft. | FR-04, FR-09, FR-12 |
| US-03 | As a student, I want to attach evidence so that my work can be checked. | PDF or image accepted; other file types rejected; only the student and staff can open the file. | FR-05, FR-13 |

| ID | User story | Acceptance criteria | FRs |
|---|---|---|---|
| US-04 | As a student, I want to send my entry to my mentor for verification so that my hours count. | A link valid for 7 days is shown; the entry becomes Pending verification. | FR-06 |
| US-05 | As a workplace mentor, I want to approve or reject an entry from a link so that I do not need an account. | The link shows the entry; Approve sets Verified and Reject sets Rejected; the link cannot be used again. | FR-07, FR-08 |
| US-06 | As a student, I want to see the status of my entries and my verified hours so that I know what is outstanding. | Statuses are shown on the logbook; verified hours are shown against the required hours; a rejected entry can be resubmitted. | FR-09, FR-10 |
| US-07 | As a coordinator, I want to see which students are on track, awaiting verification or overdue so that I can follow up. | The dashboard lists every student with status, verified hours, entries and pending count. | FR-11 |
| US-08 | As a lecturer, I want to view student progress so that I can support students who are falling behind. | A lecturer signs in to the same dashboard and cannot open the student area. | FR-03, FR-11 |
| US-09 | As a student, I want to add each of my WIL placements with its required hours so that my hours are tracked per placement. | A placement needs a title, company, start date and required hours (more than 0); entries are added to one of my placements; a completed placement is locked. | FR-10, FR-15, FR-18 |
| US-10 | As a student, I want to upload my WIL documents so that they are kept with my placement. | The MOU, Commencement Letter and Code of Conduct are uploaded once and an attendance register every semester; only PDF, PNG and JPG files up to 10 MB are accepted; uploading again replaces the file. | FR-16 |
| US-11 | As a coordinator or lecturer, I want to open a student's profile so that I can check their hours, documents and logbook in one place. | Selecting a student on the dashboard shows each placement with its hours, documents and entries; students cannot open this page. | FR-03, FR-17 |

#### Main use case: verify a logbook entry (UC-05)

- **Actors:** Student (initiates), workplace mentor (verifies).
- **Precondition:** The student has a Draft or Rejected entry with a mentor email address.
- **Main flow:** (1) The student submits the entry for verification. (2) The system creates a one-time link valid for 7 days and sets the entry to Pending verification. (3) The student sends the link to the mentor. (4) The mentor opens the link and reviews the entry. (5) The mentor selects Approve or Reject. (6) The system marks the link as used and sets the entry to Verified or Rejected.
- **Alternative flow:** If the link is invalid, already used or expired, the system shows a message and does not change the entry.
- **Postcondition:** The entry's status and verification time are stored, and the coordinator dashboard reflects the change.

### 3.8 System Architecture

TrackWIL uses a three-layer architecture inside a single Next.js application. The detailed architecture diagram is presented in Chapter 4.

#### Presentation Layer

The presentation layer is the part of the system that users interact with through a web browser. It provides the sign-in and sign-up pages, the student's placements, logbooks and documents, the mentor verification page, and the coordinator and lecturer dashboard with student profiles. It is built with React components and styled with Tailwind CSS.

#### Application Layer

The application layer handles the business rules through Next.js Server Actions: authentication, role checks, input validation, creating placements and entries, accepting document uploads, issuing verification links, recording mentor decisions and locking completed placements. This layer makes sure that users can perform only the actions allowed for their role; for example, a student cannot open the coordinator dashboard and cannot verify their own entry, because verification requires the secret link.

#### Data Layer

The data layer stores user accounts, WIL placements, logbook entries, verification links and document records in an SQLite relational database, and evidence files and WIL documents in a protected folder on the server. A repository module is the only part of the code that runs SQL, which keeps the database separate from the rest of the application.

#### Communication Between the Layers

When a student submits an entry, the presentation layer calls a Server Action in the application layer. The action checks that the user is signed in and owns the entry, validates the request and saves the change through the data layer, then returns the result so that the page can show the new status. The mentor's approval follows the same path, starting from the verification link.

### 3.9 Ethical, Privacy and Security Considerations

#### User Authentication and Access Control

TrackWIL requires students, coordinators and lecturers to sign in. Passwords are stored only as salted hashes (OWASP Foundation, n.d.a). Role checks run on the server for every protected page and action, so that hiding a button is never the only protection (OWASP Foundation, n.d.b). Mentors use a long random link that works only once and only for seven days.

#### Data Accuracy and Accountability

The system records when each entry is created and verified and which link was used for each decision, so the history of an entry is clear. Verification is always performed by the mentor through the link; entries are never approved automatically, and a student cannot change an entry's status themselves.

#### Consent and Responsible Data Use

No personal information about real staff or students is collected until written campus permission is granted. Participation in any later questionnaire or usability test will be voluntary, and participants will be told the purpose of the data collection and who can see it. Until then, all development, testing and demonstrations use synthetic data.

### Privacy and Protection of Personal Information

The system collects only the information necessary for WIL tracking. Personal information must be handled in line with institutional policy and the Protection of Personal Information Act (POPIA) (Republic of South Africa, 2013). Evidence files and WIL documents are stored under random names and can be opened only by the student who uploaded them and by staff.

### Security Risks Identified

The team identified security risks during design and testing, including the risk of uploading harmful files (reduced by allowing only PDF and image types, checking that each document's content matches its type, limiting files to 10 MB and storing files under random names) and the risk of a session cookie being reused or altered. These risks and their treatment are recorded in the Risk Register and evaluated in Chapter 5.

## 3.10 Chapter Summary

This chapter identified TrackWIL's stakeholders and specified eighteen prioritised functional requirements and seven measurable non-functional requirements, together with the requirements that were deferred to future versions. It explained how the requirements were gathered within the limits of the ethical clearance, justified the use of a hybrid Agile (Scrum) approach over Waterfall, and presented eleven user stories with acceptance criteria and the main verification use case. Finally, it described the three-layer architecture and the ethical, privacy and security considerations. Chapter 4 presents the detailed design and implementation.

# CHAPTER 4 - SYSTEM DESIGN & DEVELOPMENT

## 4.1 Introduction

This chapter shows how the requirements in Chapter 3 were turned into a design and a working application. It presents the system architecture, the UML diagrams (use case, class, activity and sequence), the database design, the user interface, the development environment, the implementation of the key modules, and the version-control evidence from the team's GitHub repository.

TrackWIL replaces the PDF and email logbook with a web application. Students log workplace hours and evidence for each WIL placement and upload their WIL documents, the workplace mentor verifies each entry through a one-time link without needing an account, and the WIL coordinator and lecturers can see who is on track, who is waiting on a mentor, who is overdue and who has completed their WIL. The source code is in the folder 05_Source_Code/trackwil of the repository.

## 4.2 System Architecture

TrackWIL is a single Next.js application that runs in one Node.js process. The browser communicates with this process over HTTP (HTTPS when hosted). The process serves the React pages, runs the business rules in Server Actions, and reads and writes the SQLite database. Uploaded evidence files and WIL documents are stored on the server's disk. Figure 1 shows the architecture.

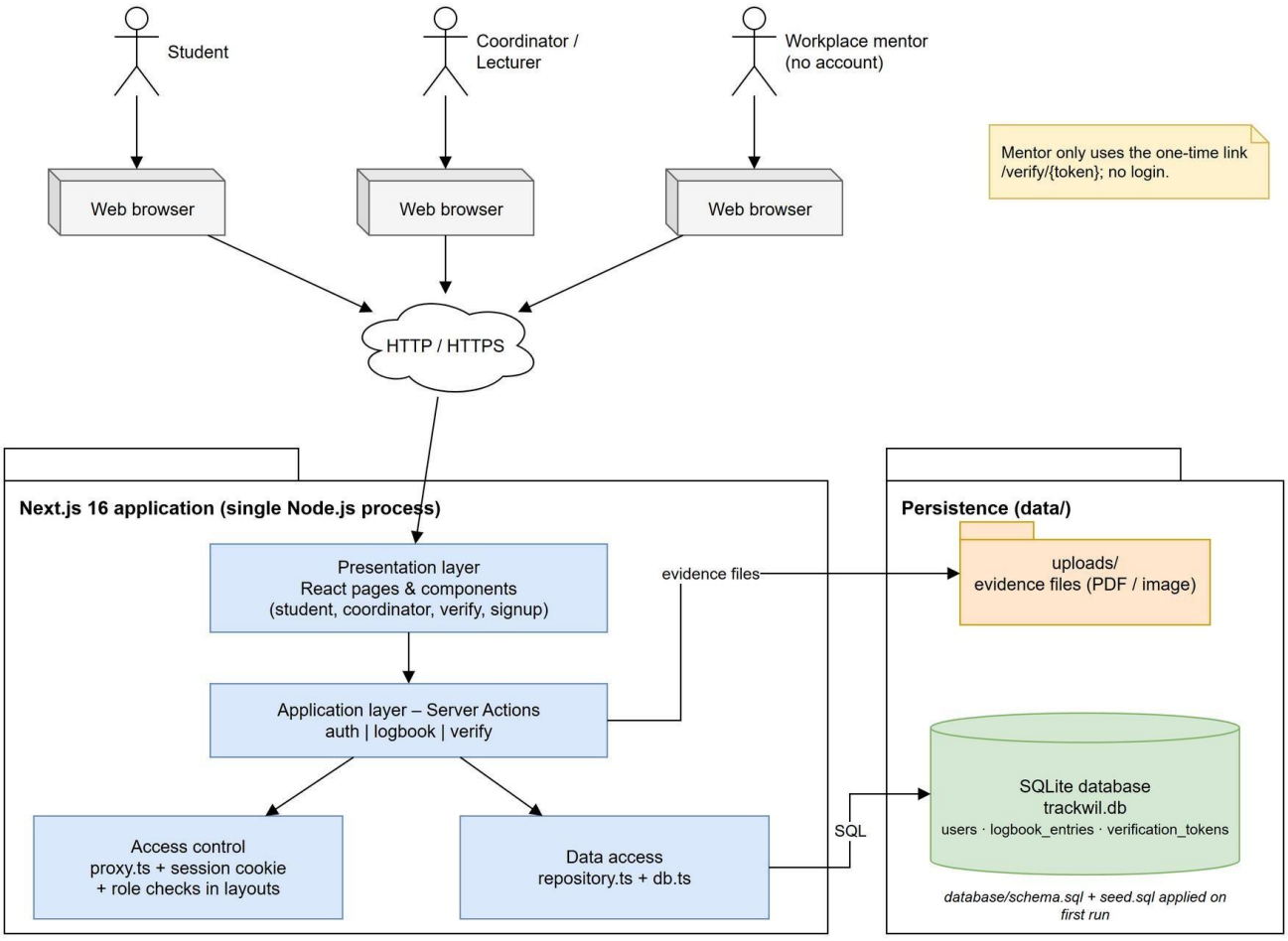



Figure 1: TrackWIL system architecture

The architecture has three logical layers. The presentation layer contains the pages and components each user sees. The application layer contains the Server Actions that validate input, check the user's role and apply the business rules. The data layer contains the repository and database modules; the repository is the only code that runs SQL, so the rest of the application never depends on how the data is stored. This separation is what allowed the team to replace the first prototype's JSON files with SQLite by changing only the data layer.

Table 13: Main components and their responsibilities

| Component | Location in code | Responsibility | Requirements |
|---|---|---|---|
| Sign-in and sign-up pages | src/app/page.tsx, src/app/signup | Collect credentials; show validation errors | FR-01, FR-02 |
| Student area | src/app/student | Placements, placement logbook and documents, new entry form, entry details, verification link | FR-04 to FR-06, FR-09, FR-10, FR-15, FR-16, FR-18 |
| Mentor verification page | src/app/verify/[token] | Show the entry and Approve / Reject buttons without sign-in | FR-07, FR-08 |
| Coordinator dashboard | src/app/coordinator | Table of all students with status, hours and missing documents, and a profile page for each student (also used by lecturers) | FR-11, FR-17 |
| Server Actions | src/app/actions (auth, placements, logbook, documents, verify) | Validation, role checks and business rules | FR-01 to FR-08, FR-14 to FR-16, FR-18 |
| Access control | src/proxy.ts, layouts and every protected page (requireRole in src/lib/session.ts) | Redirect signed-out users; each page checks the role before loading any data | FR-03 |
| Evidence route | src/app/api/evidence/[fileName] | Serve evidence only to the owner and staff | FR-13 |
| Document route | src/app/api/documents/[id] | Serve WIL documents only to the student, coordinators and lecturers | FR-16 |
| Repository and database | src/lib/repository.ts, src/lib/db.ts | All SQL queries and transactions; schema and seed on first run | FR-12, NFR-04 |
| Metrics | src/lib/logbook-metrics.ts | Verified hours, placement completion and each student's status | FR-10, FR-11, FR-18 |
| Document rules | src/lib/documents.ts | Document types, file checks and the semesters that need an attendance register | FR-16 |

**Data flow example (submitting an entry).** The student clicks Submit for verification. The browser calls the submitEntryForVerificationAction Server Action, which loads the signed-in user, checks that the student owns the entry and that it is a Draft or Rejected entry with a mentor email, creates a random token and asks the repository to save the token and change the entry's status in one transaction. The page then shows the new status and the link.

**Technology stack justification.** The table below summarises the technologies chosen and why. The comparison of alternatives is discussed in Chapter 2, Section 2.4.

Table 14: Technology stack and justification

| Layer | Technology | Why it was chosen |
|---|---|---|
| Front end | React 19 with Tailwind CSS 4 | Component-based pages; utility classes give a consistent, responsive layout quickly |
| Application | Next.js 16 (App Router, Server Actions) with TypeScript 5 | One project and one language for pages and server logic; type checking catches errors early |
| Database | SQLite through Node.js's built-in node:sqlite module | Real relational database with keys and constraints, but no server to install; one file that is easy to reset for demonstrations |
| Security | Node.js crypto (scrypt, randomBytes) | Standard, well-tested functions for password hashing and unguessable tokens; no extra dependencies |
| Version control | Git and GitHub | Branches, pull requests and a full history of each member's work |

### 4.3 UML Design

Four UML diagrams describe the system from different views: the use case diagram (what each user can do), the class diagram (the main data types and code modules), the activity diagram (the end-to-end workflow) and the sequence diagram (the messages exchanged during mentor verification). The diagrams were drawn in draw.io, and their source files are in 04_System_Design_and_Development/diagrams.

## Use Case Diagram

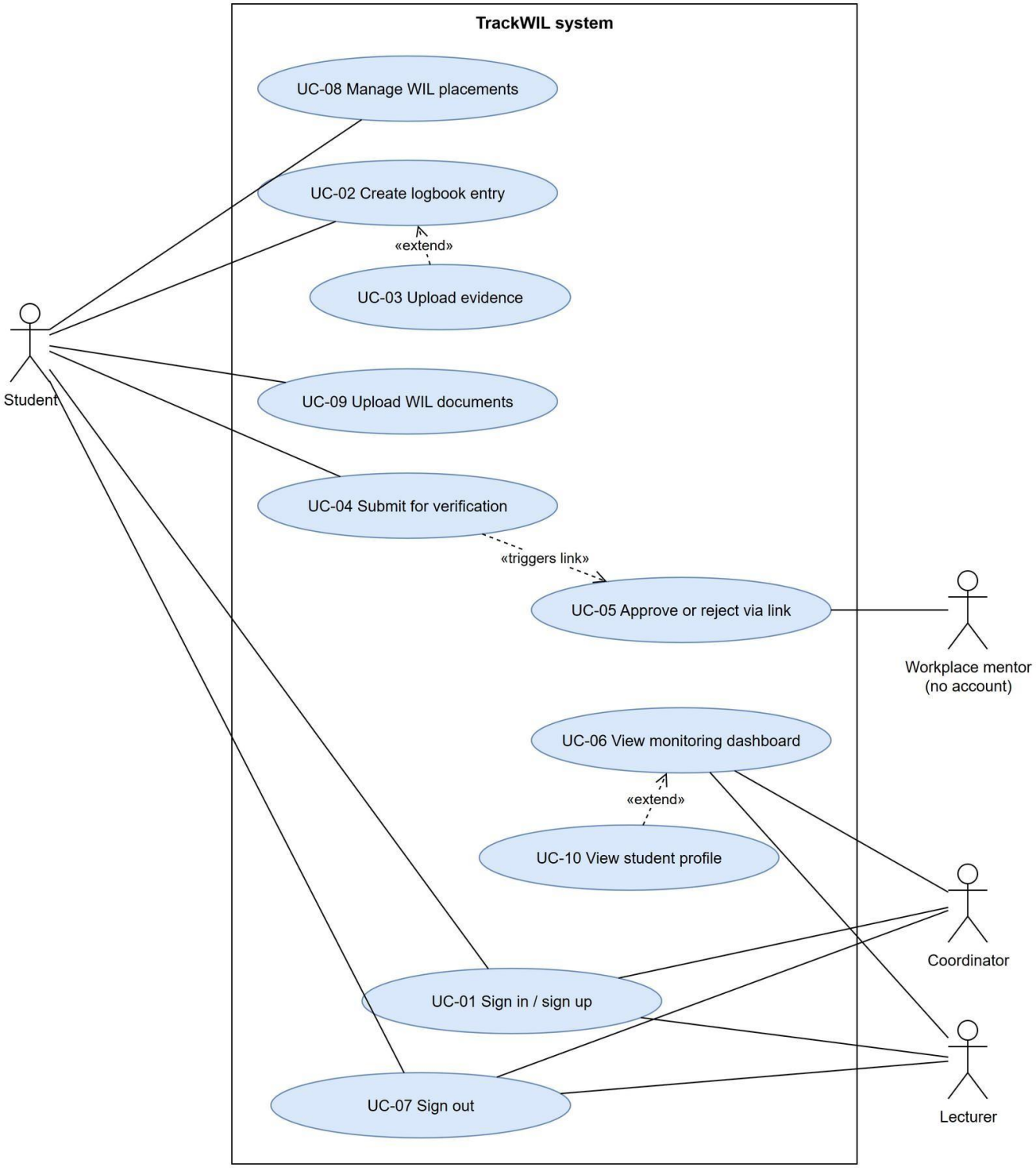



Figure 2: Use case diagram

Figure 2 shows four actors. The student signs in or signs up (UC-01), manages their WIL placements (UC-08), creates logbook entries (UC-02), optionally uploads evidence (UC-03, which extends UC-02 because it is optional), uploads the WIL documents (UC-09), submits entries for verification (UC-04) and signs out (UC-07). Submitting an entry triggers the link that the workplace mentor uses to approve or reject the entry (UC-05); the mentor has no account and takes part in no other use case. The coordinator and the lecturer both sign in, view the monitoring dashboard (UC-06), from which they can open a student's profile (UC-10, which extends UC-06), and sign out. Each use case maps to the functional requirements listed in the SRS, Section 5.

## Class Diagram

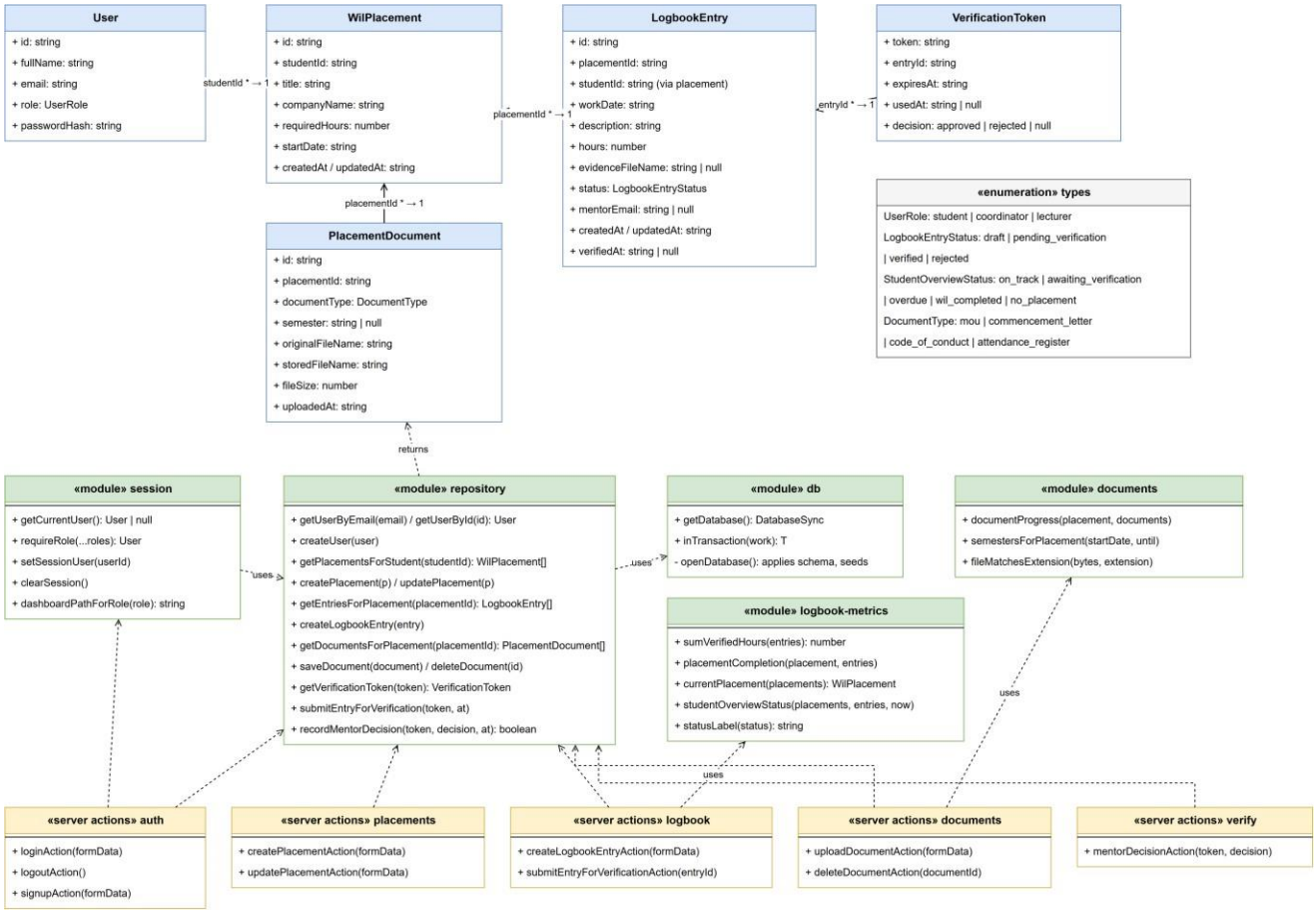



Figure 3: Class diagram of the main types and modules

Figure 3 shows the five domain types that are stored in the database (User, WilPlacement, LogbookEntry, VerificationToken and PlacementDocument) and the enumerations they use. A WilPlacement belongs to one User through studentId; a LogbookEntry and a PlacementDocument each belong to one WilPlacement through placementId; and a VerificationToken belongs to one LogbookEntry through entryId. The lower half shows how the code is organised: the Server Actions (auth, placements, logbook, documents and verify) depend on the session and repository modules, the repository depends on the db module, the logbook-metrics module calculates verified hours, placement completion and the dashboard status, and the documents module holds the document rules. Because Next.js code is organised in modules and functions rather than classes, modules are shown with the «module» and «server actions» stereotypes.

## Activity Diagram

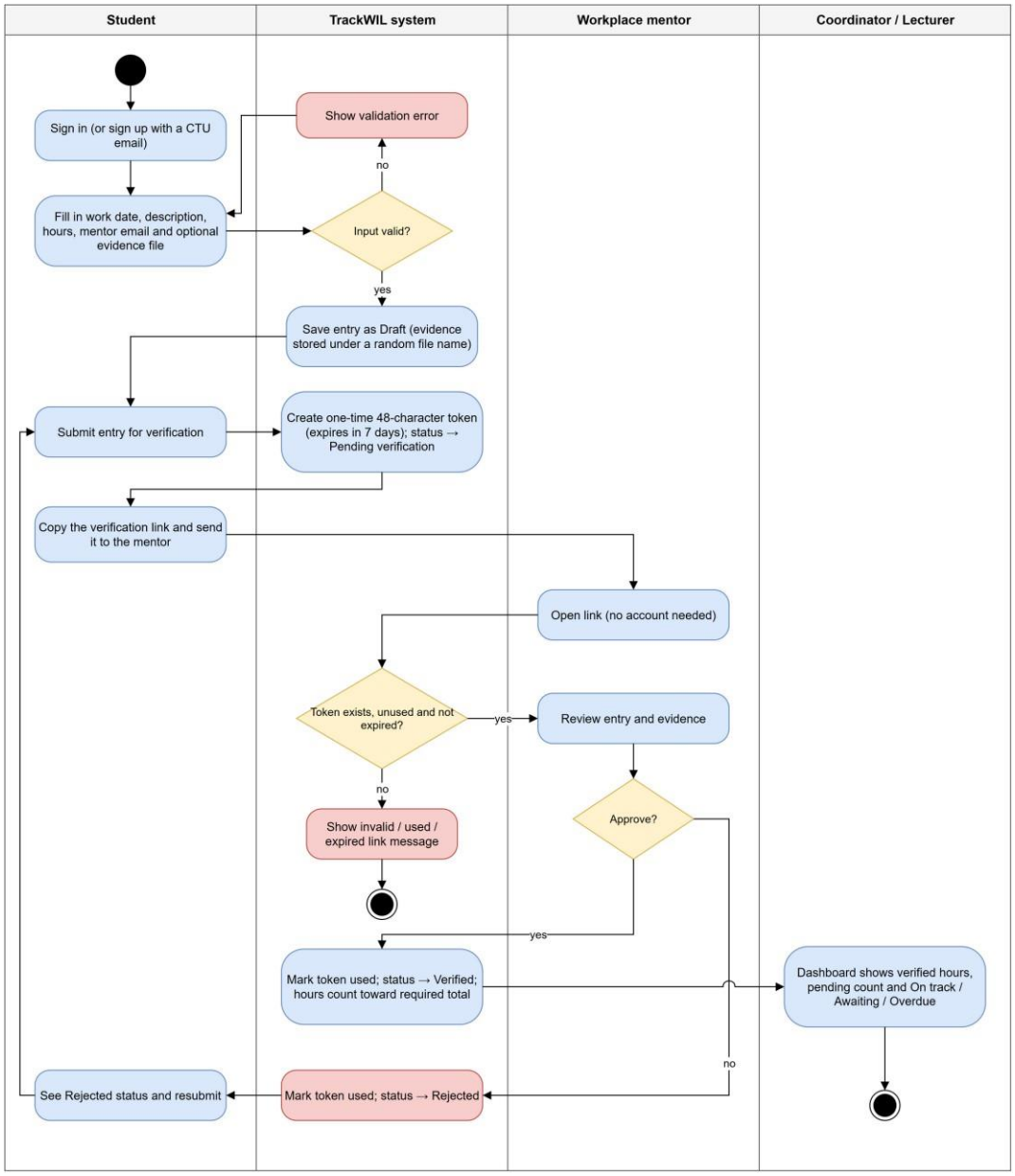



Figure 4: Activity diagram of the logbook and verification workflow

Figure 4 follows one logbook entry from creation to verification across four swimlanes. Invalid input returns the student to the form. A valid entry is saved as a Draft, then submitted, which creates a token and sets the status to Pending verification. When the mentor opens the link the system checks that the token exists, is unused and has not expired; if not, an error is shown and nothing changes. An approval sets the entry to Verified and adds its hours to the student's total on the dashboard. A rejection sets it to Rejected, and the student can resubmit it, which creates a new token.

### Sequence Diagram

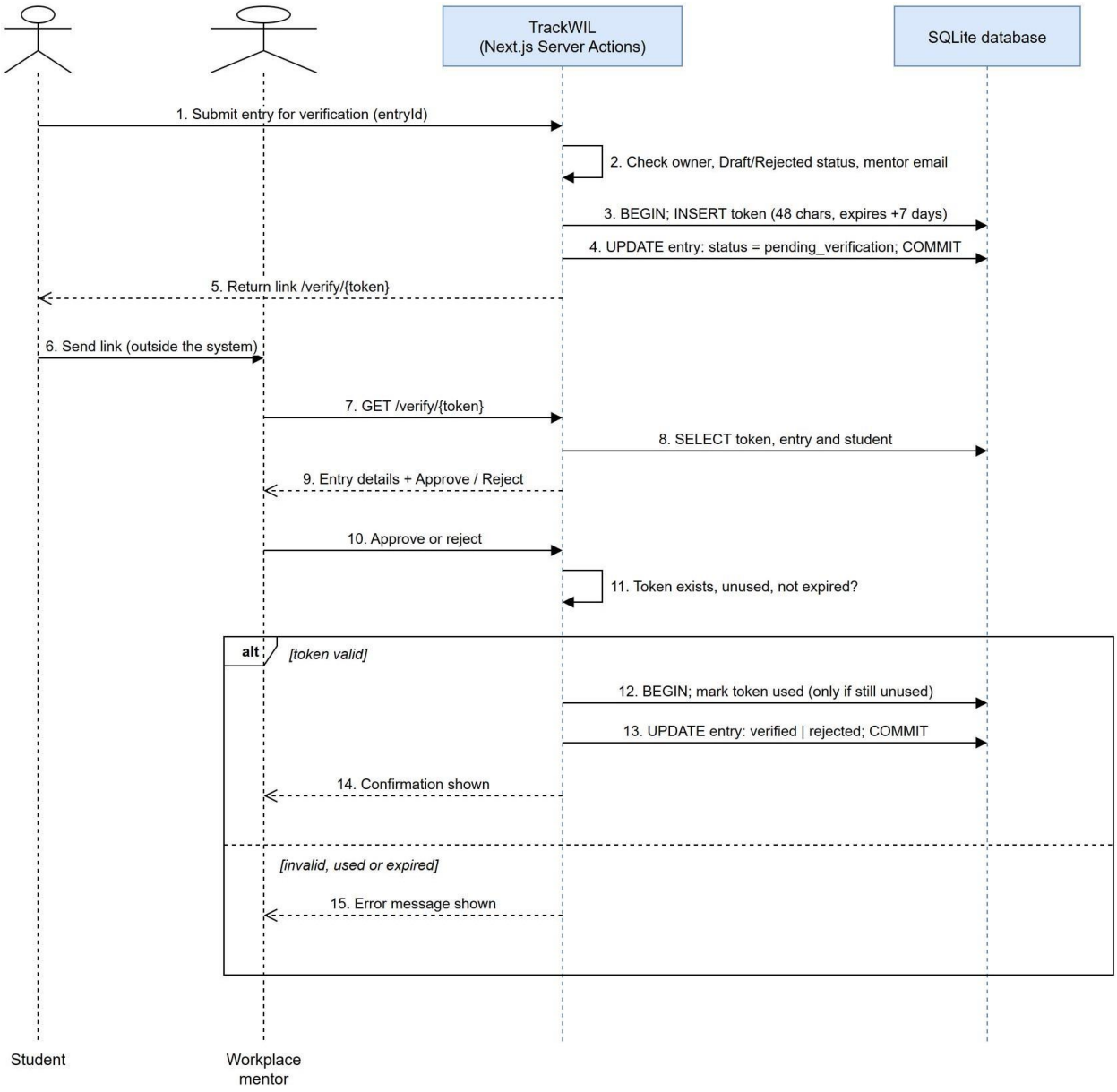



Figure 5: Sequence diagram of mentor verification

Figure 5 shows the messages between the student, the mentor, the TrackWIL server and the SQLite database during verification. Steps 3–4 and 12–13 each run inside a database transaction, so a token is never saved without the matching status change. Step 12 updates the token only if it is still unused, which prevents two decisions being recorded if the mentor clicks twice or opens the link in two tabs. The alt frame shows the two outcomes of the token check.

## 4.4 Database Design

TrackWIL uses an SQLite relational database with five tables. The schema is in 05_Source_Code/trackwil/database/schema.sql and the synthetic seed data is in seed.sql; both are applied automatically the first time the application starts. Figure 6 shows the entity-relationship diagram, and the full data dictionary is in the Database Design document.

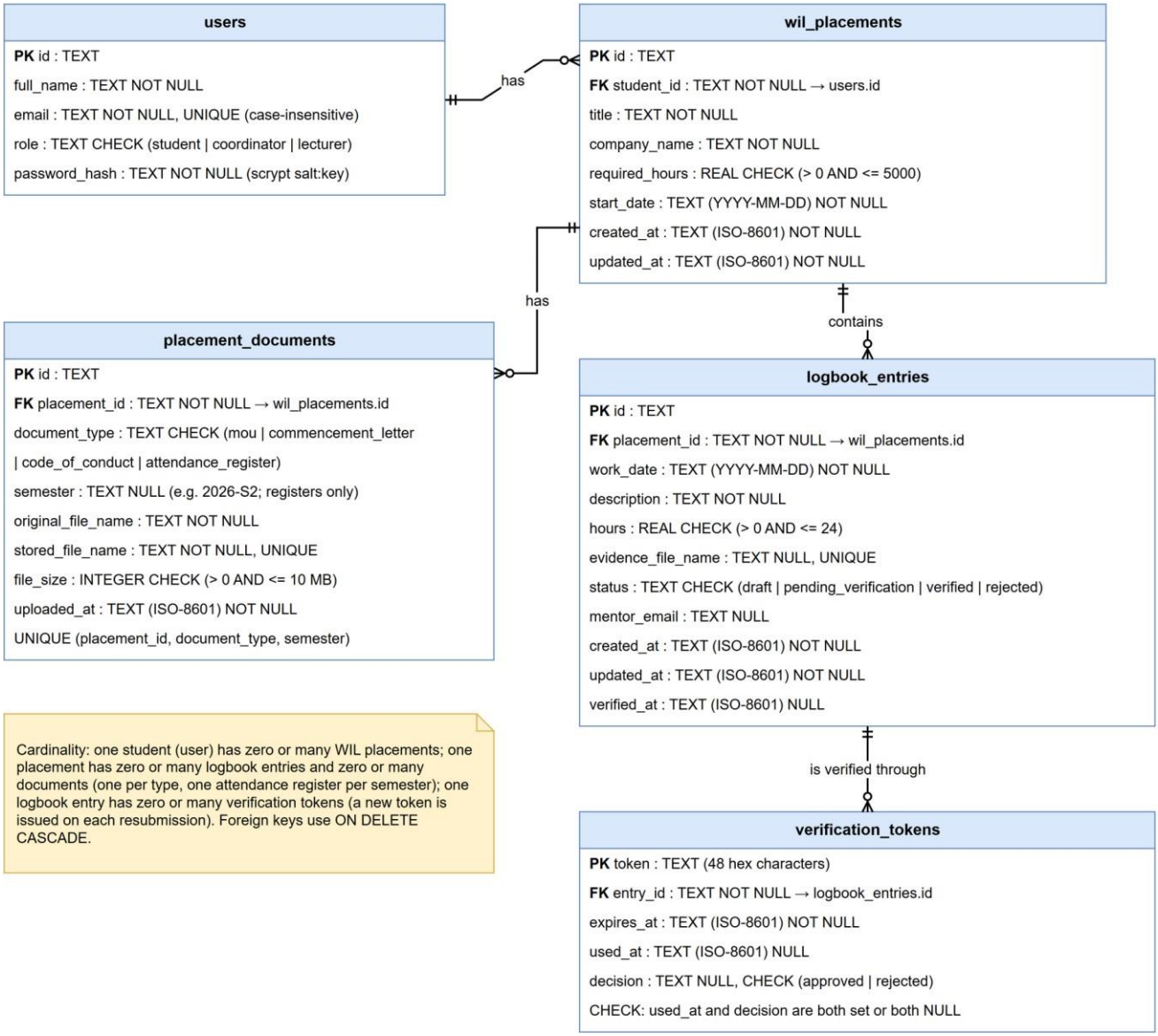



Figure 6: Entity-relationship diagram (SQLite)

**Entities and keys.** Users stores every account (students, coordinators and lecturers), with id as the primary key and a unique, case-insensitive email. wil_placements stores one row per WIL placement, with id as the primary key, student_id as a foreign key to users and the hours the placement requires. logbook_entries stores one row per day of work, with id as the primary key and placement_id as a foreign key to wil_placements. placement_documents stores one row per uploaded WIL document, with placement_id as a foreign key to wil_placements; the file itself is kept on disk under a random name. verification_tokens stores one row per verification link, with the token itself as the primary key and entry_id as a foreign key to logbook_entries.

**Relationships and cardinality.** One student has zero or many WIL placements, and each placement belongs to exactly one student. One placement has zero or many logbook entries and zero or many documents (one of each type and one attendance register per semester), and each belongs to exactly one placement. One entry has zero or many verification tokens (a new token is created each time a rejected entry is resubmitted), and each token belongs to exactly one entry. All foreign keys use ON DELETE CASCADE, so deleting a user cannot leave orphaned placements, entries, documents or tokens.

**Normalisation.** The design is in third normal form. Every column holds a single value (1NF); every table has a single-column primary key, so no column depends on only part of a key (2NF); and no non-key column depends on another non-key column (3NF). For example, the student's name is stored once in users and not copied into each entry, and an entry stores only its placement_id, because the student can be found through the placement. Values that can be calculated, such as verified hours, whether a placement is WIL completed and a student's status, are not stored at all but calculated from the entries when the dashboard is displayed, so they can never be out of date. The mentor's email is stored on the entry because the mentor belongs to the placement for that piece of work and mentors have no account of their own.

**Integrity rules.** CHECK constraints limit role and status to their allowed values, require hours to be more than 0 and at most 24 and a placement's required hours to be more than 0 and at most 5 000, limit document types to the four WIL documents, and require a token's used_at and decision to be set together. A unique index allows only one document of each type per placement and one attendance register per semester. The application validates the same rules first and shows a friendly message; the database constraints are a second line of defence.

**Design trace.** The database design supports FR-04 (entry fields), FR-06 to FR-08 (tokens with expiry, use time and decision), FR-11 (status calculated from entries), FR-12 (timestamps), FR-15 and FR-18 (placements with their required hours, from which completion is calculated) and FR-16 (documents).

### 4.5 User Interface Design

The user interface was designed as a working prototype rather than paper wireframes: each screen was built in the first sprint, reviewed by the team and refined in the next sprint. The screenshots below were captured from the application running with the synthetic demonstration data.

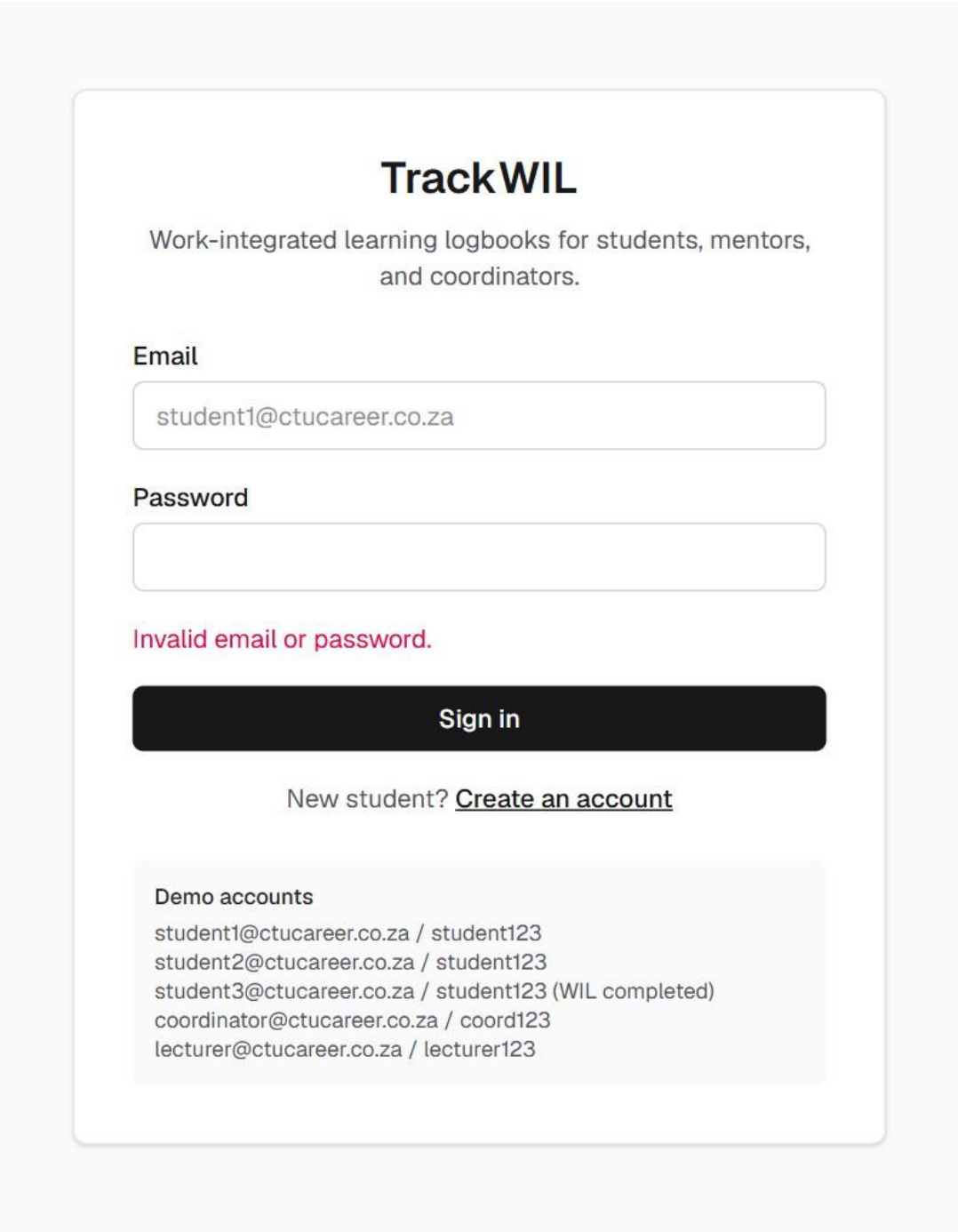



Figure 7: Sign-in page with an error for a wrong password

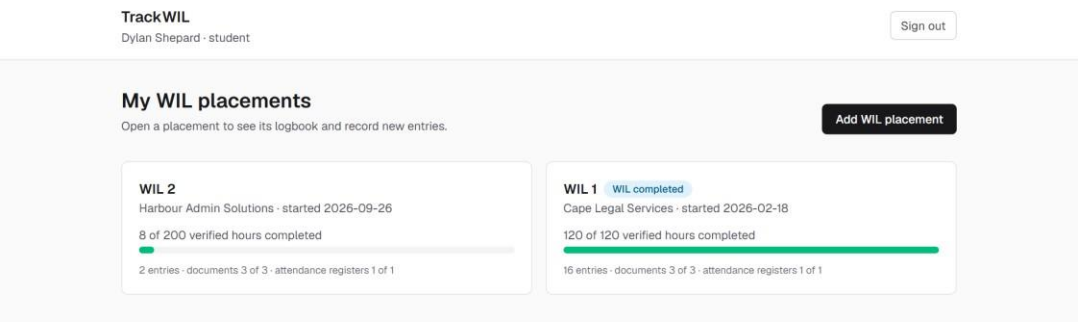



Figure 8: Student's WIL placements with hours and document progress (WIL 1 completed)

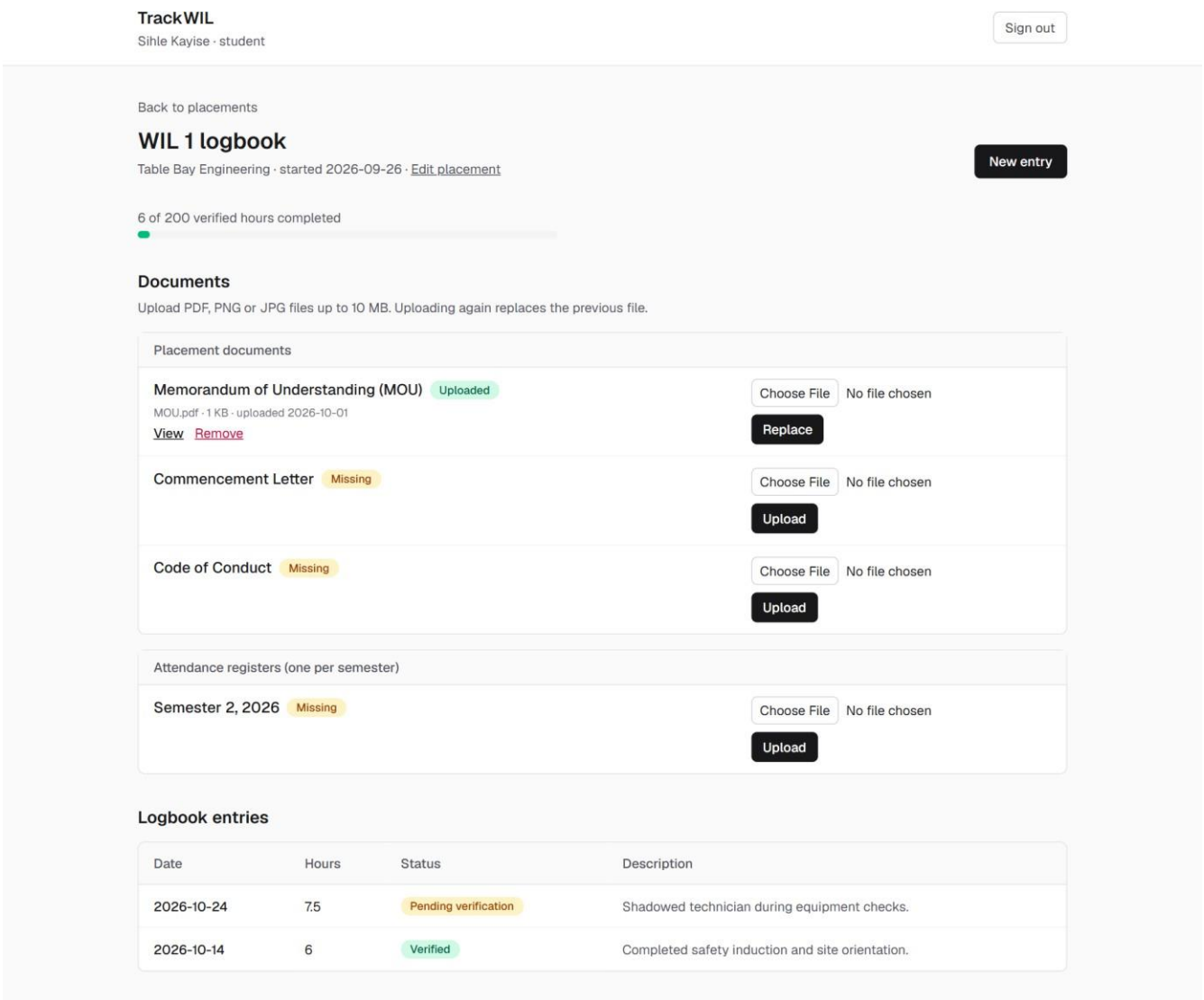



Figure 9: Placement logbook with verified hours, WIL documents and entry statuses

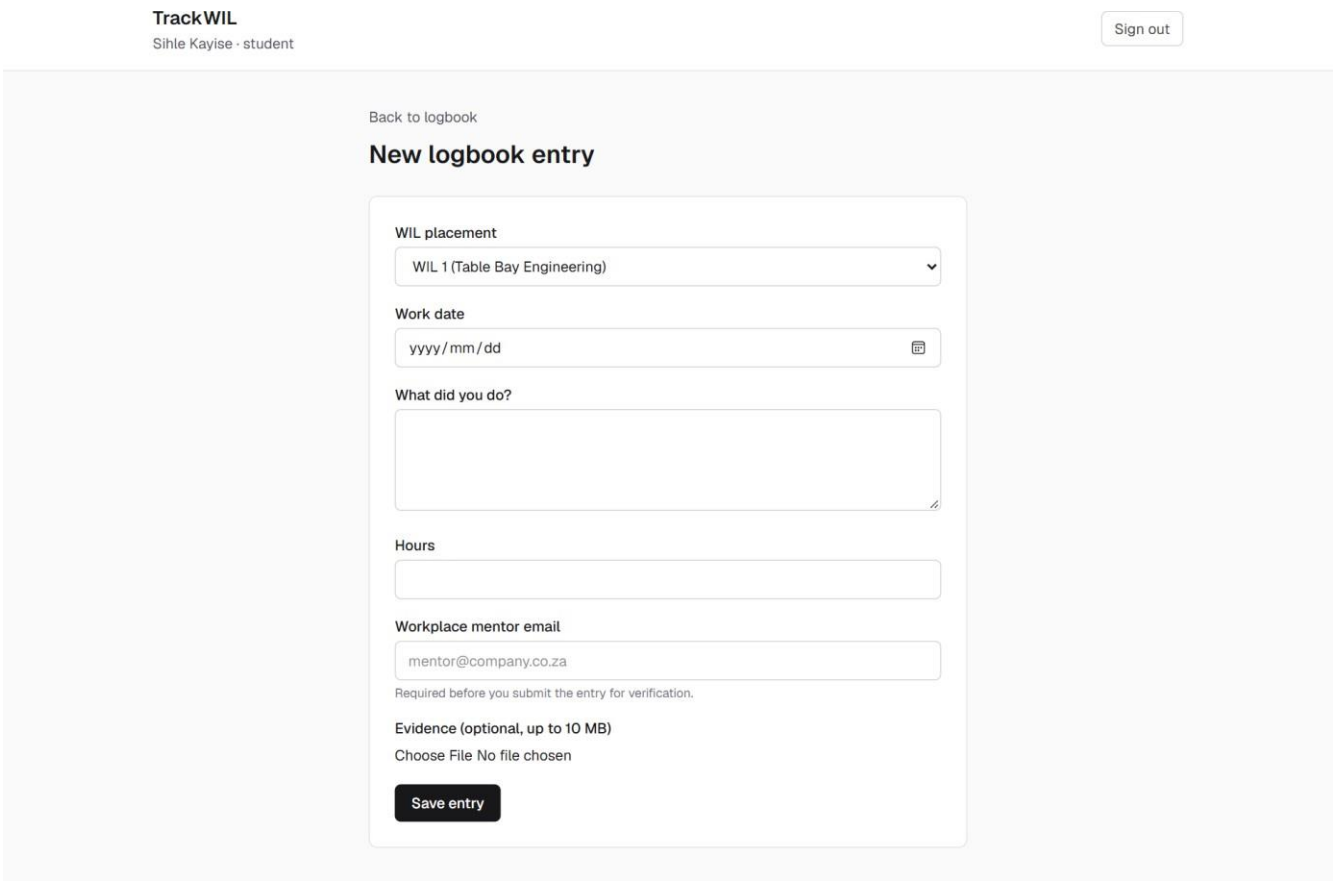



Figure 10: New logbook entry form

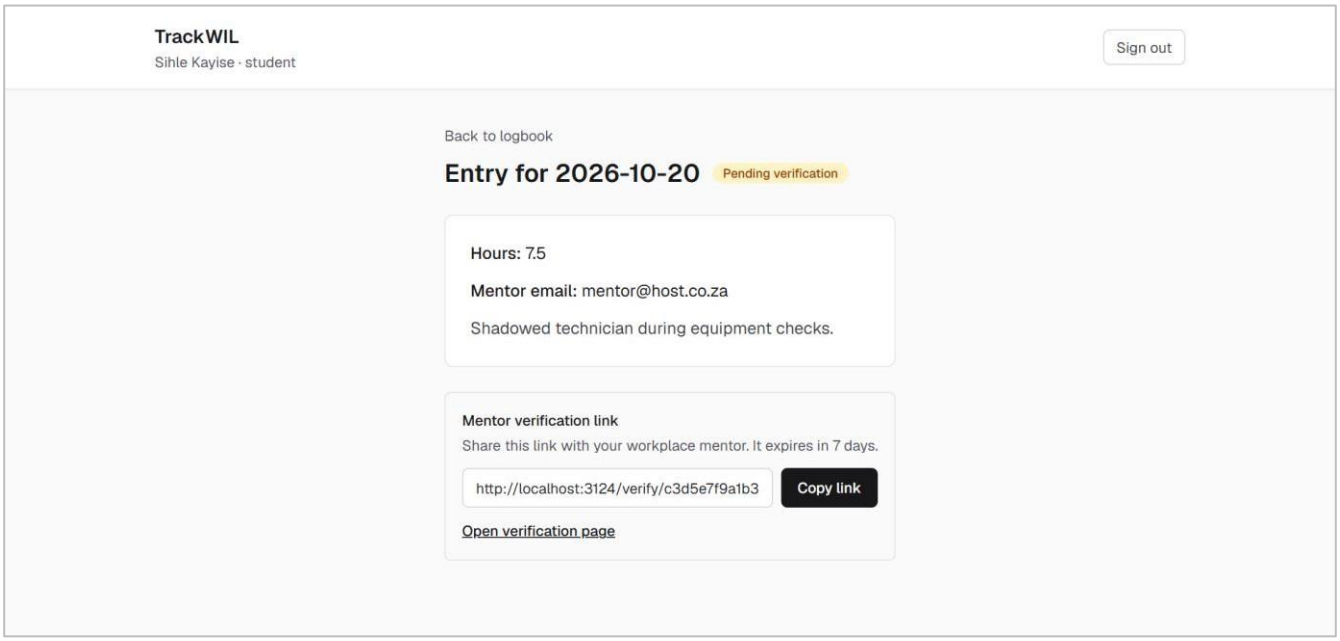



Figure 11: Entry details with the mentor verification link

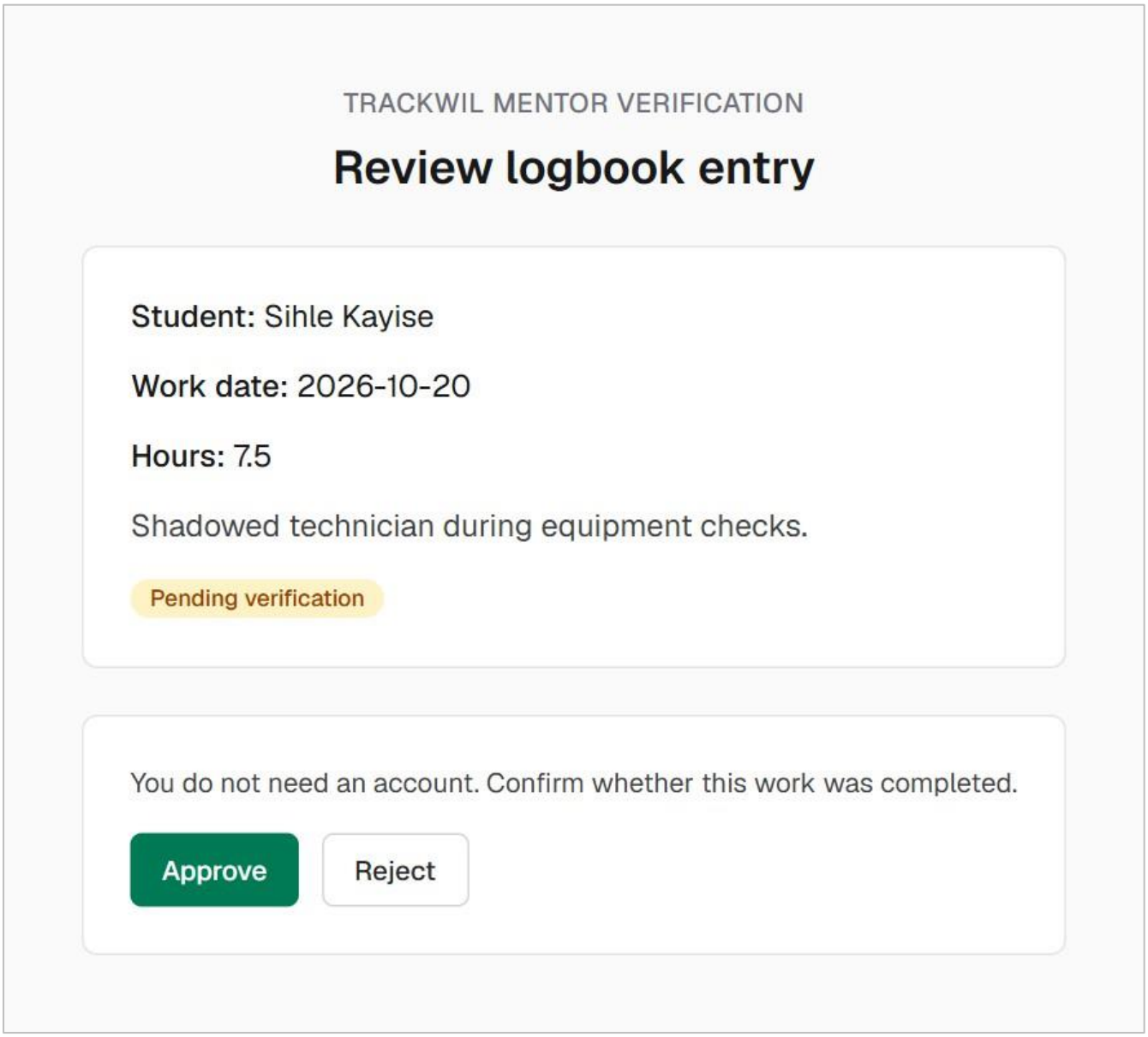



Figure 12: Mentor verification page (no sign-in required)

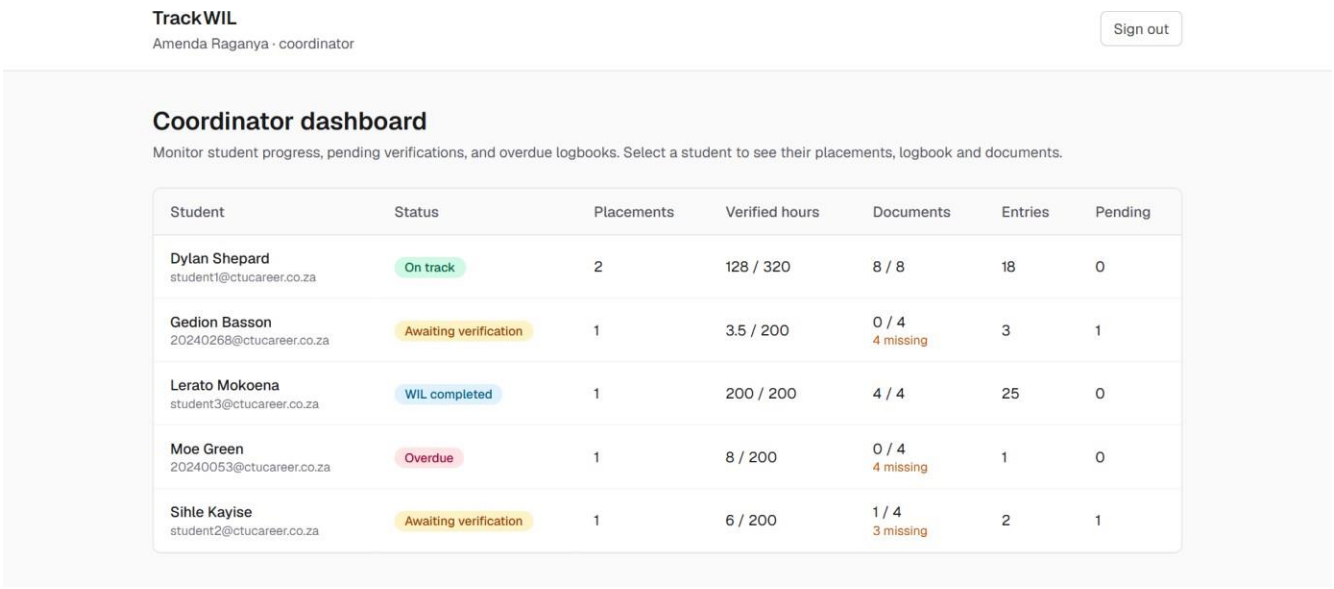



Figure 13: Coordinator dashboard with each student's status, hours and missing documents

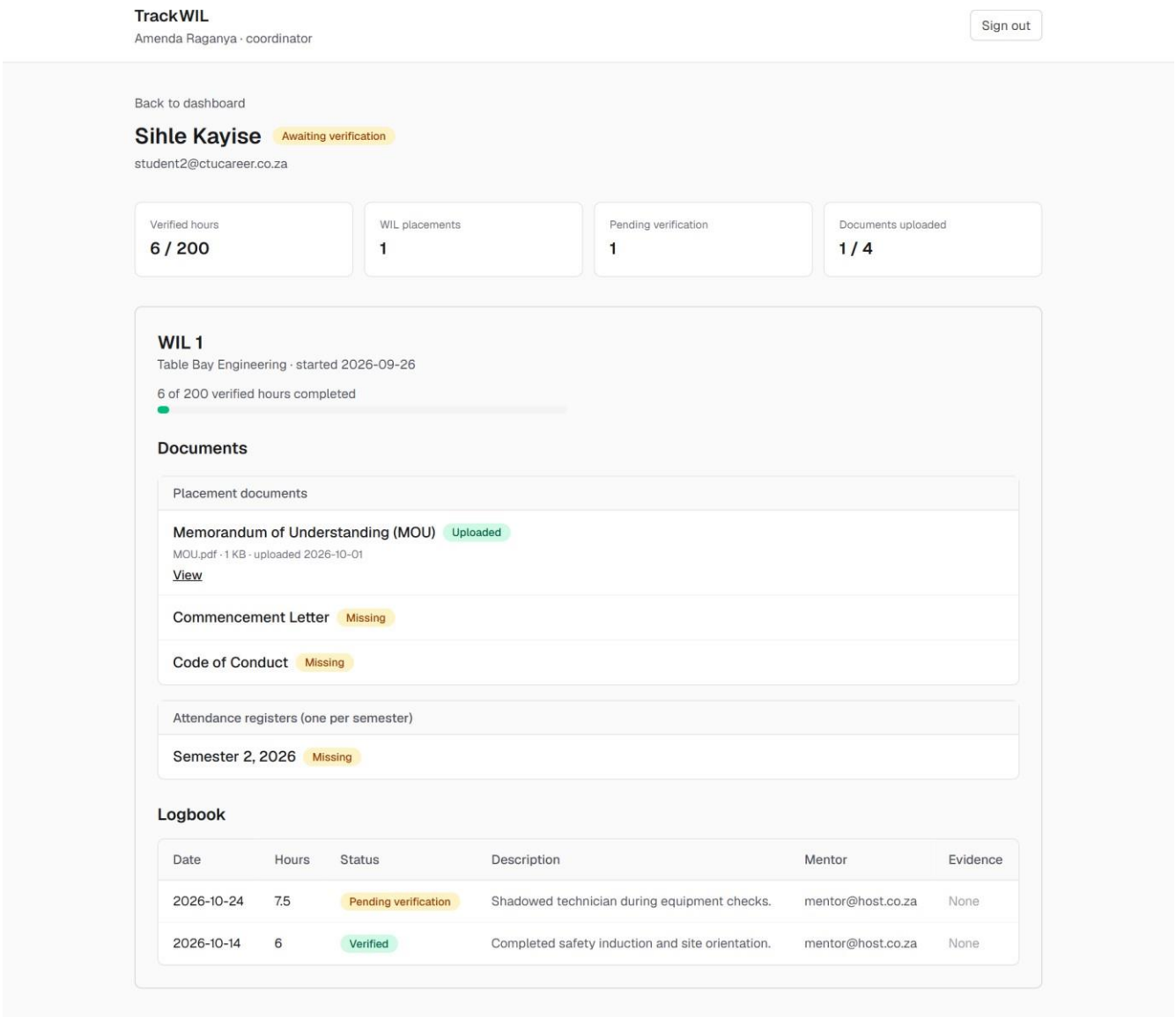



Figure 14: Student profile for coordinators and lecturers: placement hours, documents (view only) and logbook

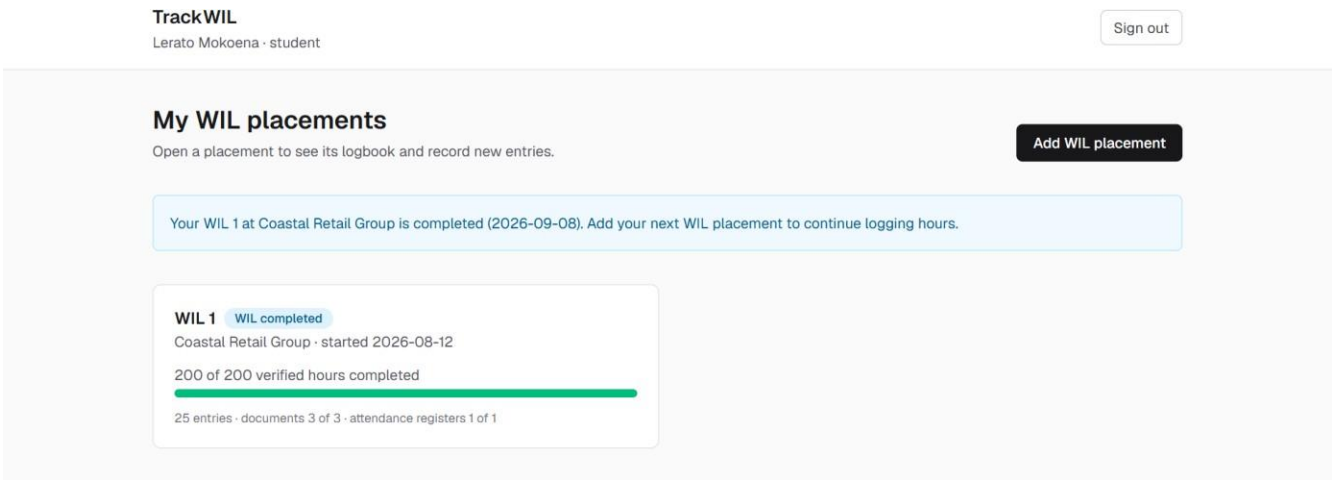



Figure 15: A completed placement: the student is asked to add the next WIL placement

Table 15: Usability decisions

| Decision | Reason | Screen |
|---|---|---|
| Plain-language error messages next to the form (for example "Invalid email or password.") | Users can see what went wrong and correct it without leaving the page | Sign-in, sign-up, new entry |
| Each status shown as a coloured badge with a text label | Colour makes status quick to scan; the text means it does not rely on colour alone | Logbook, entry details, dashboard |
| Progress shown as "X of Y verified hours completed" with a bar for each placement | The student's most important question is answered first | Placements, placement logbook |
| Each document listed as Uploaded or Missing, with one row per document and per semester | Students and staff can see at a glance which paperwork is outstanding | Placement logbook, student profile |
| A completed placement shows a WIL completed badge and hides the New entry and upload buttons | The student cannot change a finished placement by accident and is told to add the next one | Placements, placement logbook |
| Only four fields on the new-entry form, with evidence marked as optional and a hint under the mentor email | Keeps daily logging quick (NFR-01) | New entry |
| Copy link button and a note that the link expires in 7 days | The student can paste the link into an email or chat without retyping it | Entry details |
| Mentor page needs no sign-in and has two clear buttons | Mentors are busy and external; verification should take seconds | Mentor verification |
| One table on the dashboard with status first, then hours and pending count | Coordinators can see at a glance which students need follow-up | Coordinator dashboard |
| The header always shows the user's name, role and a Sign out button | Users always know which account they are using | All signed-in screens |

## 4.6 Development Environment

Table 16: Development environment

| Item | Tool or version |
|---|---|
| Languages | TypeScript; HTML and CSS through JSX and Tailwind; SQL (schema and queries) |
| Framework | Next.js 16.4.3 (App Router, Server Actions) with React 19.3 |
| Runtime | Node.js 24 (minimum 22.13 for the built-in node:sqlite module) |
| Database | SQLite (file data/trackwil.db), schema in database/schema.sql |
| Libraries | Tailwind CSS 4; Node.js crypto, fs and node:sqlite (built in) |
| Code quality | TypeScript compiler (type checking) and ESLint 9 with the Next.js rules |
| IDE | Visual Studio Code |
| Design tools | draw.io (UML, ERD and architecture diagrams) |
| Version control | Git and GitHub (https://github.com/Noobey32/TrackWIL) |
| Cloud services | None; the prototype runs locally |
| Operating system | Microsoft Windows 11 |

To run the application, install Node.js 22.13 or newer and run npm install and npm run dev in 05_Source_Code/trackwil, then open http://localhost:3000. The README lists the demonstration accounts. The command npm run db:reset restores the demonstration data.

## 4.7 Implementation

This section explains the most important modules. Short code excerpts are included only where they show how a requirement or security rule is enforced.

### Project Structure

Table 17: Source code structure (05_Source_Code/trackwil)

| Folder or file | Contents |
|---|---|
| database/ | schema.sql (tables, keys, constraints), seed.sql (synthetic data) and demo-documents/ (synthetic PDFs for the seed data) |
| scripts/reset-db.mjs | Rebuilds the database from the SQL scripts |
| src/app/ | Pages and routes: sign-in, signup, student (placements, entries), coordinator (dashboard, student profiles), verify, api/evidence, api/documents |
| src/app/actions/ | Server Actions: auth.ts, placements.ts, logbook.ts, documents.ts, verify.ts |
| src/components/ | Forms (placement, entry, document upload), status badges, hours progress bar, documents section, header, copy-link button |
| src/lib/ | db.ts, repository.ts, session.ts, password.ts, signup-validation.ts, logbook-metrics.ts, documents.ts, types.ts |
| src/proxy.ts | Redirects signed-out visitors away from protected areas |

### Authentication and Passwords

Passwords are never stored. On sign-up the password is hashed with scrypt and a random 16-byte salt, and on sign-in the hash is recalculated and compared in constant time, as recommended by OWASP (OWASP Foundation, n.d.a). Sign-up also checks that the email is a CTU student address and that the password meets the strength rules (FR-02).

```
export function hashPassword(password: string): string {
  const salt = randomBytes(16).toString("hex");
  const derivedKey = scryptSync(password, salt, KEY_LENGTH);
  return `${salt}:${derivedKey.toString("hex")}`;
}
```

After a successful sign-in the server sets an HttpOnly session cookie and redirects the user to /student or /coordinator, depending on their role.

### Role-Based Access

Access is checked on the server at two levels. The proxy redirects any visitor without a session away from /student and /coordinator. Each protected page then checks the user's role itself, through requireRole in session.ts, before it loads any data, so that a coordinator cannot open the student area and a student cannot open the dashboard. A check in the area's layout alone is not enough, because Next.js renders a layout and its page at the same time: during testing this let page data be sent in the body of the redirect response (DEF-07). Every Server Action also checks the role and ownership again before changing data, for example:

```
const entry = await getLogbookEntryById(entryId);
if (!entry || entry.studentId !== user.id) {
  return { error: 'Entry not found.' };
}
```

### Logbook Entries and Evidence Upload

createLogbookEntryAction validates the date, description and hours (more than 0 and at most 24) before saving. An evidence file is accepted only if its extension is PDF, PNG, JPG, JPEG or WEBP, and it is saved under a random UUID name so that a user can never choose or overwrite a file name on the server (OWASP Foundation, n.d.c). Files are served through /api/evidence/[fileName], which returns 401 to signed-out visitors and 403 to anyone other than the owning student, a coordinator or a lecturer (FR-13).

### WIL Placements, Documents and Completion

Every logbook entry belongs to one of the student's WIL placements, and each placement stores its own required hours (FR-15); createLogbookEntryAction checks that the chosen placement belongs to the signed-in student. Documents are uploaded with uploadDocumentAction (FR-16). A file must be a PDF, PNG or JPG of at most 10 MB, and its first bytes must match its type, so a renamed program is refused. It is saved under a random name, and the database allows only one document of each type per placement and one attendance register per semester, so uploading again replaces the previous file. Documents are served by /api/documents/[id] with the same access rules as evidence. placementCompletion in logbook-metrics.ts adds up a placement's verified hours in the order they were approved and marks the placement WIL completed once they reach its required hours (FR-18). Every action that changes a placement (a new entry, a submission, a document upload or removal, or an edit) checks this first and refuses a completed placement.

### Mentor Verification Tokens

When a student submits an entry, the server creates a token from 24 random bytes (48 hexadecimal characters, 192 bits of randomness), which cannot practically be guessed, and sets it to expire after 7 days. The token and the new Pending verification status are saved in one transaction. When the mentor decides, the token is marked as used only if it is still unused, and the entry is updated in the same transaction:

```
const tokenUpdate = database
  .prepare(`UPDATE verification_tokens SET used_at = ?, decision = ?
    WHERE token = ? AND used_at IS NULL`)
  .run(decidedAt, decision, tokenValue);
if (tokenUpdate.changes === 0) {
  return false; // already used, e.g. a double click
}
```

### Monitoring Dashboard

The dashboard calculates each student's status each time it is displayed, from their current placement (the one that started most recently). A student with no placement is shown as No placement, and one whose current placement is completed as WIL completed. Otherwise, a student with an entry awaiting verification is shown as Awaiting verification; a student whose latest verification, or the placement start if nothing is verified yet, is more than 14 days ago is shown as Overdue; all other students are On track. Selecting a student opens their profile (FR-17), which shows every placement with its hours, documents and logbook. Verified hours are the sum of the hours of Verified entries only.

### Database Layer

db.ts opens the SQLite file, turns on foreign-key enforcement, applies schema.sql and loads seed.sql if the database is empty. It also provides an inTransaction helper that commits all statements together or rolls them all back. repository.ts contains every SQL query and returns plain TypeScript objects, so pages and actions never contain SQL.

### Known Security Limitation

During the security review the team found that the session cookie stores the user's id without a signature. The cookie is HttpOnly, so it cannot be read by JavaScript on the page, but a user who edits the cookie in their own browser could change the id to impersonate another account. This defect is recorded in the Defect Log, and the planned fix is to sign the cookie (or use a random session token stored in the database) so that a modified cookie is rejected. Chapter 5 reports the retest.

## 4.8 Version Control

The repository on GitHub (https://github.com/Noobey32/TrackWIL) holds both the documentation and the source code. The capstone pack is organised into numbered folders from 00_Project_Administration to 12_Assessment, with the application in 05_Source_Code/trackwil, the design files and diagrams in 04_System_Design_and_Development, and each weekly submission in its own folder.

### Branching and Pull Requests

The team agreed that each member creates a branch named after themselves and the task (name/task, for example moe/system-design), finishes the work there, and merges it into main through a pull request so that the others can review it. main is kept as the stable branch.

Table 18: Branches in the repository

| Branch | Purpose | Merged into main |
|---|---|---|
| main | Stable, reviewed work | (base branch) |
| app-dev | First application development (Next.js scaffold and MVP) | Yes |
| moe/system-design | UML, ERD and architecture diagrams and the Chapter 4 draft | Yes |
| gedion/SRS_Documents_Chapter_3 | Software Requirements Specification and Chapter 3 | Pull request #2 |
| Dylan/Revise-chapters-1-and-2 | Revisions to Chapters 1 and 2 after Week 2 feedback | Yes |
| gedion/Final_Changes_Week_3_Submission | Week 3 submission compilation | Pull request #3 |
| sihle-trackwil | Chapter 5, test plan and test cases | Pull request #4 |

| Branch | Purpose | Merged into main |
|---|---|---|
| gedion/sqlite-database | Replace JSON storage with the SQLite database | Pull request #5 |
| gedion/docs-match-build | Update the documentation to match the final build | Pull request (Week 4) |
| gedion/wil-placements | WIL placements with their own required hours | Pull request #11 |
| gedion/fix-new-entry-page | Fix the error on the new entry page (DEF-06) | Pull request #12 |
| gedion/student-documents | Upload of WIL documents for each placement | Pull request #13 |
| gedion/coordinator-student-profile | Student profile for staff; fix for page data sent in redirects (DEF-07) | Pull request #14 |
| gedion/wil-completion | Mark placements WIL completed and lock them | Pull request #15 |
| gedion/nextjs-security-upgrade | Upgrade Next.js to 16.4.3 (DEF-04) | Pull request #16 |
| gedion/documents-docs-update | Update the documentation for the new features | Pull request (final submission) |

### Contribution by Member

Table 19: Commits and main contributions by member (at the time of writing)

| Member | GitHub account | Commits | Main contributions shown in the history |
|---|---|---|---|
| Willem (Gedion) Basson | Cyber-4 | 20 | Project proposal and topic acceptance; administration folder (00); repository structure; SRS and Chapter 3; Week 2 and Week 3 submission compilation; SQLite database migration; merging pull requests #2, #3 and #5 |
| Moegammad Green | Noobey32 (repository owner) | 16 | Next.js scaffold and MVP; JSON storage and mentor verification; student sign-up validation; code documentation; draw.io diagrams; Chapter 4 draft; project charter and work plan |
| Dylan Shepard | FyreGH | 12 | Chapter 2 literature review and review matrix; reference list; formatting of Chapters 1–3; revised Chapters 1 and 2 table; Week 3 submission folder |
| Sihle Kayise | sihle588 | 4 | Chapter 1 and Chapter 3 uploads; Chapter 5 draft, test plan and test cases; pull request #4 |

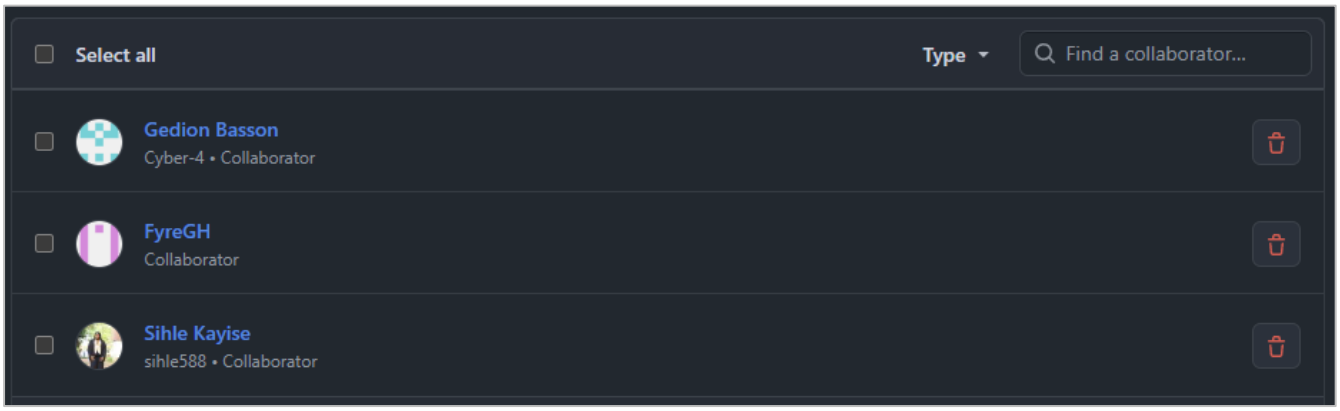



Figure 16: Collaborators on the GitHub repository

The commit history shows work on 7, 8, 9, 12, 14, 15, 22, 23 and 26 October 2026, spread across the project rather than in one final upload. The team recognises that the number of commits differs between members, and the Individual Contribution Forms explain each member's work in more detail, with references to specific commits and files.

### 4.9 Chapter Summary

This chapter presented TrackWIL's three-layer Next.js architecture, the use case, class, activity and sequence diagrams, the normalised SQLite database design and the user interface with its usability decisions. It described the development environment and the implementation of authentication, role-based access, evidence upload, WIL placements, documents and completion, one-time verification tokens, the monitoring dashboard and student profiles, and the database layer, and it recorded one known security limitation that will be fixed and retested. Finally, it summarised the repository structure, branches, pull requests and each member's contribution.

The core features implemented are: student sign-up and sign-in; WIL placements with their own required hours; logbook entries with optional evidence; upload of the WIL documents for each placement; verified hours against each placement's required hours, with automatic WIL completion; submission for verification with a 7-day one-time link; mentor approval or rejection without an account; and the coordinator and lecturer dashboard with a profile for each student. Chapter 5 tests these features against the requirements.

# CHAPTER 5 - TESTING, RESULTS & EVALUATION

## 5.1 Introduction

This chapter reports how TrackWIL was tested and evaluates the system against the requirements in Chapter 3. The purpose of testing was to show that every functional requirement works as specified, to measure the non-functional requirements wherever possible, and to find and fix defects before the final demonstration. The chapter describes the test strategy, summarises the test cases and their results, explains the defects that were found and corrected, presents the performance and usability evaluation, and evaluates each requirement. All test cases, defects and traceability links are recorded in the testing workbooks in the folder 06_Testing_Results_and_Evaluation.

## 5.2 Test Strategy

Testing followed the Test Plan and combined manual and automated testing at the system level.

- **Manual functional testing (TC-001 to TC-014).** Sihle Kayise tested the main user journeys in the browser on 22 October 2026: signing in and out as a student and as a coordinator, viewing and creating logbook entries, navigation, and invalid sign-in and empty-field validation.
- **Automated system testing (TC-110 to TC-144 and TC-150 to TC-175).** A test script (tests/system-tests.mjs, run with npm run test:system) sends the same requests as the browser to the running application and checks both the response and the database. It covers every functional requirement, the security rules, database constraints and response times, and can be repeated after every change. On 26 October 2026 it was extended with 26 tests for WIL placements, documents, the student profile and WIL completion, and for the retests of DEF-05 to DEF-07.
- **Browser and device checks (TC-145 to TC-146).** The main screens were checked in Google Chrome 155 and Microsoft Edge 155 at a phone width of 375 pixels and at desktop width.
- **Code and data reviews (TC-147 to TC-148).** Type checking, linting, the README set-up steps and the stored data were reviewed.
- **Usability testing with users (TC-149).** Planned, but blocked until written campus permission is received.

**Environment.** Windows 11 Pro; Node.js 24.21; TrackWIL production build (npm run build, npm start) with the SQLite database reset to the synthetic seed data before each run; Chrome 155.0 and Edge 155.0. No real personal data was used.

**Pass criteria.** A test passes when the actual result matches the expected result recorded in the workbook. Defects are logged, fixed, and retested with the same test case, and the full automated suite is rerun after every fix (regression testing).

## 5.3 Test Cases and Results

Table 20: Summary of test results

| Test group | Test cases | Requirements | Pass | Fail | Blocked |
|---|---|---|---|---|---|
| Manual functional tests (Sihle Kayise) | TC-001 to TC-014 | FR-01, FR-04, FR-09 to FR-11, FR-14 | 14 | 0 | 0 |

| Test group | Test cases | Requirements | Pass | Fail | Blocked |
|---|---|---|---|---|---|
| Sign-in, sign-up and access control | TC-110 to TC-121 (incl. TC-120b) | FR-01 to FR-03, NFR-03 | 13 | 0 | 0 |
| Logbook entries and evidence | TC-122 to TC-126 | FR-04, FR-05, FR-13 | 5 | 0 | 0 |
| Verification links | TC-127 to TC-137 | FR-06 to FR-08 | 11 | 0 | 0 |
| Logbook, dashboard, timestamps and sign-out | TC-138 to TC-141 | FR-09 to FR-12, FR-14 | 4 | 0 | 0 |
| Security, integrity and performance | TC-142 to TC-144 | NFR-02 to NFR-04 | 3 | 0 | 0 |
| WIL placements | TC-150 to TC-156 | FR-03, FR-10, FR-15 | 7 | 0 | 0 |
| WIL documents | TC-157 to TC-165 | FR-03, FR-16, NFR-03 | 9 | 0 | 0 |
| Student profile and access control | TC-166 to TC-171 | FR-03, FR-11, FR-17, NFR-03 | 6 | 0 | 0 |
| WIL completion | TC-172 to TC-174 | FR-11, FR-18 | 3 | 0 | 0 |
| Sign-in page demo accounts | TC-175 | FR-01 | 1 | 0 | 0 |
| Browser, code and data checks | TC-145 to TC-148 | NFR-05 to NFR-07 | 4 | 0 | 0 |
| Usability with first-time students | TC-149 | NFR-01 | 0 | 0 | 1 |
| Total | 81 test cases | | 80 | 0 | 1 |

All 81 test cases are recorded in the Test Cases and Results workbook with their preconditions, steps, test data, expected and actual results, tester, date and evidence. The automated run logs (evidence/system_test_run_2026-10-23.txt and evidence/system_test_run_2026-10-26.txt) contain the actual output of every automated test. The final regression run on 26 October 2026, on Next.js 16.4.3, passed all 62 automated tests.

The results that matter most for the research aim are:

- **Account-free verification works end to end.** Submitting an entry produced a 48-character link that expires after exactly 7.00 days (TC-127). The mentor could open it without signing in (TC-130), approval set the entry to Verified with a timestamp (TC-131), and a rejected entry could be resubmitted with a new link (TC-132).
- **Links cannot be misused.** A used link (TC-133), an unknown link (TC-134) and an expired link (TC-135) were all refused without changing the entry, and two simultaneous approvals on the same link recorded only one decision (TC-136).
- **Access control holds.** Signed-out visitors, students and staff could reach only their own areas, and a blocked page sent none of its data (TC-117 to TC-120b, TC-171). Evidence files returned 200 to the owner and to a coordinator, 403 to another student, 401 to a signed-out visitor and 404 to a path-traversal attempt (TC-126).

- **Monitoring is accurate.** The dashboard showed Dylan as On track, Moe as Overdue and Sihle as Awaiting verification, matching the seed data (TC-139), and the logbook showed the correct verified hours (TC-138).
- **Placements, documents and completion work as specified.** Each placement counts its own verified hours (TC-155); documents are kept once per type and once per semester and are replaced on a new upload (TC-157 to TC-159); renamed, empty and oversized files are refused (TC-160); and a placement is marked WIL completed as soon as its required hours are verified, after which every change to it is refused (TC-172, TC-173).

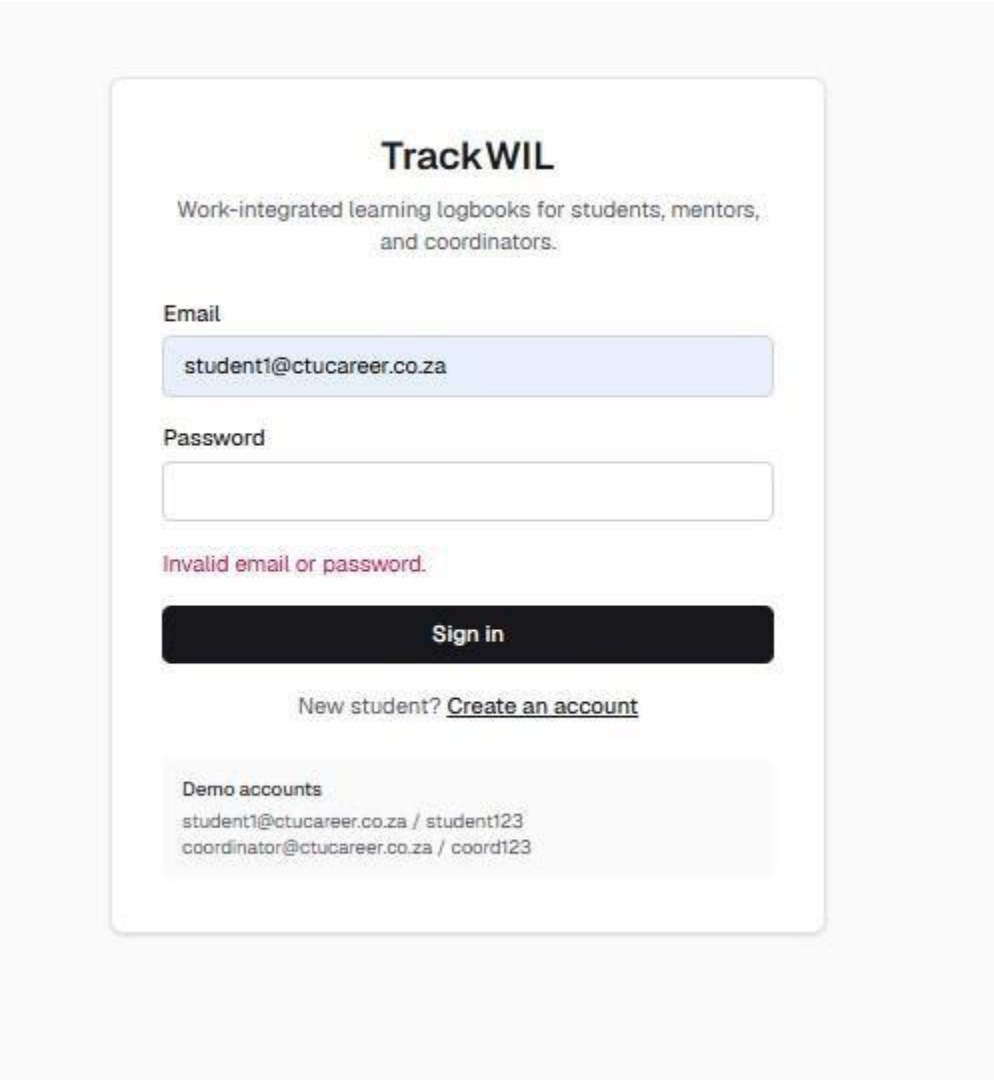



Figure 17: Manual test TC-011: an invalid sign-in is refused with a clear message (Sihle Kayise, 22 October 2026)

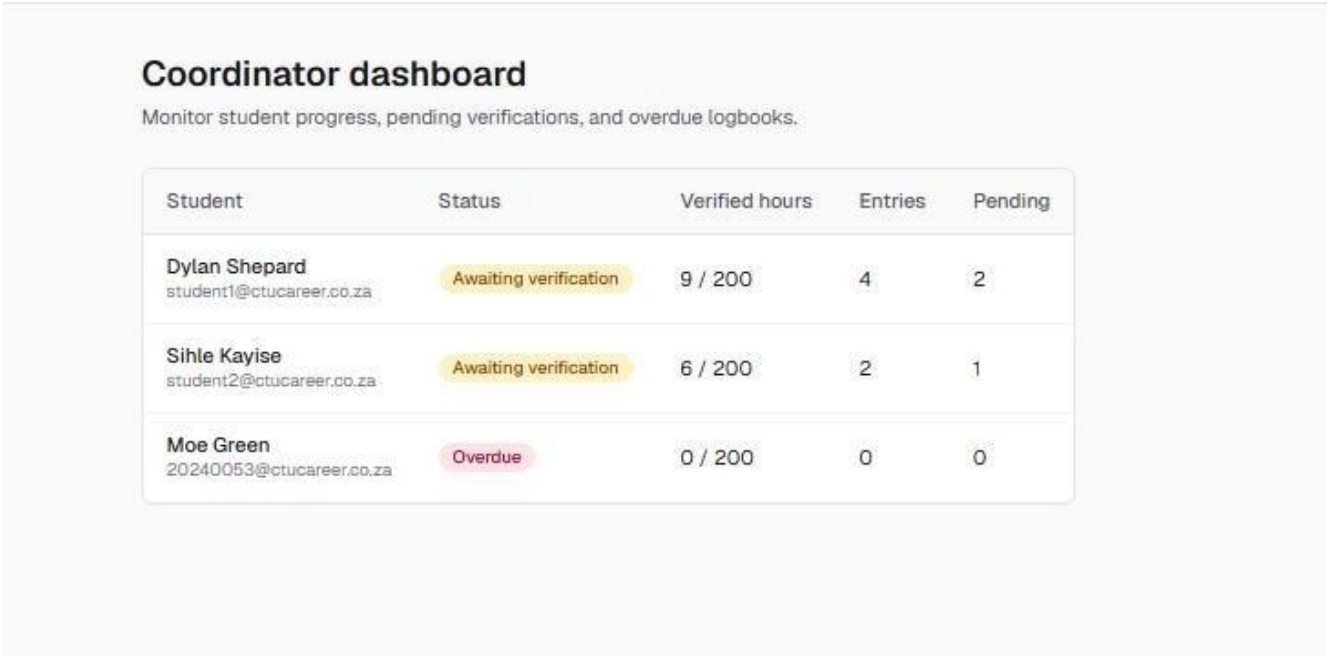



Figure 18: Manual test TC-009: coordinator dashboard during manual testing (Sihle Kayise, 22 October 2026)

## 5.4 Defects and Corrections

Seven defects were logged during testing. All seven were fixed, and each was retested with its own test case, followed by a full regression run on 26 October 2026.

Table 21: Defects found, fixes and retest results

| ID | Defect | Severity | Fix | Retest |
|---|---|---|---|---|
| DEF-01 | The session cookie held the user id without a signature. Setting the cookie to "coordinator-1" opened the coordinator dashboard without a password (reproduced on the earlier build: HTTP 200). | Critical | The cookie is now the user id plus an HMAC-SHA256 signature, checked in constant time; edited cookies are rejected. | Closed: TC-121 pass |
| DEF-02 | If a mentor link expired before the mentor responded, the entry stayed Pending verification and could never be resubmitted. | Medium | An entry with no active link can be resubmitted; the entry page shows an expiry notice. | Closed: TC-137 pass |
| DEF-03 | On a 375-pixel phone screen the logbook and dashboard tables widened the page to 580 pixels and cut off columns. | Medium | Full-width page container; tables scroll inside their box. | Closed: TC-145 pass (Chrome and Edge) |
| DEF-04 | The build reports a known security vulnerability in Next.js 16.4.0. | High | Upgraded Next.js to 16.4.3 (pull request #16) and reran the full test suite. | Closed: 62 automated tests pass; npm audit reports no vulnerabilities |
| DEF-05 | The sign-in page lists demo accounts but not the lecturer account. | Low | Every demo account, including the lecturer, is now listed. | Closed: TC-175 pass |

| ID | Defect | Severity | Fix | Retest |
|---|---|---|---|---|
| DEF-06 | The new entry and edit placement pages showed an error screen, because database rows were passed to the browser in a form React does not accept. | Medium | The repository copies every row into a plain object; the timing test now also fails on any page error. | Closed: TC-156 pass |
| DEF-07 | A role check only in a layout redirected the browser, but Next.js still sent the page's data in the redirect response: a student, or anyone with a made-up session cookie, could read every student's name, email and hours. | Critical | Every protected page checks the role itself (requireRole) before loading any data. | Closed: TC-171 pass; TC-118, TC-119 and TC-121 now check the response body |

DEF-01 is the most important finding. It was found by reviewing the session code against the OWASP guidance used in Chapter 2, and it shows the value of testing security as well as functionality: every functional test passed before the fix, yet any user could have impersonated a coordinator. The defect was first reproduced on the previous build, then fixed and retested, and the automated suite now includes a test that forged and tampered cookies are rejected, so the problem cannot return unnoticed.

DEF-07 teaches a similar lesson. It was found while testing the student profile (FR-17), but it also affected the coordinator dashboard, which had passed every earlier test: those tests accepted any redirect as blocked without checking what the response contained. The access-control tests now check that the protected content is absent from the response, and they failed on the earlier build before the fix.

### 5.5 Performance / Usability Evaluation

**Performance (NFR-02).** Each main page was requested 10 times on the test computer with the demonstration data. Every response was far below the 2-second target.

Table 22: Measured response times (TC-144, 26 October 2026)

| Page or action | Average (ms) | Maximum (ms) | Target |
|---|---|---|---|
| Sign-in page (/) | 3 | 5 | under 2 000 ms |
| Student placements (/student) | 4 | 4 | under 2 000 ms |
| Placement logbook | 5 | 7 | under 2 000 ms |
| New entry form | 4 | 4 | under 2 000 ms |
| Coordinator dashboard | 5 | 7 | under 2 000 ms |
| Student profile | 5 | 5 | under 2 000 ms |
| Mentor verification page | 3 | 5 | under 2 000 ms |
| Create a logbook entry | 10 | 10 | under 2 000 ms |

These times were measured on the same computer as the server, so they show that the application and database are fast. They do not include internet delays, which would add time in a hosted deployment.

**Compatibility (NFR-06).** All main screens worked in Chrome 155 and Edge 155. At 375 pixels the first check found DEF-03; after the fix every screen fits the phone width, and the mentor verification page, which mentors are most likely to open on a phone, needs no sideways scrolling at all.

**Usability (NFR-01).** The interface was designed for quick use (Chapter 4, Section 4.5): an entry needs four fields, statuses use colored labels with text, and the mentor page has two buttons. However, the planned timed task with first-time students (TC-149) could not be run because written campus permission for testing with students has not yet been received. NFR-01 is therefore not yet proven by user data, and this is stated as a limitation.

## 5.6 Requirements Evaluation

Table 23: Evaluation against the requirements

| Requirement | Result | Evidence |
|---|---|---|
| FR-01 to FR-03: sign-in, sign-up, role-based access | Met | TC-001, TC-008, TC-011, TC-013, TC-110 to TC-121, TC-168, TC-171, TC-175 |
| FR-04 to FR-05: logbook entries and evidence | Met | TC-005, TC-006, TC-012, TC-014, TC-122 to TC-125 |
| FR-06 to FR-08: submission and one-time mentor links | Met | TC-127 to TC-137 |
| FR-09 to FR-11: statuses, logbook and monitoring dashboard | Met | TC-002 to TC-004, TC-009, TC-138, TC-139, TC-170, TC-174 |
| FR-12 to FR-14: timestamps, protected evidence, sign-out | Met | TC-007, TC-010, TC-126, TC-140, TC-141 |
| FR-15: WIL placements with their own required hours | Met | TC-150 to TC-156 |
| FR-16: WIL documents | Met | TC-157 to TC-165 |
| FR-17: student profile for coordinators and lecturers | Met | TC-166 to TC-170 |
| FR-18: WIL completion and locking | Met | TC-172 to TC-174 |
| NFR-01 Usability | Partially met: designed for, not yet measured with users | TC-149 blocked |
| NFR-02 Performance | Met (local measurement) | TC-144 |
| NFR-03 Security | Met after the DEF-01, DEF-04 and DEF-07 fixes | TC-121, TC-126, TC-127, TC-142, TC-160, TC-162, TC-171 |
| NFR-04 Reliability and integrity | Met | TC-136, TC-143 |
| NFR-05 Maintainability | Met | TC-147 |
| NFR-06 Compatibility | Met after DEF-03 fix | TC-145, TC-146 |
| NFR-07 Privacy | Met | TC-148 |

All eighteen functional requirements are met, and six of the seven non-functional requirements are met. The Requirements Traceability Matrix links every requirement to its design component, implementation files and test cases. The requirements deferred in Chapter 3 (mentor accounts, written mentor feedback, mentor access to evidence, a separate lecturer dashboard, coordinator review of documents, notifications and reports) were not part of version 1.0 and are carried into the recommendations in Chapter 6.

### 5.7 User / Stakeholder Feedback

No feedback has yet been collected from real students, mentors, coordinators or lecturers, because the Ethical Clearance Confirmation does not allow personal data to be collected before written campus permission is granted. The evaluation therefore relies on testing against the requirements and on the feedback that was available:

- **Supervisor feedback on the proposal (7 October 2026)** narrowed the scope to three core features, which allowed them to be completed and tested thoroughly.
- **Supervisor feedback on Week 2** asked the team to remove template formatting and fix scope inconsistencies; these changes were made in the revised chapters.
- **Internal review during testing led to improvements to the system: the signed session cookie (DEF-01), resubmission of expired links (DEF-02), the mobile layout (DEF-03) and, after the Week 4 submission, the role check on every page (DEF-07).**
- The team's review of the WIL process after the Week 4 submission added WIL placements, document uploads, a student profile for staff and WIL completion (FR-15 to FR-18).

If permission is granted before the final submission, a short task-based test and questionnaire will be run with volunteer students and staff, and the results will be added to this section.

### 5.8 Chapter Summary

TrackWIL was tested with 81 test cases: 14 manual tests, 62 automated system tests and 5 browser, code and data checks. Eighty passed and one, the usability test with students, is blocked until ethical permission is granted. Testing found seven defects, including two critical access-control vulnerabilities, and all seven were fixed and retested. All functional requirements and six of the seven non-functional requirements are met, and response times were far below the target. Within the limits of synthetic data and the absence of user feedback, the results show that TrackWIL supports the full WIL logbook workflow: recording daily work for each placement, keeping the WIL documents, account-free mentor verification and progress monitoring up to WIL completion. Chapter 6 draws conclusions from these results.

# CHAPTER 6 - CONCLUSION & RECOMMENDATIONS (OUTLINE)

## 6.1 Introduction

- Restate the problem: daily WIL logbooks are managed through paper, PDF and email, so records are lost, verification is slow and staff cannot see who is behind.
- Restate the aim: design, develop and evaluate TrackWIL, a web application for daily logbooks, account-free mentor verification and progress monitoring.

## 6.2 Summary of Findings

- The core workflow works end to end: placements, entries, evidence, documents, one-time mentor link, verification, hour totals, WIL completion and the monitoring dashboard (Chapter 5: 80 of 81 test cases passed; 1 blocked).
- Account-free verification through a single-use, 7-day link is practical and secure when the token is long and random, used only once and protected by a transaction.
- Security testing found two critical access-control defects (a forgeable session cookie and page data sent in redirect responses) that the functional tests had missed; both were fixed and retested.
- Performance is far below the 2-second target (average 3 to 5 ms per page locally).
- Usability has not yet been measured with real users because ethical permission is pending.

## 6.3 Achievement of Objectives

Table 24: Achievement of objectives (to be expanded)

| Objective | Status | Evidence |
|---|---|---|
| 1. Analyse the current WIL documentation process and its challenges | Achieved | Chapters 1 and 2; requirements gathering in Chapter 3 |
| 2. Design a logbook system with account-free mentor verification | Achieved | Chapter 4: architecture, UML, ERD, UI design |
| 3. Develop the student logbook, mentor page and shared dashboard | Achieved | Working application; Chapter 4.7; FR-01 to FR-18 met |
| 4. Implement secure storage and status tracking | Achieved | SQLite schema with constraints and transactions; TC-136, TC-140, TC-143 |
| 5. Test and evaluate TrackWIL against its requirements | Largely achieved | Chapter 5; usability test with users outstanding |

## 6.4 Answers to Research Questions

- **RQ1 (challenges of the current process):** lost or unreturned documents, delays waiting for signatures, no single view of progress, and extra work for external mentors (Chapters 1 and 2).
- **RQ2 (digital workflow with link-based verification):** a one-time link lets the mentor verify in two clicks without an account, and the status and time are recorded immediately (TC-127 to TC-137).
- **RQ3 (role-specific views):** students see their own placements, logbooks, hours and documents, mentors see only the one entry in their link, and staff see every student's status and profile (FR-03, FR-11, FR-17).

- **RQ4 (central database):** one record per entry with its status and timestamps removes conflicting copies and makes outstanding work visible (Chapter 4.4; TC-139, TC-140).
- **RQ5 (extent of improvement):** all functional requirements and six of seven non-functional requirements are met; the efficiency gain for real users still needs to be measured with the timed usability test.

### 6.5 Limitations

- Synthetic data only; no feedback from real students, mentors or staff yet (ethical permission pending).
- Usability (NFR-01) not measured with users.
- Response times measured locally, not on a hosted server.
- Mentors cannot view evidence files or leave written feedback; no email notifications.
- Uploaded documents are stored and can be viewed, but coordinators cannot yet accept them or ask for a new upload.
- A four-week build by a team of four limited the scope.

### 6.6 Recommendations

- Keep Next.js and the other dependencies up to date before any real use (DEF-04 was fixed by upgrading to 16.4.3).
- Run the timed usability test and questionnaire once permission is granted, and use the results to refine the forms.
- Host the application with HTTPS, set TRACKWIL_SESSION_SECRET and back up the database daily before using real data.
- Agree data-retention and consent rules with the campus in line with POPIA.

### 6.7 Future Work

- Written mentor feedback on rejected entries, and secure mentor access to evidence through the link.
- Automatic email notifications for verification requests and expiring links.
- Coordinator review of uploaded documents (accept, or ask the student for a new upload).
- Reports and data export for audits; an audit log of changes.
- Migration from SQLite to PostgreSQL for a multi-user hosted deployment.

### 6.8 Final Conclusion

- TrackWIL shows that a small, focused web application can replace the paper and email logbook process with a secure, traceable workflow.
- Its main contribution is account-free mentor verification combined with automatic progress monitoring.
- The project met its functional objectives; its effect on real users is the main question for the next stage.

## References


Beck, K. et al. (2001) Manifesto for Agile Software Development. Available at: https://agilemanifesto.org/ (Accessed: 22 October 2026).

Hains-Wesson, R. et al. (2026) [TO COMPLETE: full title, journal, volume, issue, pages and DOI or URL of the systematic literature review on placement supervision cited in Section 2.2].

InPlace Software (n.d.) Work Integrated Learning Placement Software. Available at: https://inplacesoftware.com/higher-education/work-integrated-learning/ (Accessed: 9 October 2026).

Mahara Project (n.d.) About ePortfolios. Available at: https://mahara.org/view/view.php?id=3 (Accessed: 9 October 2026).

Mell, P. and Grance, T. (2011) The NIST Definition of Cloud Computing. NIST Special Publication 800-145. Gaithersburg, MD: National Institute of Standards and Technology. Available at: https://doi.org/10.6028/NIST.SP.800-145 (Accessed: 9 October 2026).

OpenJS Foundation (n.d.) Node.js documentation: SQLite. Available at: https://nodejs.org/api/sqlite.html (Accessed: 22 October 2026).

OWASP Foundation (n.d.a) Password Storage Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (Accessed: 22 October 2026).

OWASP Foundation (n.d.b) Authorization Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html (Accessed: 9 October 2026).

OWASP Foundation (n.d.c) File Upload Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html (Accessed: 9 October 2026).

PebblePad (2026) PebblePad for Assessors. Available at: https://helphub.pebblepad.com/support/solutions/articles/101000539607-pebblepad-for-assessors (Accessed: 9 October 2026).

Republic of South Africa (2013) Protection of Personal Information Act 4 of 2013. Pretoria: Government Printer.

Schuster, L. and Glavas, C. (2017) 'Exploring the dimensions of electronic work integrated learning (eWIL)'. Available at: https://www.sciencedirect.com/science/article/abs/pii/S1747938X17300155 (Accessed: 9 October 2026).

Schwaber, K. and Sutherland, J. (2020) The Scrum Guide. Available at: https://scrumguides.org/scrum-guide.html (Accessed: 22 October 2026).

Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson.

Sonia Online (n.d.) [TO COMPLETE: page title and URL of the Sonia Online placement management website cited in Sections 2.2 and 2.3] (Accessed: 9 October 2026).

SQLite (n.d.) About SQLite. Available at: https://www.sqlite.org/about.html (Accessed: 22 October 2026).

Vercel (n.d.) Next.js Documentation. Available at: https://nextjs.org/docs (Accessed: 22 October 2026).

# Appendices

## Appendix A: Source Code and How to Run TrackWIL

The source code is in the folder 05_Source_Code/trackwil of the repository https://github.com/Noobey32/TrackWIL. To run it, install Node.js 22.13 or newer, then run npm install and npm run dev in that folder and open http://localhost:3000. The database is created and filled with synthetic data automatically.

Table 25: Demonstration accounts (synthetic data)

| Role | Email | Password |
|---|---|---|
| Student | student1@ctucareer.co.za | student123 |
| Student | student2@ctucareer.co.za | student123 |
| Student (WIL completed) | student3@ctucareer.co.za | student123 |
| Coordinator | coordinator@ctucareer.co.za | coord123 |
| Lecturer | lecturer@ctucareer.co.za | lecturer123 |

## Appendix B: Supporting Documents

Table 26: Supporting documents in the repository

| Document | Location |
|---|---|
| Software Requirements Specification | 03_Requirements_and_Methodology |
| Database Design document, UML and ER diagrams, UI screenshots | 04_System_Design_and_Development |
| Test Plan, Test Cases and Results, Defect Log, Requirements Traceability Matrix, test evidence | 06_Testing_Results_and_Evaluation |
| Project Charter, Sprint Backlog, Risk, Issue and Change Register, Weekly Progress Reports | 09_Project_Management_Evidence |
| Demonstration script and slide outline | 11_Presentation_and_Demonstration |
| Individual Contribution Forms | 12_Assessment |

## Appendix C: Mobile Test Evidence

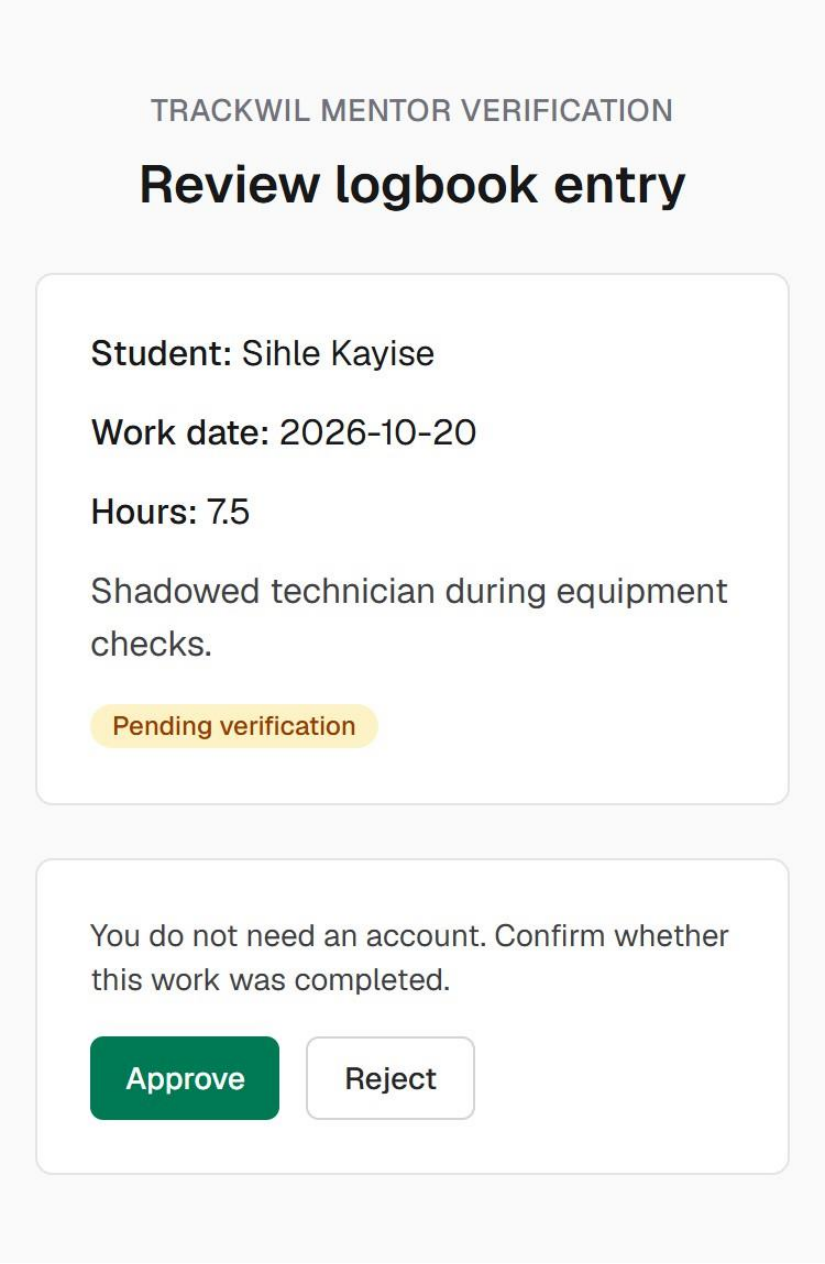



Figure 19: Mentor verification page after the DEF-03 fix (TC-145)

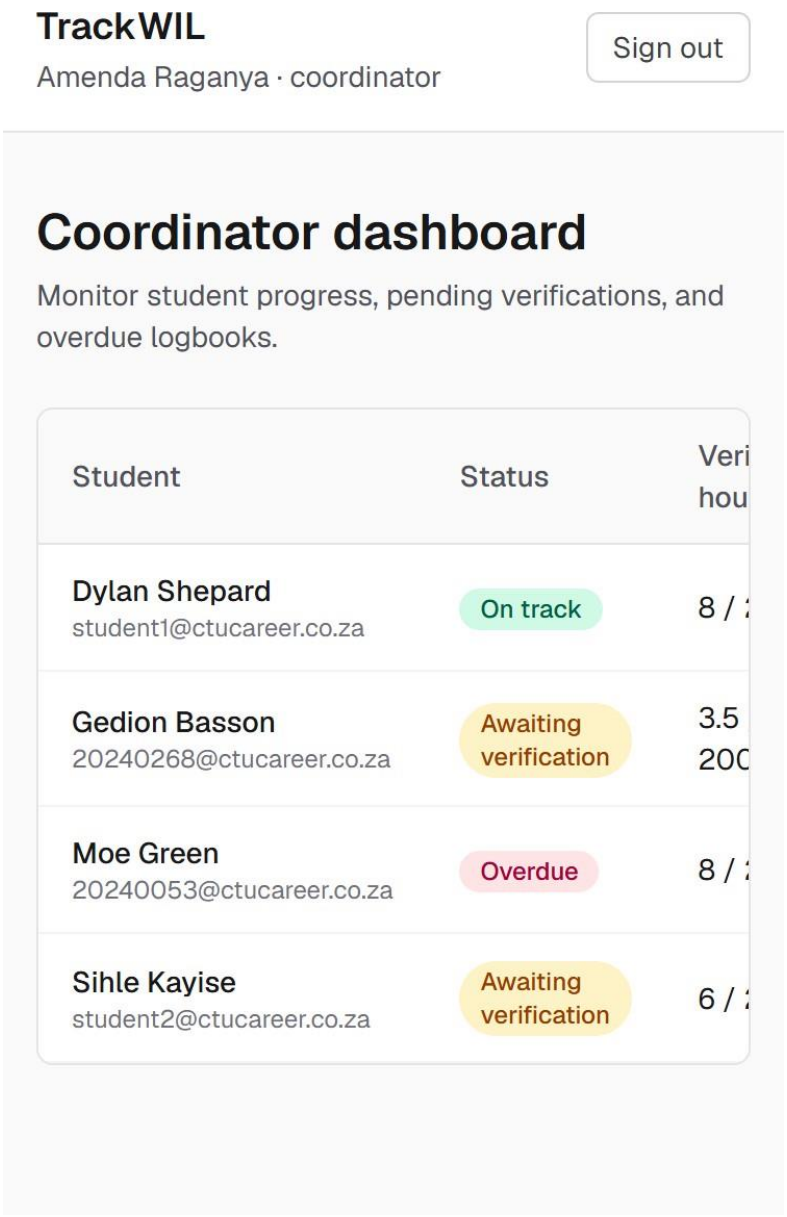



Figure 20: Coordinator dashboard after the DEF-03 fix: the table scrolls inside its box (TC-145)

# SOFTWARE TEST PLAN

## 1. Test Objectives

The purpose of testing is to show that TrackWIL works correctly, meets the requirements in the Software Requirements Specification (SRS) and is ready for the live demonstration. Testing must demonstrate that:

- students, coordinators and lecturers can sign in and are sent to the correct dashboard, and invalid sign-ins are refused (FR-01 to FR-03);
- students can create entries, attach evidence and submit them for verification (FR-04 to FR-06);
- workplace mentors can approve or reject an entry through a one-time link without an account, and used, expired or unknown links are refused (FR-07, FR-08);
- statuses, verified hours and the monitoring dashboard are correct (FR-09 to FR-11);
- timestamps, evidence protection and sign-out work (FR-12 to FR-14);
- students can add WIL placements, upload their WIL documents and see a completed placement locked, and coordinators and lecturers can open each student's profile (FR-15 to FR-18);
- the non-functional requirements for performance, security, integrity, maintainability, compatibility and privacy are met (NFR-02 to NFR-07), and usability is measured when permission allows (NFR-01);
- defects are recorded, fixed and retested before the final submission.

## 2. Scope

**Included:** every functional requirement FR-01 to FR-18 and every non-functional requirement NFR-01 to NFR-07 in the SRS, tested through the user interface, the Server Actions and the database.

**Excluded:** load testing with many simultaneous users; formal penetration testing by an external party; browsers other than current Chrome and Edge; features deferred to future versions (mentor accounts, written feedback, coordinator review of documents, notifications, reports); testing with real personal data.

## 3. Test Environment

*Table 1: Test environment*

| Item | Details |
|---|---|
| Hardware | Windows laptop used for development |
| Operating system | Microsoft Windows 11 Pro |
| Runtime | Node.js 24.21 |
| Application | TrackWIL production build (npm run build, then npm start) |
| Database | SQLite (data/trackwil.db), reset to the synthetic seed data with npm run db:reset before each run |

| Item | Details |
|---|---|
| Browsers | Google Chrome and Microsoft Edge |
| Test tools | Manual testing in the browser; automated system tests (tests/system-tests.mjs, npm run test:system); browser DevTools device mode |
| Test data | Synthetic accounts: student1, student2 and student3@ctucareer.co.za (student123), coordinator@ctucareer.co.za (coord123), lecturer@ctucareer.co.za (lecturer123); seeded placements (two of them completed), entries in every status, one expired link and synthetic demo documents |

## 4. Test Types

*Table 2: Test types used*

| Type | Purpose | Test cases |
|---|---|---|
| Functional (manual) | Check the main user journeys through the interface | TC-001 to TC-014 |
| System (automated) | Check every functional requirement end to end, including the database | TC-110 to TC-141, TC-150 to TC-175 |
| Security | Access control, forged cookies, evidence protection, password storage, link misuse | TC-117 to TC-121, TC-126, TC-133 to TC-136, TC-142, TC-161, TC-162, TC-168, TC-171, TC-173 |
| Validation | Required fields, hour limits, file types, password and email rules | TC-012, TC-114 to TC-116, TC-123, TC-125, TC-128, TC-151, TC-160 |
| Integrity | Database constraints and transactions | TC-136, TC-143 |
| Performance | Response times against the 2-second target | TC-144 |
| Compatibility | Chrome and Edge, phone and desktop widths | TC-145, TC-146 |
| Maintainability and privacy | Type checking, linting, README, stored data | TC-147, TC-148 |
| Usability | Timed task with first-time students (with consent) | TC-149 |
| Regression | Rerun the full automated suite after every fix | All automated tests |

## 5. Entry and Exit Criteria

**Testing can begin when:**

- the features to be tested are merged into a branch and the production build succeeds;

- the database has been reset to the seed data;
- test accounts and test data are available;
- the test cases, with their expected results, are written in the Test Cases and Results workbook.

**Testing is complete when:**

- every planned test case has been run, or its Blocked status is explained;
- actual results and Pass, Fail or Blocked statuses are recorded with the tester, date and evidence;
- every defect is in the Defect Log; all Critical and High defects that affect users are fixed and retested;
- the full automated suite passes on the final build;
- the Requirements Traceability Matrix links every requirement to its test cases and result;
- the results are summarised in Chapter 5.

## 6. Roles and Responsibilities

*Table 3: Testing roles*

| Role | Member | Responsibilities |
|---|---|---|
| Test lead and QA | Dylan Shepard | Reviews test cases and results; checks that requirements are covered; confirms the exit criteria |
| Manual tester | Sihle Kayise | Runs and records the manual test cases (TC-001 to TC-014); captures screenshots |
| Automated testing and fixes | Gedion Basson | Writes and runs the automated system tests; fixes and retests defects; keeps the workbooks up to date |
| Developer | Moegammad Green | Investigates and fixes assigned defects; supports retesting |
| Whole team | All members | Reviews the results and agrees whether the system is ready for the demonstration |

## 7. Defect Management

1. **Record:** each defect gets an ID (DEF-xx), the date, the feature, a description with steps to reproduce, the severity (Low, Medium, High or Critical) and the priority.
2. **Reproduce:** the defect is confirmed on the affected build before any fix is made.
3. **Assign:** the defect is assigned to a member, who fixes it on a branch named name/task.
4. **Fix:** the fix is committed with a message that names the defect and is merged through a pull request.
5. **Retest:** the original test case is run again and the full automated suite is rerun (regression).
6. **Close or reopen:** if the retest passes, the defect is closed with the commit and test-case reference; otherwise it is reopened.

## 8. Schedule

*Table 4: Test schedule*

| Date | Activity |
|---|---|
| 22 October 2026 | Manual functional testing (TC-001 to TC-014) |
| 22 October 2026 | Database migration verified with 28 automated checks |
| 23 October 2026 | Automated system tests, security review, browser and mobile checks; DEF-01 to DEF-03 fixed and retested |
| 24 to 27 October 2026 | DEF-04 and DEF-05 fixes; usability test if permission is granted; regression run |
| 26 October 2026 | New features tested (placements, documents, student profile, WIL completion; TC-150 to TC-175); DEF-04 to DEF-07 fixed and retested; full regression run of the 62 automated tests |
| 28 October 2026 | Final regression run on the submission build |

Type text here

## SOFTWARE TEST CASES & RESULTS

| Test ID | Requirement ID | Feature | Precondition | Test Steps | Test Data | Expected Result | Actual Result | Status | Tester | Date | Evidence |
|---|---|---|---|---|---|---|---|---|---|---|---|
| TC-001 | FR-01 | Student Login | Student account exists and system is online | 1. Open login page. 2. Enter valid student credentials. 3. Click Login. | Valid student username and password | Student should successfully log in and be redirected to dashboard. | Student logged in successfully and dashboard displayed. | Pass | Sihle Kayise | 2026/10/22 | Manual test in Chrome (screen check) |
| TC-002 | FR-10 | Student Dashboard | Student is logged in | 1. Log in as student. 2. View dashboard. | Student account | Dashboard should load and all functions should be accessible. | Dashboard appeared and worked correctly. | Pass | Sihle Kayise | 2026/10/22 | Manual test in Chrome (screen check) |
| TC-003 | FR-09 | View Existing Logbook Entry | Student has an existing logbook entry | 1. Open dashboard. 2. Click on an existing entry date. | Existing logbook entry | Selected entry details should open successfully. | Entry displayed successfully. | Pass | Sihle Kayise | 2026/10/22 | Manual test in Chrome (screen check) |
| TC-004 | FR-09, FR-10 | Back to Logbook Navigation | Student is viewing a logbook entry | 1. Open an entry. 2. Click "Back to Logbook". | Existing entry | User should be redirected to Logbook Dashboard. | Redirected successfully to Logbook Dashboard. | Pass | Sihle Kayise | 2026/10/22 | Manual test in Chrome (screen |
| TC-005 | FR-04 | Create New Logbook Entry | Student is logged in | 1. Click "New Entry". 2. Complete all fields. 3. Submit entry. | Logbook information (date, activities, hours worked, comments) | New entry form should open and accept user input. | New entry created successfully and all fields were functional. | Pass | Sihle Kayise | 2026/10/22 | Manual test in Chrome (screen check) |
| TC-006 | FR-04, FR-10 | Save Logbook Entry | New logbook entry has been completed | 1. Fill in entry form. 2. Click "Save Entry". | Completed logbook details | Entry should save successfully and appear on dashboard. | Entry saved successfully and displayed on logbook dashboard. | Pass | Sihle Kayise | 2026/10/22 | Manual test in Chrome (screen check) |
| TC-007 | FR-14 | Student Sign Out | Student is logged in | 1. Click "Sign Out". | N/A | User should be logged out and redirected to login page. | Redirected to login page successfully. | Pass | Sihle Kayise | 2026/10/22 | Manual test in Chrome (screen check) |
| TC-008 | FR-01 | Coordinator Login | Coordinator account exists | 1. Open login page. 2. Enter valid coordinator credentials. 3. Click Login. | Valid coordinator username and password | Coordinator should successfully log in. | Coordinator signed in successfully. | Pass | Sihle Kayise | 2026/10/22 | Manual test in Chrome (screen check) |
| TC-009 | FR-11 | Coordinator Dashboard | Coordinator is logged in | 1. Log in as coordinator. 2. View dashboard. | Uploaded student logbook entries | Dashboard should display students who submitted entries. | Submitted student entries were visible. | Pass | Sihle Kayise | 2026/10/22 | Manual test in Chrome (screen check) |
| TC-010 | FR-14 | Coordinator Sign Out | Coordinator is logged in | 1. Click "Sign Out". | N/A | User should be redirected to login page. | Redirected successfully to login page. | Pass | Sihle Kayise | 2026/10/22 | Manual test in Chrome (screen check) |
| TC-011 | FR-01 | Student Login with Invalid Credentials | Student login page is accessible | 1. Open login page. 2. Enter invalid username and/or password. 3. Click Login. | Invalid usemame/password | System should display an error message and deny access. | Error message displayed and login was unsuccessful. | Pass | Sihle Kayise | 2026/10/22 | Manual test in Chrome (screen check) |
| TC-012 | FR-04 | Save Logbook Entry with Required Fields Empty | Student is logged in and on the New Entry page | 1. Click "New Entry". 2. Leave one or more required fields blank. 3. Click "Save Entry". | Incomplete logbook entry | System should validate input and prevent saving until all required fields are completed. | Validation message displayed and entry was not saved. | Pass | Sihle Kayise | 2026/10/22 | Manual test in Chrome (screen check) |
| TC-013 | FR-01 | Coordinator Login with Invalid Credentials | Coordinator login page is accessible | 1. Enter incorrect login credentials. 2. Click Login. | Invalid coordinator usemame/password | System should display an error message and deny access. | Error message displayed and login was unsuccessful. | Pass | Sihle Kayise | 2026/10/22 | Manual test in Chrome (screen check) |
| TC-014 | FR-04, NFR-04 | Verify Saved Data After Logout and Login | Student has previously saved a logbook entry | 1. Create and save a logbook entry. 2. Sign out. 3. Sign back in. 4. Open the logbook dashboard. | Previously saved logbook entry | Saved entry should still be available on the dashboard. | Saved entry remained visible after logging back in. | Pass | Sihle Kayise | 2026/10/22 | Manual test in Chrome (screen check) |
| TC-110 | FR-01 | Sign in (student) | Seeded student account exists | 1. Send the sign-in form with the email in capital letters and the correct password. | STUDENT2@ctucareer.co.za / student123 | Signed in; session cookie set; redirected to /student. | redirect=/student; session cookie set | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |
| TC-111 | FR-01 | Sign in with wrong password | Seeded student account exists | 1. Send the sign-in form with a wrong password. | student2@ctucareer.co.za / wrong-password | Message 'Invalid email or password.'; no session cookie. | message="Invalid email or password."; cookie not set | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |
| TC-112 | FR-01, FR-03 | Sign in (coordinator and lecturer) | Seeded staff accounts exist | 1. Sign in as coordinator. 2. Sign in as lecturer. | coordinator@ / coord123; lecturer@ / lecturer123 | Both are redirected to /coordinator. | coordinator -> /coordinator; lecturer -> /coordinator | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |
| TC-113 | FR-02 | Sign-up (valid) | Email not yet registered | 1. Send the sign-up form with a valid CTU student email and a strong password. | <year>0199@ctucareer.co.za / Strong#Pass1 | Student account created; signed in; redirected to /student. | redirect=/student; user row role=student | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |

| Test ID | Requirement ID | Feature | Precondition | Test Steps | Test Data | Expected Result | Actual Result | Status | Tester | Date | Evidence |
|---|---|---|---|---|---|---|---|---|---|---|---|
| TC-114 | FR-02 | Sign-up password rules | Sign-up page open | 1. Try five passwords that each break one rule. | short1!, lowercase1!, UPPERCASE1!, NoNumbers!!, NoSpecial12 | Each attempt is rejected with the message for the broken rule. | short1! -> "Password must be at least 8 characters."; lowercase1! -> "Password must include at least one uppercase letter."; UPPERCASE1! -> "Password must include at least one lowercase letter."; NoNumbers!! -> "Password must include at least one number."; NoSpecial12 -> "Password must include at least one special character." | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_20 26-10-23.txt |
| TC-115 | FR-02 | Sign-up email rules | Sign-up page open | 1. Try a Gmail address. 2. Try a malformed CTU address. | someone@gmail.com; 1999@ctucareer.co.za | Both rejected with 'Please use a valid CTU email.' | gmail -> "Please use a valid CTU email."; 1999@ctucareer.co.za -> "Please use a valid CTU email." | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_20 26-10-23.txt |
| TC-116 | FR-02 | Sign-up duplicate email | TC-113 account exists | 1. Sign up again with the same email in capital letters. | Same email, upper case | Rejected with 'An account with this email already exists.' | message="An account with this email already exists." | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_20 26-10-23.txt |
| TC-117 | FR-03 | Signed-out access | Not signed in | 1. Open /student. 2. Open /coordinator. | No cookie | Both redirect (307) to the sign-in page. | /student -> 307 /; /coordinator -> 307 / | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_20 26-10-23.txt |
| TC-118 | FR-03 | Coordinator blocked from student area | Signed in as coordinator | 1. Open /student. | Coordinator session | Redirected to /coordinator; no student page content in the response. | status 307; redirected to /coordinator: true; student page content in response: false | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-119 | FR-03 | Student blocked from dashboard | Signed in as student | 1. Open /coordinator. | Student session | Redirected to /student; no dashboard content in the response. | status 307; redirected to /student: true; dashboard content in response: false | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-120 | FR-03 | Student cannot view another student's entry | Signed in as Sihle | 1. Open /student/entries/entry-1 (Dylan's entry). | entry-1 | 404 Not Found; entry text not shown. | status 404; entry text shown: false | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_20 26-10-23.txt |
| TC-120b | FR-03 | Student cannot submit another student's entry | Signed in as Sihle | 1. Call Submit for verification on entry-1. | entry-1 | Rejected with 'Entry not found.' | message="Entry not found." | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_20 26-10-23.txt |
| TC-121 | FR-03, NFR-03 | Forged / tampered session cookie (DEF-01 retest) | DEF-01 fix deployed | 1. Open /coordinator with cookie 'coordinator-1'. 2. Open it with 'coordinator-1' plus another user's signature. | Forged cookies | Both rejected; no dashboard content in the response. | unsigned 'coordinator-1' -> status 307, dashboard content in response: false; another user's signature -> status 307, dashboard content in response: false | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-122 | FR-04 | Create entry | Signed in as student | 1. Submit the new-entry form with valid values. | 2026-10-22, description, 4.5 h, mentor email | Entry saved with status Draft and opened. | redirect=/student/entries/631b4788-c53e-4eb8-8da2-aeca1b8c00b5; status=draft; hours=4.5 | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_20 26-10-23.txt |
| TC-123 | FR-04 | Entry validation | Signed in as student | 1. Try hours 30, 0 and -2. 2. Try an empty description. | Invalid hours / empty description | Each attempt rejected with a message; nothing saved. | hours=30 desc=yes -> "Hours for one day cannot be more than 24."; hours=0 desc=yes -> "Work date, description, and positive hours are required."; hours=-2 desc=yes -> "Work date, description, and positive hours are required."; hours=5 desc=empty -> "Work date, description, and positive hours are required." | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_20 26-10-23.txt |
| TC-124 | FR-05 | Evidence upload (PNG) | Signed in as student | 1. Create an entry with a PNG file attached. | photo.png | Entry saved; file stored under a random UUID name. | stored as e2f009fa-ad72-418b-99a5-ffad6f2bf8c9.png | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_20 26-10-23.txt |
| TC-125 | FR-05 | Evidence upload (blocked type) | Signed in as student | 1. Create an entry with an .exe file attached. | virus.exe | Rejected with 'Evidence must be a PDF or image file.' | message="Evidence must be a PDF or image file." | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_20 26-10-23.txt |
| TC-126 | FR-13 | Evidence download access | TC-124 file exists | 1. Download as owner, another student, signed-out visitor and coordinator. 2. Try a path-traversal file name. | Stored evidence file | Owner 200, other student 403, signed out 401, coordinator 200, traversal 404. | owner 200; other student 403; signed out 401; coordinator 200; path traversal attempt 404 | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_20 26-10-23.txt |
| TC-127 | FR-06 | Submit for verification | Draft entry with mentor email | 1. Submit the entry for verification. | TC-122 entry | 48-character link valid for 7 days; status Pending verification. | token length=48; status=pending_verification; expires after 7.00 days | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_20 26-10-23.txt |
| TC-128 | FR-06 | Submit without mentor email | Draft entry without mentor email | 1. Submit the entry for verification. | Entry with empty mentor email | Rejected with 'Add a mentor email ...'. | message="Add a mentor email on the entry before submitting." | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_20 26-10-23.txt |

| Test ID | Requirement ID | Feature | Precondition | Test Steps | Test Data | Expected Result | Actual Result | Status | Tester | Date | Evidence |
|---|---|---|---|---|---|---|---|---|---|---|---|
| TC-129 | FR-06 | Resubmit with active link | Entry pending with a valid link | 1. Submit the same entry again. | TC-127 entry | Rejected; still only one link for the entry. | message="Only draft or rejected entries, or entries whose link has expired, can be submitted."; tokens for entry=1 | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |
| TC-130 | FR-07 | Mentor page without sign-in | Valid link from TC-127 | 1. Open /verify/<token> without a session. | Token from TC-127 | Page shows student, date, hours, description and Approve / Reject. | status 200; shows student name, description and Approve/Reject buttons | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |
| TC-131 | FR-07, FR-12 | Mentor approves | Valid link | 1. Click Approve. | Token from TC-127 | Entry Verified; verification time stored. | status=verified; verified_at=2026-10-22T22:54:02.000Z | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |
| TC-132 | FR-07, FR-10 | Mentor rejects; student resubmits | New entry submitted | 1. Mentor rejects. 2. Student submits again. | New test entry | Status Rejected, then a new link and Pending verification. | after reject: rejected; resubmitted with new link: true; status now pending_verification | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |
| TC-133 | FR-08 | Reuse a used link | Link used in TC-131 | 1. Call Reject with the used link. 2. Open the link page. | Used token | Refused 'already been used'; page says used; entry still Verified. | action -> "This verification link has already been used."; page shows "already used": true; status still verified | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |
| TC-134 | FR-08 | Unknown link | None | 1. Call Approve with an unknown token. 2. Open its page. | 48 zeros | Refused 'invalid'; page 404. | action -> "This verification link is invalid."; page status 404 | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |
| TC-135 | FR-08 | Expired link | Seeded link expired 3 days ago | 1. Call Approve with the expired token. 2. Open its page. | Seeded expired token (entry-7) | Refused 'expired'; page shows expiry; entry unchanged. | action -> "This verification link has expired."; page mentions expiry: true; status pending_verification | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |
| TC-136 | FR-08, NFR-04 | Double click on Approve | Fresh valid link | 1. Send two Approve requests at the same time. | Token from TC-132 | Exactly one decision recorded; the other refused. | successful decisions=1; second response "This verification link has already been used."; token decision=approved | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |
| TC-137 | FR-06, FR-08 | Resubmit after link expired (DEF-02 retest) | DEF-02 fix deployed; entry pending with an expired link | 1. Open the entry. 2. Submit again. | Entry whose link expired | Expiry notice and Submit button shown; new link issued. | page shows expiry notice and Submit button: true; resubmitted with new link: true; status pending_verification | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |
| TC-138 | FR-09, FR-10 | Placement logbook view | Signed in as Sihle | 1. Open /student/placements/placement-sihle-1. | Sihle's entries | Statuses shown; 'X of 200 verified hours completed' correct for the placement. | expected "12.5 of 200 verified hours"; statuses shown: Draft, Pending verification, Verified | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_2026-10-26.txt |
| TC-139 | FR-11 | Monitoring dashboard | Signed in as lecturer | 1. Open /coordinator. | Seed data | Dylan On track, Moe Overdue, Sihle Awaiting verification. | Dylan=On track; Moe=Overdue; Sihle=Awaiting verification; Gedion=Awaiting verification | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |
| TC-140 | FR-12 | Automatic timestamps | TC-131 completed | 1. Read the entry and link rows in the database. | TC-127 entry and link | created, verified and used times stored in ISO-8601; decision approved. | created 2026-10-22T22:54:01.841Z; verified 2026-10-22T22:54:02.000Z; link used 2026-10-22T22:54:02.000Z (approved) | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |
| TC-141 | FR-14 | Sign out | Signed in as student | 1. Click Sign out. 2. Open /student. | Student session | Cookie removed; redirected to sign-in; /student redirects. | redirect=/; cookie cleared: true; /student afterwards -> 307 | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |
| TC-142 | NFR-03 | Password storage | TC-113 account exists | 1. Read the password_hash column. | TC-113 account | Salt:key hash format; plain password not stored. | stored value format: <32 hex salt>:<128 hex key>; plain password present: false | Pass | Gedion Basson | 2026/10/23 | evidence/system_test_run_2026-10-23.txt |

| Test ID | Requirement ID | Feature | Precondition | Test Steps | Test Data | Expected Result | Actual Result | Status | Tester | Date | Evidence |
|---|---|---|---|---|---|---|---|---|---|---|---|
| TC-143 | NFR-04 | Database constraints | Fresh copy of schema.sql | 1. Insert eleven kinds of invalid rows. | Hours 30, bad status, entry for a missing placement, placement with 0 hours, placement for a missing student, bad role, duplicate email, unknown document type, register without a semester, second MOU, token without decision | Every insert rejected by a constraint. | hours = 30: rejected (CHECK constraint failed); unknown status: rejected (CHECK constraint failed); entry for missing placement: rejected (FOREIGN KEY constraint failed); placement with 0 required hours: rejected (CHECK constraint failed); placement for missing student: rejected (FOREIGN KEY constraint failed); unknown role: rejected (CHECK constraint failed); duplicate email (case): rejected (UNIQUE constraint failed); document with an unknown type: rejected (CHECK constraint failed); attendance register without a semester: rejected (CHECK constraint failed); second MOU for one placement: rejected (UNIQUE constraint failed); token used without decision: rejected (CHECK constraint failed) | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_2026-10-26.txt |
| TC-144 | NFR-02 | Response times | Demo data loaded | 1. Request seven pages 10 times each. 2. Time one entry creation. | /, /student, placement logbook, /student/entries/new, /coordinator, student profile, /verify/<token> | Every response under 2 seconds and no page errors. | / avg 3 ms, max 5 ms; /student avg 4 ms, max 4 ms; /student/placements/placement-sihle-1 avg 5 ms, max 7 ms; /student/entries/new avg 4 ms, max 4 ms; /coordinator avg 5 ms, max 7 ms; /coordinator/students/student-1 avg 5 ms, max 5 ms; /verify/<token> avg 3 ms, max 5 ms; create entry 10 ms; pages with errors: none | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_2026-10-26.txt |
| TC-145 | NFR-06 | Mobile layout (375 px) | App running; Chrome 155 and Edge 155 | 1. Open sign-in, logbook, new entry, mentor page and dashboard at 375 x 812 px. | Demo data | Every screen fits the screen width; nothing cut off. | First run: logbook and dashboard tables made the page 580 px wide and cut off columns (DEF-03). Retest after fix: page width 375 px in Chrome and Edge; tables scroll inside their box. | Pass | Gedion Basson | 2026/10/23 | evidence/mobile_375px_chrome, evidence/mobile_375px_edge |
| TC-146 | NFR-06, FR-14 | Cross-browser sign in and sign out (Edge) | App running; Edge 155 | 1. Sign in as student in Edge. 2. Click Sign out. 3. Open /student. | student2@ctucareer.co.za / student123 | Same behaviour as Chrome. | After sign in: /student with session cookie. After sign out: / and no cookie. /student then redirects to /. | Pass | Gedion Basson | 2026/10/23 | Browser check (headless Edge 155.0.4283.62) |
| TC-147 | NFR-05 | Maintainability checks | Source code on branch gedion/fix-defects | 1. Run npx tsc --noEmit. 2. Run npm run lint. 3. Follow the README set-up steps. | Source code | No type or lint errors; set-up in five commands or fewer. | TypeScript: 0 errors. ESLint: 0 problems. Set-up: npm install, npm run dev (2 commands; database created automatically). | Pass | Gedion Basson | 2026/10/23 | README.md; terminal output |
| TC-148 | NFR-07 | Privacy review of stored data | Demo database | 1. Review users and entries stored by seed.sql. | database/seed.sql | Only synthetic data and team members' own names; no staff or other students' personal data. | Seed data contains only the four team members, one synthetic student (Lerato Mokoena), a synthetic coordinator and a synthetic lecturer, invented entries and placements, and synthetic demo PDFs marked as demonstration data; no other personal data is stored. | Pass | Gedion Basson | 2026/10/26 | database/seed.sql; database/demo-documents |

| Test ID | Requirement ID | Feature | Precondition | Test Steps | Test Data | Expected Result | Actual Result | Status | Tester | Date | Evidence |
|---|---|---|---|---|---|---|---|---|---|---|---|
| TC-149 | NFR-01 | Usability: first-time student creates and submits an entry in under 3 minutes | Written campus permission and participant consent | 1. Give a first-time student the task without help. 2. Time it. | Volunteer students | Task completed in under 3 minutes. | Not run: written campus permission for testing with students has not been received (Risk R01). Will be run if permission is granted before 29 October. | Blocked | | | Ethical Clearance Confirmation |
| TC-150 | FR-15 | Add a WIL placement | Signed in as a new student | 1. Send the add-placement form. | WIL 1, Test Host Company, 320 hours, 2026-10-01 | Placement saved for the student; redirected to its page. | redirect=/student/placements/348812c8-3305-4981-badc-0215b55718d8; title=WIL 1; company=Test Host Company; required hours=320 | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-151 | FR-15 | Placement validation | Signed in as a new student | 1. Send the form with 0, -10 and 6000 hours, no title, no company and no start date. | Invalid placement details | Each attempt rejected with a message; no placement saved. | 0 hours -> "Required hours must be a positive number."; -10 hours -> "Required hours must be a positive number."; 6000 hours -> "Required hours cannot be more than 5000."; no title -> "Title, company and start date are required."; no company -> "Title, company and start date are required."; no start date -> "Title, company and start date are required."; extra placements saved: 0 | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-152 | FR-03, FR-15 | Edit a placement | TC-150 placement exists | 1. Change own placement to 300 hours. 2. Try to edit Sihle's placement. | 300 hours; placement-sihle-1 | Own placement updated; the other student's placement is refused and unchanged. | own placement now 300 hours; editing Sihle's placement -> "Placement not found."; Sihle's hours still 200 | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-153 | FR-04, FR-15 | Entry on another student's placement | Signed in as a new student | 1. Create an entry on Sihle's placement. 2. Create one on an unknown placement. | placement-sihle-1; no-such-placement | Both rejected with a message. | Sihle's placement -> "Choose one of your WIL placements for this entry."; unknown placement -> "Choose one of your WIL placements for this entry." | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-154 | FR-03, FR-15 | Placement page access | Signed in as a new student | 1. Open Sihle's placement page. 2. Open own placement page. | placement-sihle-1; TC-150 placement | Other student's placement not shown; own placement shown. | Sihle's placement shown: false; own placement shown: true | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-155 | FR-10, FR-15 | Hours per placement | Signed in as Dylan | 1. Open My WIL placements. | Dylan's WIL 1 and WIL 2 | Each placement shows its own verified hours against its own required hours. | WIL 1 120 of 120 shown: true; WIL 2 8 of 200 shown: true | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-156 | FR-15 | New entry and edit pages (DEF-06 retest) | Student with a placement | 1. Open the new entry page. 2. Open the edit placement page. | TC-150 placement | Both pages load (200) and list the placement. | new entry page 200 (placement listed: true); edit page 200 (form shown: true) | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-157 | FR-16 | Upload an MOU | Student with a placement | 1. Upload a PDF as the MOU. | Signed MOU.pdf | Saved under a random name; the original name is shown. | stored as fff02138-1706-4f5a-a87d-6bb5af3d95e2.pdf; file on disk: true; name shown to users "Signed MOU.pdf" | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-158 | FR-16 | Replace a document | TC-157 MOU exists | 1. Upload another PDF as the MOU. | Signed MOU v2.pdf | Still one MOU; the old file is deleted. | MOU rows: 1; new file 303ca862-888c-47b7-ad65-d0b798b67e3f.pdf; old file removed: true | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-159 | FR-16 | Attendance register per semester | Placement that spans two semesters | 1. Upload a register for the current semester twice and for the first semester once. 2. Try invalid semesters. | 2026-S1, 2026-S2; 2026-S3, 2020-S1, 2099-S1, empty | One register per semester; invalid semesters rejected. | registers before: 0; after uploading 2026-S2 twice and 2026-S1 once: 2026-S1, 2026-S2; "2026-S3" -> "Choose a semester between the placement start and now."; "2020-S1" -> "Choose a semester between the placement start and now."; "2099-S1" -> "Choose a semester between the placement start and now."; "" -> "Choose a semester between the placement start and now." | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |

| Test ID | Requirement ID | Feature | Precondition | Test Steps | Test Data | Expected Result | Actual Result | Status | Tester | Date | Evidence |
|---|---|---|---|---|---|---|---|---|---|---|---|
| TC-160 | FR-16, NFR-03 | Invalid or oversized documents | Student with a placement | 1. Upload an .exe, a program renamed to .pdf, a PNG renamed to .pdf, an empty file, a 10.5 MB PDF and an unknown document type. | Six invalid uploads | Every upload rejected with a message; nothing saved. | .exe file -> "Documents must be PDF, PNG or JPG files."; program renamed to .pdf -> "Documents must be PDF, PNG or JPG files."; PNG renamed to .pdf -> "Documents must be PDF, PNG or JPG files."; empty file -> "Choose a file to upload."; 10.5 MB PDF -> "The file is larger than 10 MB."; unknown document type -> "Choose which document you are uploading."; code of conduct rows saved: 0 | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-161 | FR-03, FR-16 | Upload to another placement | Signed in as a new student; coordinator | 1. Upload to Sihle's placement as another student. 2. Upload as the coordinator. | placement-sihle-1 | Both refused; nothing saved. | other student's placement -> "Placement not found."; coordinator upload accepted: false; letters on Sihle's placement: 0 | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-162 | FR-16, NFR-03 | Document download access | TC-158 MOU exists | 1. Download as the owner, another student, signed out, the coordinator and the lecturer. 2. Request an unknown id. | Document id | 200 for the owner and staff (PDF), 403 for another student, 401 signed out, 404 for an unknown id. | owner 200 (application/pdf); other student 403; signed out 401; coordinator 200; lecturer 200; unknown id 404 | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-163 | FR-16 | Remove a document | TC-158 MOU exists | 1. Remove it as another student. 2. Remove it as the owner. | Document id | Other student refused; the owner removes the record and the file. | other student -> "Document not found." (document kept: true); owner -> row and file removed: true | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-164 | FR-16 | Document status on pages | Seed data | 1. Open Dylan's WIL 2, Sihle's WIL 1 and Dylan's placements. | Seeded documents | Uploaded and Missing shown correctly; the card shows documents 3 of 3. | Dylan WIL 2 complete (no Missing): true; Sihle shows Uploaded and Missing: true; placement card shows documents 3 of 3: true | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-165 | FR-16, NFR-02 | Large document upload | Student with a placement | 1. Upload a 9.5 MB PDF through the student page. | 9.5 MB PDF | Accepted; the stored size matches. | response: success; stored size 9961479 of 9961479 bytes | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-166 | FR-17 | Coordinator opens a student profile | Signed in as coordinator | 1. Open Dylan's profile. | student-1 | Both placements, logbook entries and document links shown; no upload, replace or remove buttons. | status 200; both placements: true; logbook entry shown: true; document link shown: true; read-only (no upload/replace/remove): true | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-167 | FR-17 | Lecturer opens a student profile | Signed in as lecturer | 1. Open Sihle's profile. | student-2 | Profile shown. | status 200; shows Sihle and her placement: true | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-168 | FR-03, FR-17 | Profile blocked for students | Signed in as Sihle | 1. Open Dylan's profile. | student-1 | Redirected; none of Dylan's data in the response. | status 307; Dylan's placement in the response to Sihle: false | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-169 | FR-17 | Unknown or staff profile | Signed in as coordinator | 1. Open the profile of the coordinator account and of an unknown id. | coordinator-1; no-such-student | 404 Not Found for both. | coordinator account -> 404; unknown id -> 404 | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-170 | FR-11, FR-17 | Dashboard links and missing documents | Signed in as lecturer | 1. Open the dashboard. | Seed data | Each student links to their profile; missing documents are shown. | profile links: true; Moe row shows missing: true; Sihle row shows missing: true | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-171 | FR-03, NFR-03 | Blocked pages send no data (DEF-07 retest) | DEF-07 fix deployed | 1. Request staff pages as a student and with a made-up cookie. 2. Request a student page as the coordinator. | Five requests | Each is redirected and none of the page's data is in the response. | student -> /coordinator: 307, leaked nothing; made-up cookie -> /coordinator: 307, leaked nothing; student -> another student's profile: 307, leaked nothing; made-up cookie -> student profile: 307, leaked nothing; coordinator -> a student's placement page: 307, leaked nothing | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |
| TC-172 | FR-18 | Automatic WIL completion | Student with a 4-hour placement | 1. Add a 4-hour entry, submit it and approve it as the mentor. 2. Open the placement, the dashboard and Dylan's profile. | 4-hour placement | Placement marked WIL completed after approval; New entry hidden; Lerato shown as WIL completed. | 4-hour placement before approval completed: false; after approval: true (New entry button hidden: true); Lerato on dashboard: WIL completed; Dylan's profile marks WIL 1 completed: true | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_20 26-10-26.txt |

| Test ID | Requirement ID | Feature | Precondition | Test Steps | Test Data | Expected Result | Actual Result | Status | Tester | Date | Evidence |
|---|---|---|---|---|---|---|---|---|---|---|---|
| TC-173 | FR-18 | Completed placement is locked | TC-172 placement completed | 1. Try a new entry, submitting a draft, uploading, editing, and removing a document of Dylan's WIL 1. | Completed placements | Every change refused with the locked message; nothing changed. | new entry -> "This WIL placement is completed and locked. Add a new placement to continue."; submit remaining draft -> "This WIL placement is completed and locked. Add a new placement to continue."; upload document -> "This WIL placement is completed and locked. Add a new placement to continue."; edit placement -> "This WIL placement is completed and locked. Add a new placement to continue."; remove document (Dylan, WIL 1) -> "This WIL placement is completed and locked. Add a new placement to continue."; entries still 2; draft still draft; required hours still 4; Dylan's MOU kept: true | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_2026-10-26.txt |
| TC-174 | FR-11, FR-15 | No placement status | New sign-up | 1. Open the dashboard. 2. Add a placement that starts today. 3. Open the dashboard again. | Fresh Student | No placement, then On track. | before adding a placement: No placement; after adding one that starts today: On track | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_2026-10-26.txt |
| TC-175 | FR-01 | Demo accounts listed (DEF-05 retest) | App running | 1. Open the sign-in page. | None | All five demo accounts are listed. | listed 5 of 5: student1@ctucareer.co.za, student2@ctucareer.co.za, student3@ctucareer.co.za, coordinator@ctucareer.co.za, lecturer@ctucareer.co.za | Pass | Gedion Basson | 2026/10/26 | evidence/system_test_run_2026-10-26.txt |

## DEFECT / BUG LOG

| Defect ID | Date | Feature | Description | Severity | Priority | Reported By | Assigned To | Status | Fix / Resolution | Retest Result | Evidence |
|---|---|---|---|---|---|---|---|---|---|---|---|
| DEF-01 | 22 Oct 2026 | Sign-in / sessions | The session cookie stored the user id without a signature. Setting the cookie to "coordinator-1" in a browser opened the coordinator dashboard without a password. Reproduced on build 7080e7f: HTTP 200 and the dashboard was shown. | Critical | High | Gedion Basson | Gedion Basson | Closed | The cookie is now <userId>.<HMAC-SHA256 signature> and is checked in constant time; the secret comes from TRACKWIL_SESSION_SECRET or is generated once into data/session-secret. Commit 5268c2e. | Pass: TC-121 (forged and tampered cookies rejected); TC-110 to TC-119 still pass | evidence/system_test_run_2026-10-23.txt |
| DEF-02 | 22 Oct 2026 | Verification links | If a mentor link expired before the mentor responded, the entry stayed Pending verification with no Submit button, and the server refused to resubmit it ("Only draft or rejected entries can be submitted."). Reproduced on build 7080e7f. | Medium | Medium | Gedion Basson | Gedion Basson | Closed | A pending entry with no active link can be submitted again for a new link; the entry page shows an expiry notice and the Submit button. Commit 5268c2e. | Pass: TC-137; TC-129 confirms an entry with an active link still cannot be resubmitted | evidence/system_test_run_2026-10-23.txt |
| DEF-03 | 23 Oct 2026 | Responsive layout | On a 375 px phone screen the logbook and dashboard tables made the page 580 px wide and cut off the right-hand columns with no way to scroll. | Medium | Medium | Gedion Basson | Gedion Basson | Closed | The page container is now full width and each table scrolls inside its own box (overflow-x-auto). Commit 5268c2e. | Pass: TC-145 (page width 375 px in Chrome 155 and Edge 155) | evidence/mobile_375px_chrome; evidence/mobile_375px_edge |
| DEF-04 | 22 Oct 2026 | Dependencies | The production build reports that Next.js 16.4.0 is affected by a known security vulnerability. | High | Medium | Gedion Basson | Gedion Basson | Closed | Upgraded Next.js from 16.4.0 to 16.4.3 (eslint-config-next to match) on branch gedion/nextjs-security-upgrade. Commit 992573b, pull request #16. | Pass: the build no longer reports the advisory; npm audit finds 0 vulnerabilities; all 62 automated tests pass | evidence/system_test_run_2026-10-26.txt |
| DEF-05 | 23 Oct 2026 | Sign-in page | The demo accounts box on the sign-in page lists the student and coordinator accounts but not the lecturer account. | Low | Low | Gedion Basson | Gedion Basson | Closed | The demo accounts box lists all five demo accounts, including the lecturer. Branch gedion/documents-docs-update. | Pass: TC-175 | evidence/system_test_run_2026-10-26.txt; screenshots/01_login.png |
| DEF-06 | 26 Oct 2026 | New entry / edit placement pages | Opening the new entry page or the edit placement page showed an error screen: node:sqlite returns rows without a prototype, and React refuses to pass them from a Server Component to a Client Component. Reproduced on build 4c0ff41: HTTP 500. | Medium | High | Gedion Basson | Gedion Basson | Closed | The repository copies every row into a plain object; the timing test now also fails on any page error. Commit 2f3037d, pull request #12. | Pass: TC-156 (both pages 200); TC-144 reports no page errors | evidence/system_test_run_2026-10-26.txt |
| DEF-07 | 26 Oct 2026 | Access control | A role check only in an area's layout redirected the browser, but Next.js still rendered the page and sent its data in the body of the 307 response: a signed-in student, or anyone with a made-up session cookie, could read every student's name, email and hours from /coordinator. Reproduced on build 96ef01e. | Critical | High | Gedion Basson | Gedion Basson | Closed | Every protected page and layout calls requireRole() before it loads any data. Commit 899f600, pull request #14. | Pass: TC-171; TC-118, TC-119 and TC-121 now check the response body (they failed on the earlier build) | evidence/system_test_run_2026-10-26.txt |

## REQUIREMENTS TRACEABILITY MATRIX

| Requirement ID | Requirement Description | Type | Priority | Design / Component | Implementation Evidence | Test Case ID | Test Result | Final Status |
|---|---|---|---|---|---|---|---|---|
| FR-01 | Sign in with email and password and go to the dashboard for the user's role. | Functional | Must | UC-01; sign-in page; auth Server Action | src/app/actions/auth.ts (loginAction); src/lib/password.ts; src/lib/session.ts | TC-001, TC-008, TC-011, TC-013, TC-110, TC-111, TC-112 | Pass | Met |
| FR-02 | Student sign-up with a CTU student email and a strong password. | Functional | Should | UC-01; sign-up page | src/app/actions/auth.ts (signupAction); src/lib/signup-validation.ts | TC-113, TC-114, TC-115, TC-116 | Pass | Met |
| FR-03 | Restrict each page and action to the roles allowed to use it. | Functional | Must | Access control layer (proxy, layouts, page-level role checks, session) | src/proxy.ts; src/lib/session.ts (requireRole on every protected page and layout); ownership checks in actions | TC-112, TC-117, TC-118, TC-119, TC-120, TC-120b, TC-121, TC-154, TC-168, TC-171 | Pass | Met |
| FR-04 | Create a daily logbook entry (date, description, hours 0-24, mentor email). | Functional | Must | UC-02; logbook_entries table | src/app/actions/logbook.ts (createLogbookEntryAction); src/components/new-entry-form.tsx | TC-005, TC-006, TC-012, TC-014, TC-122, TC-123 | Pass | Met |
| FR-05 | Attach one optional evidence file (PDF, PNG, JPG, WEBP). | Functional | Should | UC-03; data/uploads | src/app/actions/logbook.ts (extension allow-list, random file names) | TC-124, TC-125 | Pass | Met |
| FR-06 | Submit a Draft or Rejected entry for verification; 7-day one-time link; status Pending. | Functional | Must | UC-04; verification_tokens table; sequence diagram steps 1-5 | src/app/actions/logbook.ts (submitEntryForVerificationAction); repository.submitEntryForVerification (transaction) | TC-127, TC-128, TC-129, TC-137 | Pass | Met |
| FR-07 | Mentor opens the link without an account and approves or rejects the entry. | Functional | Must | UC-05; mentor verification page | src/app/verify/[token]/page.tsx; src/app/actions/verify.ts | TC-130, TC-131, TC-132 | Pass | Met |
| FR-08 | A link works only once and only before it expires. | Functional | Must | Sequence diagram alt frame; activity diagram | src/app/actions/verify.ts; repository.recordMentorDecision (WHERE used_at IS NULL) | TC-133, TC-134, TC-135, TC-136, TC-137 | Pass | Met |
| FR-09 | Show each entry's status (Draft, Pending verification, Verified, Rejected). | Functional | Must | Status badges | src/components/status-badge.tsx; src/lib/logbook-metrics.ts | TC-003, TC-004, TC-138 | Pass | Met |
| FR-10 | Student views each placement's entries, newest first, and its verified hours against its required hours. | Functional | Must | Student logbook page | src/app/student/page.tsx; src/app/student/placements/[id]/page.tsx; sumVerifiedHours | TC-002, TC-006, TC-132, TC-138, TC-155 | Pass | Met |
| FR-11 | Coordinators and lecturers see each student's status for their current placement, verified hours, entries, pending count and missing documents. | Functional | Must | UC-06; coordinator dashboard | src/app/coordinator/page.tsx; studentOverviewStatus, currentPlacement | TC-009, TC-139, TC-170, TC-174 | Pass | Met |
| FR-12 | Record created, updated, verified and link-used times automatically. | Functional | Should | Timestamp columns in the ERD | src/lib/repository.ts; database/schema.sql | TC-131, TC-140 | Pass | Met |
| FR-13 | Evidence files only downloadable by the owner, coordinators and lecturers. | Functional | Must | Evidence route | src/app/api/evidence/[fileName]/route.ts | TC-126 | Pass | Met |

| Requirement ID | Requirement Description | Type | Priority | Design / Component | Implementation Evidence | Test Case ID | Test Result | Final Status |
|---|---|---|---|---|---|---|---|---|
| FR-14 | Sign out ends the session. | Functional | Must | UC-07; header | src/app/actions/auth.ts (logoutAction); src/components/app-header.tsx | TC-007, TC-010, TC-141, TC-146 | Pass | Met |
| FR-15 | Add WIL placements, each with its own required hours; every entry belongs to a placement. | Functional | Must | UC-08; wil_placements table; placement pages | src/app/actions/placements.ts; src/app/student/placements; src/lib/repository.ts | TC-150, TC-151, TC-152, TC-153, TC-154, TC-155, TC-156, TC-174 | Pass | Met |
| FR-16 | Upload WIL documents: MOU, Commencement Letter and Code of Conduct per placement; Attendance Register per semester. | Functional | Must | UC-09; placement_documents table; documents section | src/app/actions/documents.ts; src/lib/documents.ts; src/app/api/documents/[id]/route.ts | TC-157, TC-158, TC-159, TC-160, TC-161, TC-162, TC-163, TC-164, TC-165 | Pass | Met |
| FR-17 | Coordinators and lecturers open a student's profile with placements, documents and logbook. | Functional | Must | UC-10; student profile page | src/app/coordinator/students/[id]/page.tsx | TC-166, TC-167, TC-168, TC-169, TC-170 | Pass | Met |
| FR-18 | A placement is WIL completed when its verified hours reach its required hours, and is then locked. | Functional | Must | Status rules; locks in the Server Actions | src/lib/logbook-metrics.ts (placementCompletion, studentOverviewStatus); checks in src/app/actions | TC-172, TC-173, TC-174 | Pass | Met |
| NFR-01 | A first-time student creates and submits an entry in under 3 minutes. | Non-Functional | Should | Four-field entry form; Copy link button | src/components/new-entry-form.tsx | TC-149 | Not Tested | Partially Met |
| NFR-02 | Pages and actions respond within 2 seconds. | Non-Functional | Should | SQLite with indexes; server rendering | src/lib/db.ts; database/schema.sql (indexes) | TC-144 | Pass | Met |
| NFR-03 | Salted password hashes, HttpOnly signed session cookie, 128+ bit tokens, server-side checks. | Non-Functional | Must | Security design (Chapter 4.7) | src/lib/password.ts; src/lib/session.ts; src/app/actions/logbook.ts (randomBytes(24)) | TC-121, TC-126, TC-127, TC-142, TC-160, TC-162, TC-171 | Pass | Met |
| NFR-04 | Database constraints reject invalid data; multi-step updates are atomic. | Non-Functional | Must | ERD constraints; transactions | database/schema.sql; src/lib/db.ts (inTransaction) | TC-136, TC-143 | Pass | Met |
| NFR-05 | TypeScript, layered modules, lint passes, README set-up in five commands or fewer. | Non-Functional | Should | Three-layer architecture | Source structure; README.md | TC-147 | Pass | Met |
| NFR-06 | Works in current Chrome and Edge; usable at 375 px and 1280 px. | Non-Functional | Should | Responsive Tailwind layout | src/app/*/layout.tsx; overflow-x-auto tables | TC-145, TC-146 | Pass | Met |
| NFR-07 | Only data needed for WIL tracking is stored; synthetic data only until consent. | Non-Functional | Must | Data minimisation; Ethical Clearance | database/seed.sql; database/demo-documents | TC-148 | Pass | Met |

# FINAL SYSTEM DEMONSTRATION SCRIPT AND SLIDE OUTLINE

This plan tells the story from the problem to the results and then demonstrates the key user journey live. Every member presents a part and answers questions on their own work. Planned length: about 15 minutes, followed by questions. Dress code: formal business attire for all members.

## 1. Slide Outline

Table 1: Slide outline and presenters

| No. | Slide | Content | Presenter | Time |
|---|---|---|---|---|
| 1 | Title | TrackWIL: Work-Integrated Learning logbook tracking; Group 3; team names; GitHub link | Gedion Basson | 0:30 |
| 2 | The problem | Paper, PDF and email logbooks: lost documents, slow sign-offs, no live view of who is behind; who is affected | Gedion Basson | 1:00 |
| 3 | Aim and objectives | One aim and five objectives; research questions linked to them | Gedion Basson | 1:00 |
| 4 | What exists already | InPlace, Sonia Online, Mahara/PebblePad compared; the gap: account-free mentor verification plus monitoring | Dylan Shepard | 1:00 |
| 5 | Requirements and method | 18 functional and 7 measurable non-functional requirements; Agile sprints aligned with the weekly submissions | Sihle Kayise | 1:00 |
| 6 | Design | Architecture diagram; ERD (5 tables); verification sequence diagram | Moegammad Green | 1:30 |
| 7 | Live demonstration | Switch to the browser (Section 4 below) | Moegammad Green and Gedion Basson | 5:00 |
| 8 | Testing and results | 81 test cases, 80 passed, 1 blocked; automated suite; response times | Dylan Shepard | 1:30 |
| 9 | Defects found and fixed | DEF-01 forged session cookie and DEF-07 data in redirect responses (both critical), DEF-02 expired links, DEF-03 mobile layout: found, fixed, retested | Dylan Shepard | 1:00 |
| 10 | Conclusion and future work | Objectives achieved; limitations (synthetic data, usability not yet measured); future work | Sihle Kayise | 1:00 |
| 11 | Team and Git evidence | Each member's role and contribution; branches and pull requests | All members | 0:30 |
| 12 | Questions | Thank you; questions | All members | — |

## 2. Opening

Gedion Basson introduces the team and the project: "We are Group 3 and we built TrackWIL, a web application that replaces paper and email WIL logbooks. Students record their daily work and keep their WIL documents for each placement, workplace mentors verify it from a link without needing an account, and coordinators and lecturers can see straight away who is on track and who is falling behind."

## 3. Problem, Objectives and Design

- **Problem (Gedion):** students wait for signed documents, documents are lost in email, and staff cannot see progress without chasing students.
- **Gap (Dylan):** existing placement and portfolio systems are broad and institution-configured; mentors usually need accounts.
- **Requirements and method (Sihle):** prioritised, testable requirements; four one-week sprints ending on each submission date.
- **Design (Moe):** a single Next.js application in three layers; SQLite database with five tables; the verification sequence with one-time tokens and transactions.

## 4. Live Demonstration Flow

Before the presentation: run npm run db:reset, start the application (npm run build, then npm start), open http://localhost:3000 in one normal window and one private window, and close every other application.

Table 2: Live demonstration steps and expected results

| Step | Action | Expected result | Presenter |
|---|---|---|---|
| 1 | Sign in as student2@ctucareer.co.za / student123 and open WIL 1 | Sihle's WIL 1 logbook opens: "6 of 200 verified hours completed", the MOU uploaded and the other documents Missing, one entry Pending verification | Moe |
| 2 | Upload a PDF as the Commencement Letter | The Commencement Letter shows Uploaded; uploading a file that is not a PDF or image is refused | Moe |
| 3 | Click New entry; enter today's date, a short description, 6 hours, mentor@host.co.za and attach a PDF | Entry is saved as Draft and opens | Moe |
| 4 | Try 30 hours on a second entry | Message: "Hours for one day cannot be more than 24." | Moe |
| 5 | On the saved entry, click Submit for verification, then Copy link | Status becomes Pending verification; the 7-day link is shown | Moe |
| 6 | Paste the link into the private window (no sign-in) | Mentor page shows the student, date, hours and description with Approve and Reject | Gedion |
| 7 | Click Approve | Confirmation; the entry is Verified | Gedion |
| 8 | Refresh the mentor link | "This link was already used (approved)." | Gedion |

| Step | Action | Expected result | Presenter |
|---|---|---|---|
| 9 | Back in the student window, open WIL 1 | The entry shows Verified and the hours have increased to 12 of 200 | Moe |
| 10 | Sign out; sign in as lecturer@ctucareer.co.za / lecturer123 | Dashboard: Dylan On track, Moe Overdue, Sihle and Gedion Awaiting verification, Lerato WIL completed, with hours, pending counts and missing documents | Gedion |
| 11 | Click Sihle Kayise on the dashboard | Sihle's profile: WIL 1 with 12 of 200 verified hours, the documents (view only) and the logbook | Gedion |
| 12 | (If time) Open a student URL as the lecturer | Redirected back to the dashboard: access is role-based | Gedion |
| 13 | Sign out; sign in as student3@ctucareer.co.za / student123 | Lerato's WIL 1 shows WIL completed (200 of 200) and is locked; the student is asked to add the next WIL placement | Moe |

## 5. Testing Evidence

Dylan shows the Test Cases and Results workbook and the automated run (npm run test:system: 62 of 62 passed), explains the fixed defects with the before-and-after evidence, and shows the response-time table and the 375-pixel phone screenshots.

## 6. Conclusion

Sihle summarises: all eighteen functional requirements are met; six of the seven non-functional requirements are met; usability with real users still has to be measured once ethical permission is granted. Future work: mentor feedback, coordinator review of documents, notifications, mentor access to evidence and a hosted deployment.

## 7. Backup Plan

- **Recorded walkthrough:** the 3 to 5 minute screen recording submitted in Week 4 is downloaded to the laptop and to a USB drive.
- **Screenshots:** every demonstration screen is in 04_System_Design_and_Development/screenshots and in the slides.
- **Second laptop:** a second member's laptop has the repository cloned, npm install done and the build ready.
- **Data problems:** npm run db:reset restores the demonstration data in seconds.
- **No network needed:** the application and database run locally, so the demonstration does not depend on Wi-Fi.

# INDIVIDUAL CONTRIBUTION FORM

Table 1: Student details

| Field | Details |
|---|---|
| Name | Willem (Gedion) Basson |
| Student number | 20240268 |
| Group | Group 3: TrackWIL |
| Role | Group Lead |
| GitHub account | Cyber-4 |
| Module | SPM622 (Software Engineering 2) |
| Supervisor | Ms Amenda Raganya |

## Contribution Summary

- Research: wrote and revised the project proposal and obtained topic acceptance; completed the project administration documents (group declaration, ethical clearance).
- Requirements and design: wrote the Software Requirements Specification; revised Chapters 1 to 4, the SRS and the Database Design document to match the final build; rebuilt the UML, ERD and architecture diagrams.
- Coding: replaced the JSON file storage with an SQLite database (schema, seed data, transactions); fixed the session-cookie, expired-link and mobile-layout defects.
- Testing: wrote and ran the automated system test suite (36 tests); recorded the test results, Defect Log and Requirements Traceability Matrix.
- Documentation and project management: compiled the Week 2, 3 and 4 submissions; sprint backlog, risk register and progress reports; merged pull requests #2, #3, #5, #6 and #7.
- Presentation: introduction, problem and aim slides; mentor and dashboard steps of the live demonstration.

## Git / Evidence References

All commits are in the repository https://github.com/Noobey32/TrackWIL.

- Proposal and acceptance: commits 08ea06b, e03757d, 09c4707, 78eef1e
- Administration folder and repository structure: commits 08aaa44, 725798c, 5d1b6d6
- Week 2 submission: commits 87c899a, 6380779, 5a84b15
- SRS: commit f84afa3 (pull request #2); Week 3 submission: commit 6b6b05b (pull request #3)
- SQLite database: commit 7080e7f (pull request #5); documentation update: commit 2634d57 (pull request #6)
- Project management evidence: commit 7eddd77 (pull request #7)
- Defect fixes and system tests: commit 5268c2e (branch gedion/fix-defects); tests/system-tests.mjs

## Peer Confirmation

The other group members confirm that this contribution record is accurate.

Table 2: Peer confirmation

| Group member | Confirmed (Yes / No) | Signature | Date |
|---|---|---|---|
| Moegammad Green | Yes | M.A.Green | 29/10/26 |
| Dylan Shepard | Yes | D.Shepard | 29/10/26 |
| Sihle Kayise | Yes | S. Kayise | 29/10/2026 |

## Student Declaration

I confirm that the information above accurately reflects my contribution.

Signature: GB Date: 29/10/26

ftext here

# INDIVIDUAL CONTRIBUTION FORM

Table 1: Student details

| Field | Details |
|---|---|
| Name | Moegammad Green |
| Student number | 20240053 |
| Group | Group 3: TrackWIL |
| Role | Development Lead |
| GitHub account | Noobey32 (repository owner) |
| Module | SPM622 (Software Engineering 2) |
| Supervisor | Ms Amenda Raganya |

## Contribution Summary

- Repository: created the GitHub repository and its folder structure.
- Coding: built the Next.js application and the MVP: sign-in and sessions, logbook entries with evidence upload, token-based mentor verification and the coordinator dashboard; added student sign-up with CTU email and password validation; documented the code; fixed the form action issue.
- Design: drew the draw.io diagrams and wrote the first draft of Chapter 4 (system design).
- Project management: revised the proposal after supervisor feedback; wrote the project charter, team roles and the work plan.
- Presentation: design slide and the student steps of the live demonstration.

## Git / Evidence References

All commits are in the repository https://github.com/Noobey32/TrackWIL.

- Repository set-up: commits 629b191, cb29667, bdd42c1
- Proposal revision: commit 5465df5; project charter and roles: commits 9134c50, 4ebeae7; work plan: commit 98e8363
- Next.js scaffold and MVP: commits 32c3f87, 5893fee, f1f691e
- Sign-up validation: commit 5883791; code documentation: commit af5d423; form action fix: commit 369fe55
- Diagrams and Chapter 4 draft: commits 29c2cab, ad457c0 (branch moe/system-design)

## Peer Confirmation

The other group members confirm that this contribution record is accurate.

Table 2: Peer confirmation

| Group member | Confirmed (Yes / No) | Signature | Date |
|---|---|---|---|
| Willem (Gedion) Basson | Yes | G.Basson | 29/10/26 |
| Dylan Shepard | Yes | D.Shepard | 29/10/26 |
| Sihle Kayise | Yes | S.Kayise | 29/10/2026 |

## Student Declaration

I confirm that the information above accurately reflects my contribution.

Signature: M.A.Green Date: 29/10/26

# INDIVIDUAL CONTRIBUTION FORM

Table 1: Student details

| Field | Details |
|---|---|
| Name | Dylan Shepard |
| Student number | 20240887 |
| Group | Group 3: TrackWIL |
| Role | Testing / QA |
| GitHub account | FyreGH |
| Module | SPM622 (Software Engineering 2) |
| Supervisor | Ms Amenda Raganya |

## Contribution Summary

- Research: wrote Chapter 2 (Literature Review) and completed the Literature Review Matrix; created the Harvard reference list.
- Documentation: formatted Chapters 1 to 3; revised Chapters 1 and 2 using the Week 2 feedback and wrote the revision table; created the Week 3 submission folder and added Chapters 3 and 4 and the SRS to it.
- Quality assurance: reviewed the documents for consistency; test lead in the Test Plan.
- Project management: updated the member roles in the project charter.
- Presentation: existing systems, testing results and defects slides.

## Git / Evidence References

All commits are in the repository https://github.com/Noobey32/TrackWIL.

- Chapter 2 and Literature Review Matrix: commits 12de6ad, 16f0efc
- Reference list: commit 79cf0f8
- Member roles in the charter: commits a8c797d, 7be3a5d
- Formatting and revisions: commits c5794a4, 52d753c, b4c0d76, f255d6e (branch Dylan/Revise-chapters-1-and-2)
- Week 3 submission folder: commits 917f34f, 38e8a6f; merge of Chapter 3: commit 3e1fabe

## Peer Confirmation

The other group members confirm that this contribution record is accurate.

Table 2: Peer confirmation

| Group member | Confirmed (Yes / No) | Signature | Date |
|---|---|---|---|
| Willem (Gedion) Basson | Yes | G.Basson | 29/10/26 |
| Moegammad Green | Yes | M.A.Green | 29/10/26 |
| Sihle Kayise | Yes | S.Kayise | 29/10/2026 |

## Student Declaration

I confirm that the information above accurately reflects my contribution.

Signature: D.Shepard Date: 29/10/26

# INDIVIDUAL CONTRIBUTION FORM

Table 1: Student details

| Field | Details |
|---|---|
| Name | Sihle Kayise |
| Student number | 20241224 |
| Group | Group 3: TrackWIL |
| Role | Group Member |
| GitHub account | sihle588 |
| Module | SPM622 (Software Engineering 2) |
| Supervisor | Ms Amenda Raganya |

## Contribution Summary

- Research: contributed the Chapter 1 (Introduction) draft.
- Requirements: contributed the Chapter 3 (Requirements and Methodology) draft.
- Testing: wrote the first Test Plan and the Chapter 5 draft; designed and ran the manual test cases TC-001 to TC-014 and recorded the results with screenshots.
- Presentation: requirements and methodology slide and the conclusion.

## Git / Evidence References

All commits are in the repository https://github.com/Noobey32/TrackWIL.

- Chapter 1 upload: commit 800fe83
- Chapter 3 upload: commit daf3c24
- Chapter 5 draft, Test Plan and test cases: commit 48df206 (branch sihle-trackwil, pull request #4)
- Manual test screenshots: 06_Testing_Results_and_Evaluation/evidence/manual_tests_sihle

## Peer Confirmation

The other group members confirm that this contribution record is accurate.

Table 2: Peer confirmation

| Group member | Confirmed (Yes / No) | Signature | Date |
|---|---|---|---|
| Willem (Gedion) Basson | Yes | G.Basson | 29/10/26 |
| Moegammad Green | Yes | M.A.Green | 29/10/26 |
| Dylan Shepard | Yes | D.Shepard | 29/10/26 |

## Student Declaration

I confirm that the information above accurately reflects my contribution.

Signature:  Date: 29/10/2026

# SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

## 1. Purpose and Scope

This Software Requirements Specification (SRS) defines the complete requirements for TrackWIL version 1.0, the system built and tested in this project. Chapter 3 summarises these requirements, Chapter 4 shows how they were designed and implemented, and Chapter 5 traces every requirement to its test cases in the Requirements Traceability Matrix.

TrackWIL is a web-based system that replaces the paper, PDF and email process for Work-Integrated Learning (WIL) logbooks at CTU Training Solutions. Students record daily work and evidence, workplace mentors verify each entry through a one-time link without needing an account, and coordinators and lecturers monitor every student's progress on a shared dashboard.

**In scope (version 1.0): student sign-up and sign-in; WIL placements with their own required hours; daily logbook entries with optional evidence; WIL documents for each placement (MOU, Commencement Letter, Code of Conduct and an attendance register per semester);** submission for verification through a one-time link; mentor approval or rejection; entry statuses and verified-hour totals; the coordinator and lecturer dashboard with a profile for each student; WIL completion; an SQLite database.

**Out of scope (version 1.0):** mentor accounts, written mentor feedback, coordinator review of uploaded documents, notifications, editing or deleting entries, reports and exports, integration with CTU systems, a mobile application and the use of real personal data.

Table 1: Definitions

| Term | Meaning |
|---|---|
| WIL | Work-Integrated Learning: the workplace component of a qualification |
| Entry | One day of WIL work recorded by a student for one placement (date, description, hours) |
| Verification link / token | A one-time URL containing a 48-character random token that lets a mentor verify one entry without signing in |
| Verified hours | The total hours of a placement's entries with the status Verified |
| Overdue | A student whose current placement has no verified entry in the last 14 days (counted from the placement start if nothing is verified yet) and no entry awaiting verification |
| WIL placement | One period of WIL at a host company, with its own required hours, logbook and documents |
| WIL documents | The Memorandum of Understanding (MOU), Commencement Letter and Code of Conduct (once per placement) and the Attendance Register (once per semester) |
| WIL completed | A placement whose verified hours have reached its required hours; it is then locked |

## 2. Users and Stakeholders

Table 2: User classes

| User class | Description | Access | Main tasks |
|---|---|---|---|
| Student | Enrolled CTU student completing WIL hours | Account (student) | Sign up, add WIL placements, record entries, upload evidence and WIL documents, request verification, track hours |
| Workplace mentor | Staff member at the host organisation who supervises the student | One-time link only | Review and approve or reject an entry |
| Coordinator | CTU staff member who manages WIL compliance | Account (coordinator) | Monitor all students' status, hours and documents; open student profiles |
| Lecturer | CTU lecturer who supports students academically | Account (lecturer) | Monitor all students' status, hours and documents; open student profiles |

Secondary stakeholders are the training institution (needs reliable records and POPIA compliance), the development team (needs clear, testable requirements) and the project supervisor (checks alignment with the project objectives and rubric).

## 3. Functional Requirements

Table 3: Functional requirements with acceptance criteria

| ID | Requirement | Priority | Acceptance criterion |
|---|---|---|---|
| FR-01 | Sign in with email and password and go to the dashboard for the user's role. | High | Correct details open /student for students and /coordinator for coordinators and lecturers; wrong details show "Invalid email or password." and no session is created. Email matching ignores letter case. |
| FR-02 | Student sign-up with a CTU student email (eight digits @ctucareer.co.za) and a strong password. | Medium | Password needs 8+ characters with upper case, lower case, a number and a special character; a non-CTU or duplicate email is rejected with a message; a valid sign-up signs the student in. |
| FR-03 | Role-based access to pages and actions. | High | Signed-out visitors are sent to the sign-in page; coordinators and lecturers cannot open the student area; students cannot open the dashboard; a student cannot view another student's entry or placement; a blocked page sends none of its data. |

| ID | Requirement | Priority | Acceptance criterion |
|---|---|---|---|
| FR-04 | Create a daily logbook entry. | High | The entry belongs to one of the student's open placements; work date, description and hours are required; hours must be more than 0 and at most 24; the entry is saved as Draft and opened. |
| FR-05 | Attach one optional evidence file to an entry. | Medium | PDF, PNG, JPG, JPEG and WEBP files are accepted and stored under a random name; any other type is rejected with "Evidence must be a PDF or image file." |
| FR-06 | Submit an entry for verification. | High | Only Draft or Rejected entries with a mentor email can be submitted; a 48-character token valid for 7 days is created; the entry becomes Pending verification and the link is shown with a Copy button. |
| FR-07 | Mentor verification through the link without an account. | High | The link shows the student's name, work date, hours, description and status, with Approve and Reject buttons; Approve sets Verified and records the time; Reject sets Rejected. |
| FR-08 | One-time, expiring links. | High | A used link shows "This link was already used"; an expired or unknown link shows an error; neither changes the entry; two clicks on Approve record only one decision. |
| FR-09 | Show entry status. | High | Each entry shows Draft, Pending verification, Verified or Rejected on the logbook and entry pages. |
| FR-10 | Logbook history and hour progress. | High | Each placement's logbook lists its entries newest first and shows "X of Y verified hours completed" against the placement's required hours. |
| FR-11 | Coordinator and lecturer monitoring dashboard. | High | Every student is listed with their status for their current placement (On track, Awaiting verification, Overdue, WIL completed or No placement), verified hours against required hours, number of entries, number pending and missing documents. |
| FR-12 | Automatic timestamps. | Medium | created_at, updated_at and verified_at are stored on entries; used_at and decision are stored on links. |
| FR-13 | Protected evidence download. YEs | High | The student who uploaded a file, coordinators and lecturers can open it; another student receives 403 Forbidden and a signed-out visitor receives 401 Unauthorized. |

| ID | Requirement | Priority | Acceptance criterion |
|---|---|---|---|
| FR-14 | Sign out. | High | Sign out removes the session cookie and returns to the sign-in page; protected pages then redirect to sign-in. |
| FR-15 | WIL placements with their own required hours. | High | A student adds a placement with a title, company, start date and required hours (more than 0 and at most 5 000); invalid details are rejected with a message; a placement can be edited until it is completed; entries can only be added to the student's own placements, and another student's placement cannot be opened. |
| FR-16 | WIL documents for each placement. | High | The MOU, Commencement Letter and Code of Conduct are kept once per placement and the Attendance Register once per semester; PDF, PNG and JPG files up to 10 MB whose content matches their type are accepted; uploading again replaces the file; the student can remove a document; only the student, coordinators and lecturers can open it. |
| FR-17 | Student profile for coordinators and lecturers. | High | Selecting a student on the dashboard opens their profile with each placement's verified hours, documents (view only) and logbook; students cannot open it; an unknown student or a staff account returns 404 Not Found. |
| FR-18 | WIL completion. | High | A placement is marked WIL completed as soon as its verified hours reach its required hours; new entries, submissions, document changes and edits on it are refused with a message; the student is asked to add the next placement. |

## 4. Non-Functional Requirements

Table 4: Non-functional requirements

| ID | Category | Requirement | Measure |
|---|---|---|---|
| NFR-01 | Usability | A first-time student can create and submit an entry for verification in under 3 minutes without help. | Timed task (Chapter 5) |
| NFR-02 | Performance | Pages load and actions complete within 2 seconds on the test computer with the demonstration data. | Browser timing |

| ID | Category | Requirement | Measure |
|---|---|---|---|
| NFR-03 | Security | Salted scrypt password hashes; HttpOnly session cookie; tokens with at least 128 bits of randomness; server-side role checks on every protected page and action. | Code review and security tests |
| NFR-04 | Reliability and integrity | Database constraints reject invalid data; multi-step updates are atomic transactions. | Constraint tests |
| NFR-05 | Maintainability | TypeScript, layered modules, lint passes, README setup in five commands or fewer. | Lint and README walkthrough |
| NFR-06 | Compatibility | Works in current Chrome and Edge; usable at 375 px and 1280 px wide. | Browser and responsive tests |
| NFR-07 | Privacy | Only data needed for WIL tracking is stored; synthetic data only until written consent. | Data review |

## 5. Use Cases / User Stories

Table 5: Use cases

| ID | Use case | Actor(s) | Requirements |
|---|---|---|---|
| UC-01 | Sign in / sign up | Student, coordinator, lecturer | FR-01, FR-02, FR-03 |
| UC-02 | Create logbook entry | Student | FR-04, FR-09, FR-12 |
| UC-03 | Upload evidence (extends UC-02) | Student | FR-05, FR-13 |
| UC-04 | Submit for verification | Student | FR-06 |
| UC-05 | Approve or reject via link | Workplace mentor | FR-07, FR-08 |
| UC-06 | View monitoring dashboard | Coordinator, lecturer | FR-10, FR-11 |
| UC-07 | Sign out | Student, coordinator, lecturer | FR-14 |
| UC-08 | Manage WIL placements | Student | FR-15, FR-18 |
| UC-09 | Upload WIL documents | Student | FR-16 |
| UC-10 | View student profile (extends UC-06) | Coordinator, lecturer | FR-03, FR-17 |

The user stories US-01 to US-11 and their acceptance criteria are listed in Chapter 3, Section 3.7.

## 6. Data Requirements

TrackWIL stores its data in an SQLite database with five tables. The full data dictionary is in the Database Design document.

- **users:** id, full name, email (unique, case-insensitive), role (student, coordinator or lecturer) and password hash.
- wil_placements: id, student id (foreign key to users), title, company name, required hours (more than 0 and at most 5 000), start date and the created and updated timestamps.

- **logbook_entries: id, placement id (foreign key to wil_placements),** work date, description, hours (more than 0 and at most 24), evidence file name, status, mentor email and the created, updated and verified timestamps.
- **verification_tokens:** token (primary key), entry id (foreign key to logbook_entries), expiry time, time used and decision (approved or rejected).
- placement_documents: id, placement id (foreign key to wil_placements), document type (mou, commencement_letter, code_of_conduct or attendance_register), semester (attendance registers only), original file name, stored file name, file size (at most 10 MB) and upload time; one document of each type per placement and one attendance register per semester.

**Validation rules:** every rule is checked in the server action and, where possible, again by a database constraint (foreign key, UNIQUE or CHECK), so that invalid data cannot be stored even if a check in the code is missed.

**Evidence files are stored in the data/uploads folder and WIL documents in data/uploads/documents, under random names; only the file names (and each document's original name and size) are stored in the database.**

**Retention:** demonstration data is synthetic and can be reset at any time with npm run db:reset. If real data is used after permission is granted, personal information will be deleted when the project result is finalised, unless the campus sets another period.

## 7. Interfaces

Table 6: User interface screens

| Screen | Route | Users |
|---|---|---|
| Sign in | / | All account holders |
| Sign up | /signup | New students |
| My WIL placements | /student | Students |
| Add or edit a placement | /student/placements/new, /student/placements/[id]/edit | Students |
| Placement logbook and documents | /student/placements/[id] | Students |
| New entry | /student/entries/new | Students |
| Entry details and verification link | /student/entries/[id] | Students |
| Mentor verification | /verify/[token] | Workplace mentors (no sign-in) |
| Coordinator dashboard | /coordinator | Coordinators and lecturers |
| Student profile | /coordinator/students/[id] | Coordinators and lecturers |

**Software interfaces:** the application uses Node.js's built-in node:sqlite module to access the SQLite database file data/trackwil.db, and the server file system for evidence files and WIL documents. No external services are called.

**Communication interfaces:** browsers communicate with the server over HTTP (HTTPS in a hosted deployment). Forms call Next.js Server Actions; evidence files are served by the route /api/evidence/[fileName] and WIL documents by /api/documents/[id].

## 8. Constraints and Assumptions

- **Time:** the system had to be built, tested and demonstrated by 29 October 2026, so the scope was limited to the core workflow.
- **Technology:** Next.js 16 with TypeScript and Tailwind CSS; SQLite through Node.js 22.13 or newer; Git and GitHub for version control.
- **Legal and privacy:** POPIA applies; no real personal information is collected before written campus permission, so all testing uses synthetic data.
- **Deployment:** the prototype runs locally; cloud hosting is out of scope.
- **Assumption:** users have a modern web browser and an internet or network connection to the server.
- **Assumption:** students send the verification link to the correct mentor, and mentors check the work before approving it.
- **Assumption: students enter the correct required hours for each WIL placement (the demonstration placements require 120 or 200 hours).**

## 9. Acceptance Criteria

The solution is accepted when:

1. Every High-priority functional requirement (FR-01, FR-03, FR-04, FR-06 to FR-11 and FR-13 to FR-18) passes its test cases in the Test Cases and Results workbook.

2. A student can complete the full journey (sign in, open a placement, upload a document, create an entry with evidence, submit it, mentor approves through the link, hours increase) during the live demonstration using synthetic data.

3. Coordinators and lecturers can identify students who are on track, awaiting verification, overdue or WIL completed from the dashboard, and open each student's profile.

4. Security tests confirm that role restrictions, one-time links and protected evidence downloads work, and any open security defects are documented with a planned fix.

5. All defects found during testing are logged in the Defect Log and either fixed and retested or documented as known limitations.

# DATABASE DESIGN DOCUMENT

## Database Purpose

The TrackWIL database stores everything the application needs to track Work-Integrated Learning logbooks: the accounts of students, coordinators and lecturers; each student's WIL placements with their required hours; the daily logbook entries of each placement with their hours, evidence file name and verification status; the details of each uploaded WIL document; and the one-time verification links that workplace mentors use to approve or reject entries.

The database must support these queries efficiently and reliably:

- Sign-in: find a user by email address (case-insensitive).
- Student logbook: list a placement's entries, newest first, total its verified hours and work out whether the placement is completed.
- Documents: list a placement's documents and find the document already in a slot (a type, or a type and semester) so that it can be replaced.
- Verification: find a link by its token, check that it is unused and unexpired, and record the mentor's decision on the link and the entry together.
- Monitoring dashboard: for every student, find the current placement, count entries, pending verifications and missing documents, total verified hours and find the latest verification date.
- Evidence download: find the entry that owns an evidence file to check who may open it.

SQLite was chosen because it is a full relational database (keys, constraints and transactions) that runs inside the application through Node.js's built-in node:sqlite module, so no database server has to be installed. The database is a single file, data/trackwil.db, which is created and seeded automatically when the application first starts.

## Entity-Relationship Diagram

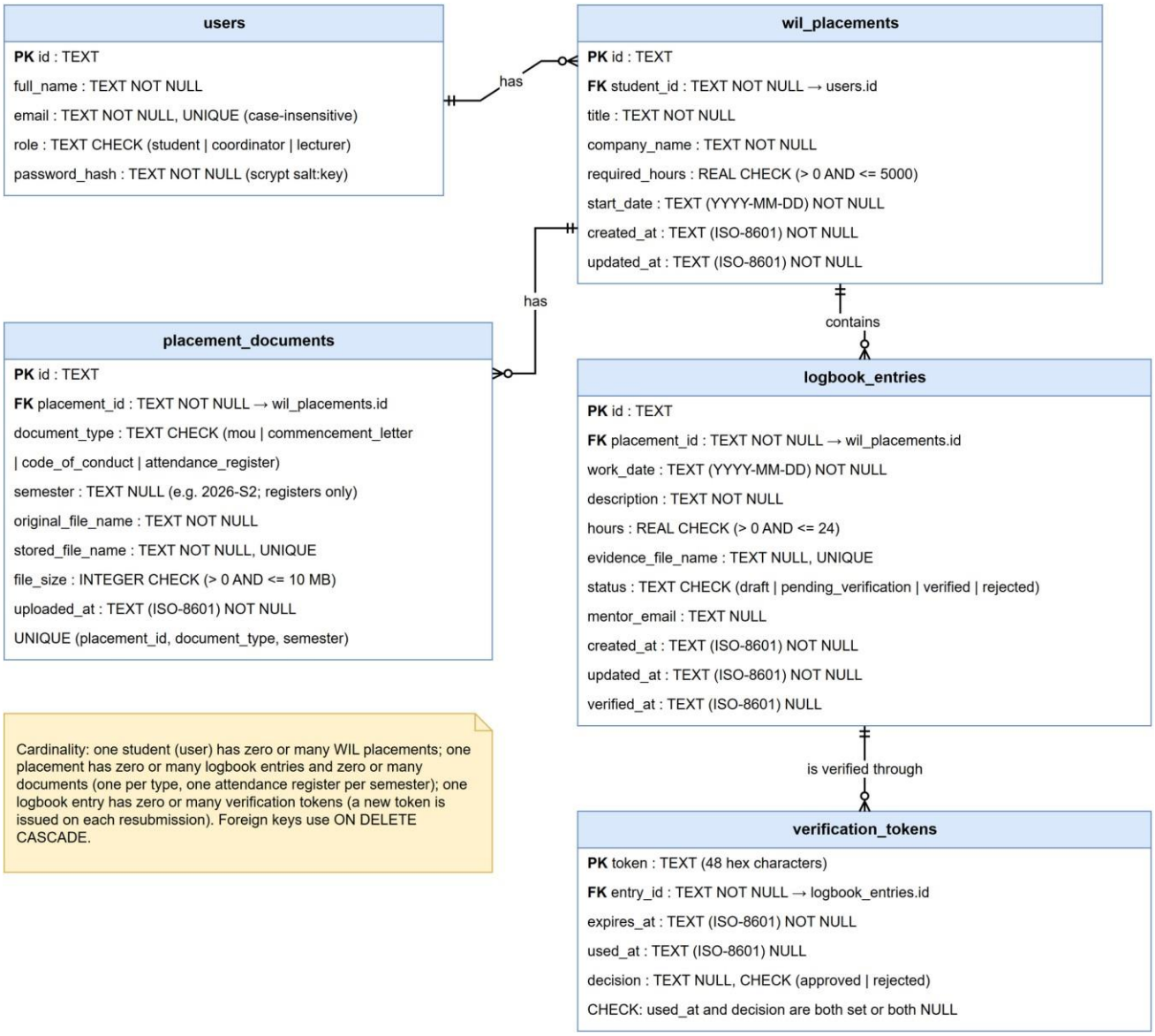



Figure 1: Entity-relationship diagram

Table 1: Relationships and cardinalities

| Relationship | Cardinality | Implemented by | Meaning |
|---|---|---|---|
| users to wil_placements | One to zero-or-many | wil_placements.student_id references users.id | A student can have any number of WIL placements; each placement belongs to exactly one student |
| wil_placements to logbook_entries | One to zero-or-many | logbook_entries.placement_id references wil_placements.id | A placement can have any number of entries; each entry belongs to exactly one placement (and so to one student) |
| logbook_entries to verification_tokens | One to zero-or-many | verification_tokens.entry_id references logbook_entries.id | An entry gets a new link each time it is submitted (including resubmission after rejection); each link belongs to exactly one entry |
| wil_placements to placement_documents | One to zero-or-many | placement_documents.placement_id references wil_placements.id | A placement has at most one document of each type and one attendance register per semester; each document belongs to exactly one placement |

Mentors are not stored as a separate entity because they have no account in version 1.0. The mentor is identified by the email address stored on each entry, and each verification is recorded on the token that the mentor used.

## Data Dictionary

All text values are stored as TEXT and numbers as REAL (file sizes as INTEGER), which are SQLite's storage classes. Dates and times are stored as ISO-8601 text in UTC (for example 2026-10-22T08:00:00.000Z), which sorts correctly as text.

Table 2: users

| Column | Type | Size / format | Null | Key | Constraint and description |
|---|---|---|---|---|---|
| id | TEXT | Up to 44 characters (e.g. student-<UUID>) | No | PK | Unique account identifier |
| full_name | TEXT | Up to about 120 characters | No | | The user's full name, shown in headers and the dashboard |
| email | TEXT | Up to about 150 characters | No | Unique | Sign-in name; UNIQUE with COLLATE NOCASE so letter case does not matter |
| role | TEXT | student, coordinator or lecturer | No | | CHECK limits the value to the three roles |
| password_hash | TEXT | 32-character salt, a colon and a 128-character key | No | | scrypt hash of the password; the password itself is never stored |

Table 3: wil_placements

| Column | Type | Size / format | Null | Key | Constraint and description |
|---|---|---|---|---|---|
| id | TEXT | UUID (36 characters) | No | PK | Unique placement identifier |
| student_id | TEXT | As users.id | No | FK | References users(id) ON DELETE CASCADE; indexed with start_date |
| title | TEXT | Up to about 60 characters | No | | The placement's name, for example WIL 1 |
| company_name | TEXT | Up to about 120 characters | No | | The host company |
| required_hours | REAL | More than 0, at most 5 000 | No | | Hours this placement requires; CHECK (required_hours > 0 AND required_hours <= 5000) |
| start_date | TEXT | YYYY-MM-DD | No | | The day the placement started; also decides which semesters need an attendance register |
| created_at | TEXT | ISO-8601 UTC | No | | When the placement was added |
| updated_at | TEXT | ISO-8601 UTC | No | | When the placement last changed |

Table 4: logbook_entries

| Column | Type | Size / format | Null | Key | Constraint and description |
|---|---|---|---|---|---|
| id | TEXT | UUID (36 characters) | No | PK | Unique entry identifier |

| Column | Type | Size / format | Null | Key | Constraint and description |
|---|---|---|---|---|---|
| placement_id | TEXT | As wil_placements.id | No | FK | References wil_placements(id) ON DELETE CASCADE; indexed with work_date |
| work_date | TEXT | YYYY-MM-DD | No | | The day the work was done |
| description | TEXT | Free text | No | | What the student did |
| hours | REAL | More than 0, at most 24 | No | | CHECK (hours > 0 AND hours <= 24) |
| evidence_file_name | TEXT | UUID plus extension | Yes | Unique | Name of the stored evidence file, or NULL if none |
| status | TEXT | draft, pending_verification, verified, rejected | No | | CHECK limits the value; default draft |
| mentor_email | TEXT | Email address | Yes | | The workplace mentor for this entry; required before submission |
| created_at | TEXT | ISO-8601 UTC | No | | When the entry was created |
| updated_at | TEXT | ISO-8601 UTC | No | | When the entry last changed |
| verified_at | TEXT | ISO-8601 UTC | Yes | | When the mentor approved the entry |

Table 5: verification_tokens

| Column | Type | Size / format | Null | Key | Constraint and description |
|---|---|---|---|---|---|
| token | TEXT | 48 hexadecimal characters | No | PK | Random value from 24 bytes (192 bits); appears in the link /verify/<token> |
| entry_id | TEXT | As logbook_entries.id | No | FK | References logbook_entries(id) ON DELETE CASCADE; indexed |
| expires_at | TEXT | ISO-8601 UTC | No | | 7 days after the link was created |
| used_at | TEXT | ISO-8601 UTC | Yes | | When the mentor used the link; NULL while unused |
| decision | TEXT | approved or rejected | Yes | | The mentor's decision; CHECK requires used_at and decision to be both set or both NULL |

Table 6: placement_documents

| Column | Type | Size / format | Null | Key | Constraint and description |
|---|---|---|---|---|---|
| id | TEXT | UUID (36 characters) | No | PK | Unique document identifier |
| placement_id | TEXT | As wil_placements.id | No | FK | References wil_placements(id) ON DELETE CASCADE |
| document_type | TEXT | mou, commencement_letter, code_of_conduct or attendance_register | No | | CHECK limits the value to the four WIL documents |
| semester | TEXT | YYYY-S1 or YYYY-S2 | Yes | | Only for attendance registers; CHECK requires a semester for a register and none for the other documents |
| original_file_name | TEXT | Up to 200 characters | No | | The name of the uploaded file, shown to users |
| stored_file_name | TEXT | UUID plus extension | No | Unique | Name of the file in data/uploads/documents |
| file_size | INTEGER | Bytes | No | | CHECK (file_size > 0 AND file_size <= 10485760), at most 10 MB |
| uploaded_at | TEXT | ISO-8601 UTC | No | | When the file was uploaded or last replaced |

## Normalisation

**First normal form (1NF).** Every column holds one value and every table has a primary key. For example, each day of work is its own row in logbook_entries rather than a list of days stored in one column of users.

**Second normal form (2NF).** Every table has a single-column primary key, so no column can depend on only part of a key.

**Third normal form (3NF).** No non-key column depends on another non-key column. A student's name is stored once in users and a placement's required hours once in wil_placements, not repeated in every entry, so changing them updates every screen at once (no update anomaly). An entry stores only its placement_id and not the student, because the student is found through the placement; storing both could let them disagree. Each token row holds only facts about that link.

**Calculated values are not stored.** Verified hours, whether a placement is WIL completed and each student's status are calculated from the entries every time a page is shown. Storing them would create duplicate data that could become wrong when an entry changes.

**Change from the prototype.** The first prototype stored each collection as a JSON file and rewrote the whole file on every change, with no keys or constraints. Moving to SQLite added enforced primary and foreign keys, CHECK constraints and transactions without changing the shape of the data.

### Security and Integrity

- **Constraints: primary keys, UNIQUE (email, evidence file name, stored document name), a unique index that allows one document of each type per placement and one attendance register per semester, foreign keys with PRAGMA foreign_keys = ON, and CHECK constraints for roles, statuses, hours, required hours, document types, semesters, file sizes and the token decision.**
- **Transactions: submitting an entry (insert token and set Pending), recording a mentor decision (mark token used and update the entry) and saving a document (find its slot, then insert or replace) each run in a single transaction,** so a failure cannot leave a half-finished change. The token is marked used only WHERE used_at IS NULL, so a link cannot be used twice.
- **Validation:** every value is validated in the Server Action before it reaches the database; the constraints are a second line of defence.
- **SQL injection:** every query uses prepared statements with ? placeholders; user input is never joined into SQL text.
- **Access control:** only the repository module runs SQL, and it is called only from server code after the user's role and ownership have been checked.
- **Sensitive data:** passwords are stored only as salted scrypt hashes. Evidence files and WIL documents are stored outside the public web folder under random names and served only to their owner and to staff; a document is accepted only if its content matches its file type.
- **Backups and recovery:** the database is a single file that can be copied while the application is stopped. For the demonstration, npm run db:reset rebuilds it from the scripts.
- **Privacy:** only synthetic data is stored until written campus permission is granted (POPIA).

### SQL Scripts

Table 7: Database scripts in the repository

| Script | Location | Purpose |
|---|---|---|
| schema.sql | 05_Source_Code/trackwil/database/schema.sql | Creates the five tables, keys, constraints and indexes |
| seed.sql | 05_Source_Code/trackwil/database/seed.sql | Inserts synthetic demonstration users, placements, entries, links and document records, with dates relative to the current date |
| demo-documents/ | 05_Source_Code/trackwil/database/demo-documents | Synthetic PDFs that the seed data refers to |
| reset-db.mjs | 05_Source_Code/trackwil/scripts/reset-db.mjs | Deletes and rebuilds the database from the two scripts and restores the demo documents (npm run db:reset) |
| db.ts | 05_Source_Code/trackwil/src/lib/db.ts | Applies the schema on start-up and seeds an empty database |
| repository.ts | 05_Source_Code/trackwil/src/lib/repository.ts | All queries and transactions used by the application |

# WEEKLY PROJECT PROGRESS REPORT

## Week / Date

Week 4 (Sprint 3): 19 to 23 October 2026. Group 3, TrackWIL. Report prepared for the Week 4 review on Friday 23 October 2026.

## Planned Work

The plan from the Week 3 report was:

- Improve the MVP (minimum viable product).
- Complete Chapter 4, including the user interface design and sections 4.6 to 4.8.
- Write the Chapter 5 draft (Testing, Results and Evaluation).
- Complete the testing workbooks (Test Plan, Test Cases and Results, Defect Log, Requirements Traceability Matrix).
- Compile the full dissertation draft (front matter and Chapters 1 to 5, with an outline of Chapter 6).
- Prepare the presentation plan (slide outline and demo script).

## Completed Work

Table 1: Work completed this week

| Work item | Owner | Evidence | Status |
|---|---|---|---|
| Chapter 5 draft, Test Plan and first test cases (TC-001 to TC-009) | Sihle Kayise | Commit 48df206; pull request #4 | Draft complete |
| MVP improvement: JSON file storage replaced with an SQLite database (schema, seed data, transactions, one-time token protection, 24-hour limit) and 28 automated end-to-end checks passed | Gedion Basson | Commit 7080e7f; pull request #5 | Complete |
| Chapters 1 to 4, the SRS and the Database Design document updated to match the system that was built | Gedion Basson | Commit 2634d57; pull request #6 | Complete |
| Chapter 4 completed (sections 4.1 to 4.9), with new class and activity diagrams, updated architecture, sequence, use case and ER diagrams, and screenshots of the running application | Gedion Basson | 04_System_Design_and_Development; pull request #6 | Complete |
| Sprint backlog, risk, issue and change register and this progress report updated | Gedion Basson | 09_Project_Management_Evidence | Complete |

**Not yet completed (carried forward):** execution of all test cases with actual results, the Defect Log and the Requirements Traceability Matrix; Chapter 6; the compiled dissertation draft; the slide outline and demo script; the Individual Contribution Forms; and the screen-recorded walkthrough.

## Challenges / Issues

- **Documentation did not match the build.** Chapters 1 and 3 and the SRS described mentor accounts, a separate lecturer dashboard and PostgreSQL. The documents were rewritten to match the built system, and the deferred features were recorded as future work (Issue I01, resolved).
- **Security defect found.** The session cookie stores the user id without a signature, so a user who edits the cookie could impersonate another account. A fix is planned for 25 October and will be retested (Issue I02).
- **Functional defect found.** If a mentor link expires before the mentor responds, the entry stays Pending verification and cannot be resubmitted (Issue I03).
- **Framework vulnerability.** The Next.js build reports a known security vulnerability in version 16.4.0; an upgrade is planned on a separate branch (Issue I04).
- **Ethical permission pending.** No written campus permission has been received, so usability testing with staff and students cannot start; all testing uses synthetic data (Risk R01).
- **Time pressure and uneven commits.** Other assessments limited the time available, and the number of commits differs between members (Risks R03 and R04).

## Next Week Plan

Week 5 (Sprint 4, final): 24 to 29 October 2026.

Table 2: Plan for the final week

| Day | Task | Owner |
|---|---|---|
| Fri 23 Oct | Record the 3-5 minute walkthrough; submit Group3_Week4.pdf by 12:00 | Gedion Basson |
| Sat 24 - Sun 25 Oct | Fix the session cookie defect (I02) and the expired-link defect (I03); add and run test cases | Gedion Basson, Moegammad Green |
| Sat 24 - Mon 26 Oct | Execute all test cases with actual results; complete the Defect Log and the Traceability Matrix | Dylan Shepard, Sihle Kayise |
| Mon 26 Oct | Complete the two Chapter 2 references; upgrade Next.js and rerun tests | Dylan Shepard, Moegammad Green |
| Mon 26 - Tue 27 Oct | Finalise Chapter 5 with results and evaluation; write Chapter 6 | Sihle Kayise, Gedion Basson |
| Tue 27 - Wed 28 Oct | Compile the final dissertation (front matter, abstract, contents, references, appendices); Turnitin check | Gedion Basson, Dylan Shepard |
| Tue 27 - Wed 28 Oct | Slides, demo script and backup plan; Individual Contribution Forms | All members |
| Thu 29 Oct | Final submission; presentation and live demonstration in business attire | All members |

## Supervisor Feedback

Comments, corrections and go-ahead.

## Sign-off

Group Representative: Gedion Basson _____ Supervisor: _____ Date: _____

## PROJECT SCHEDULE & SPRINT BACKLOG

| Task ID | Chapter/Sprint | Task / User Story | Owner | Start Date | Due Date | Priority | Status | % Complete | Evidence / Deliverable |
|---|---|---|---|---|---|---|---|---|---|
| T01 | Sprint 0: Proposal (2-8 Oct) | Write the problem project proposal and obtain topic acceptance | Gedion Basson | 02 Oct 2026 | 07 Oct 2026 | High | Complete | 100% | Project Proposal folder; Topic_Acceptance_Group3.pdf; commits 08ea06b, e03757d |
| T02 | Sprint 0: Proposal (2-8 Oct) | Revise the proposal after supervisor comments (scope narrowed to three core features) | Moegammad Green, Gedion Basson | 07 Oct 2026 | 07 Oct 2026 | High | Complete | 100% | SPM Problem Project Proposal (Revised).md; commits 5465df5, 78eef1e |
| T03 | Sprint 0: Proposal (2-8 Oct) | Create the GitHub repository, folder structure and add collaborators | Moegammad Green, Gedion Basson | 07 Oct 2026 | 08 Oct 2026 | High | Complete | 100% | github.com/Noobey32/TrackWIL; commits 629b191, cb29667, 725798c |
| T04 | Sprint 0: Proposal (2-8 Oct) | Group declaration and roles; ethical clearance confirmation (folder 00) | Gedion Basson | 08 Oct 2026 | 08 Oct 2026 | High | Complete | 100% | 00_Project_Administration; commit 08aaa44 |
| T05 | Sprint 0: Proposal (2-8 Oct) | Project charter and work plan (version 1) | Moegammad Green | 08 Oct 2026 | 09 Oct 2026 | High | Complete | 100% | 09_Project_Management_Evidence/ Project_Charter_and_Work_Plan; commits 9134c50, 4ebeae7 |
| T06 | Sprint 1 / Week 2 (9 Oct) | Chapter 1: Introduction (draft) | Sihle Kayise | 07 Oct 2026 | 09 Oct 2026 | High | Complete | 100% | 01_Introduction; commit 800fe83 |
| T07 | Sprint 1 / Week 2 (9 Oct) | Chapter 2: Literature Review and Literature Review Matrix | Dylan Shepard | 08 Oct 2026 | 09 Oct 2026 | High | Complete | 100% | 02_Literature_Review; commits 12de6ad, 16f0efc |
| T08 | Sprint 1 / Week 2 (9 Oct) | Harvard reference list | Dylan Shepard | 09 Oct 2026 | 09 Oct 2026 | Medium | Complete | 100% | Week 2 Submission/Reference list; commit 79cf0f8 |
| T09 | Sprint 1 / Week 2 (9 Oct) | Compile and submit Week 2 submission (Group3_Week 2.pdf) | Gedion Basson | 09 Oct 2026 | 09 Oct 2026 | High | Complete | 100% | Week 2 Submission folder; commits 87c899a, 5a84b15 |
| T10 | Sprint 1: App (9-15 Oct) | US-01 Next.js project set-up, sign-in, sign-out and sessions | Moegammad Green | 12 Oct 2026 | 12 Oct 2026 | High | Complete | 100% | 05_Source_Code/trackwil; commits 32c3f87, 5893fee |
| T11 | Sprint 1: App (9-15 Oct) | US-02/US-03 Logbook entries with evidence upload and hour progress | Moegammad Green | 12 Oct 2026 | 12 Oct 2026 | High | Complete | 100% | src/app/student, actions/logbook.ts; commit 5893fee |
| T12 | Sprint 1: App (9-15 Oct) | US-04/US-05 Submit for verification and mentor approval through a one-time link | Moegammad Green | 12 Oct 2026 | 12 Oct 2026 | High | Complete | 100% | src/app/verify, actions/verify.ts; commits 5893fee, 369fe55 |
| T13 | Sprint 1: App (9-15 Oct) | US-07 Coordinator dashboard (On track, Awaiting verification, Overdue) | Moegammad Green | 12 Oct 2026 | 12 Oct 2026 | High | Complete | 100% | src/app/coordinator, lib/logbook-metrics.ts; commit 5893fee |
| T14 | Sprint 1: App (9-15 Oct) | US-01 Student sign-up with CTU email and password rules | Moegammad Green | 12 Oct 2026 | 12 Oct 2026 | Medium | Complete | 100% | lib/signup-validation.ts; commit 5883791 |
| T15 | Sprint 2 / Week 3 (16 Oct) | UML, ERD and architecture diagrams (draw.io) | Moegammad Green | 14 Oct 2026 | 14 Oct 2026 | High | Complete | 100% | 04_System_Design_and_Development/diagrams; commit 29c2cab |
| T16 | Sprint 2 / Week 3 (16 Oct) | Chapter 4 draft (sections 4.1 to 4.6) | Moegammad Green | 14 Oct 2026 | 15 Oct 2026 | High | Complete | 100% | Chapter_4_System_Design_and_Development.docx; commit ad457c0 |
| T17 | Sprint 2 / Week 3 (16 Oct) | Software Requirements Specification (SRS) | Gedion Basson | 14 Oct 2026 | 14 Oct 2026 | High | Complete | 100% | 03_Requirements_and_Methodology; commit f84afa3; PR #2 |
| T18 | Sprint 2 / Week 3 (16 Oct) | Chapter 3: Requirements and Methodology (draft and formatting) | Sihle Kayise, Dylan Shepard | 14 Oct 2026 | 15 Oct 2026 | High | Complete | 100% | commits daf3c24, 52d753c |

Type text here

| Task ID | Chapter/Sprint | Task / User Story | Owner | Start Date | Due Date | Priority | Status | % Complete | Evidence / Deliverable |
|---|---|---|---|---|---|---|---|---|---|
| T19 | Sprint 2 / Week 3 (16 Oct) | Revise Chapters 1 and 2 using Week 2 feedback; revision table | Dylan Shepard | 15 Oct 2026 | 15 Oct 2026 | High | Complete | 100% | commits c5794a4, b4c0d76, f255d6e, 917f34f |
| T20 | Sprint 2 / Week 3 (16 Oct) | Compile and submit Week 3 submission with work plan and progress report | Gedion Basson, Moegammad Green | 15 Oct 2026 | 15 Oct 2026 | High | Complete | 100% | Week 3 Submission folder; commits 98e8363, 6b6b05b; PR #3 |
| T21 | Sprint 3 / Week 4 (23 Oct) | Chapter 5 draft, Test Plan and first test cases | Sihle Kayise | 20 Oct 2026 | 22 Oct 2026 | High | Complete | 100% | 06_Testing_Results_and_Evaluation ; commit 48df206; PR #4 |
| T22 | Sprint 3 / Week 4 (23 Oct) | Replace JSON file storage with an SQLite database (schema, seed, transactions) | Gedion Basson | 22 Oct 2026 | 22 Oct 2026 | High | Complete | 100% | 05_Source_Code/trackwil/database; commit 7080e7f; PR #5 |
| T23 | Sprint 3 / Week 4 (23 Oct) | Update Chapters 1-4, SRS and Database Design to match the build; class and activity diagrams; UI screenshots | Gedion Basson | 22 Oct 2026 | 23 Oct 2026 | High | Complete | 100% | commit 2634d57; PR #6 |
| T24 | Sprint 3 / Week 4 (23 Oct) | Update sprint backlog, risk register and weekly progress report | Gedion Basson | 23 Oct 2026 | 23 Oct 2026 | Medium | Complete | 100% | 09_Project_Management_Evidence (this branch) |
| T25 | Sprint 3 / Week 4 (23 Oct) | Fix defect: unsigned session cookie allows impersonation; retest | Gedion Basson | 23 Oct 2026 | 25 Oct 2026 | High | Complete | 100% | Commit 5268c2e (branch gedion/fix-defects); DEF-01 closed; TC-121 pass |
| T26 | Sprint 3 / Week 4 (23 Oct) | Execute all test cases with actual results; complete Defect Log and Requirements Traceability Matrix | Gedion Basson (automated suite); Sihle Kayise (manual TC-001 to TC-014) | 23 Oct 2026 | 26 Oct 2026 | High | Complete | 100% | Test_Cases_and_Results, Defect_Log, Requirements_Traceability_Matrix; tests/system-tests.mjs (36/36 pass) |
| T27 | Sprint 3 / Week 4 (23 Oct) | Record the 3-5 minute screen-recorded walkthrough | Gedion Basson | 23 Oct 2026 | 23 Oct 2026 | High | Not Started | 0% | Video link on first page of Group3_Week4.pdf |
| T28 | Sprint 3 / Week 4 (23 Oct) | Compile and submit Week 4 submission (Group3_Week4.pdf) | Gedion Basson | 23 Oct 2026 | 23 Oct 2026 | High | In Progress | 50% | Week 4 Submission folder |
| T29 | Sprint 4: Final (24-29 Oct) | Fix defect: entry stays Pending when its link expires; retest | Gedion Basson | 24 Oct 2026 | 23 Oct 2026 | Medium | Complete | 100% | Commit 5268c2e; DEF-02 closed; TC-137 pass |
| T30 | Sprint 4: Final (24-29 Oct) | Upgrade Next.js to a patched version and rerun tests | Moegammad Green | 24 Oct 2026 | 27 Oct 2026 | Medium | Not Started | 0% | Issue I04 in Risk Register |
| T31 | Sprint 4: Final (24-29 Oct) | Usability test and questionnaire (only with written permission and consent) | Dylan Shepard | 26 Oct 2026 | 27 Oct 2026 | Medium | Blocked | 0% | Awaiting written campus permission (Ethical Clearance Confirmation) |
| T32 | Sprint 4: Final (24-29 Oct) | Chapter 5: Testing, Results and Evaluation (final) | Sihle Kayise, Dylan Shepard | 24 Oct 2026 | 28 Oct 2026 | High | In Progress | 30% | Chapter_5_Testing_Results_and_Evaluation.docx |
| T33 | Sprint 4: Final (24-29 Oct) | Chapter 6: Conclusion and Recommendations | Gedion Basson | 26 Oct 2026 | 28 Oct 2026 | High | Not Started | 0% | 07_Conclusion_and_Recommendations |
| T34 | Sprint 4: Final (24-29 Oct) | Final dissertation: front matter, abstract, contents, lists, references and appendices | Gedion Basson | 26 Oct 2026 | 28 Oct 2026 | High | Not Started | 0% | 08_Final_Dissertation |
| T35 | Sprint 4: Final (24-29 Oct) | Complete the two incomplete Chapter 2 references | Dylan Shepard | 24 Oct 2026 | 26 Oct 2026 | Medium | Not Started | 0% | Chapter 2 reference list |
| T36 | Sprint 4: Final (24-29 Oct) | Presentation slides and demo script with backup plan | Moegammad Green, All | 24 Oct 2026 | 28 Oct 2026 | High | Not Started | 0% | 11_Presentation_and_Demonstration |
| T37 | Sprint 4: Final (24-29 Oct) | Individual Contribution Forms with Git evidence references | All members | 24 Oct 2026 | 28 Oct 2026 | High | Not Started | 0% | 12_Assessment/Individual_Contribution_Form.docx |

| Task ID | Chapter/Sprint | Task / User Story | Owner | Start Date | Due Date | Priority | Status | % Complete | Evidence / Deliverable |
|---|---|---|---|---|---|---|---|---|---|
| T38 | Sprint 4: Final (24-29 Oct) | Turnitin similarity check (below 15%) and final submission checklist | Gedion Basson | 28 Oct 2026 | 29 Oct 2026 | High | Not Started | 0% | 12_Assessment/Final_Submission_Checklist.xlsx |
| T39 | Sprint 4: Final (24-29 Oct) | Rehearse the presentation and live demonstration in business attire | All members | 28 Oct 2026 | 29 Oct 2026 | High | Not Started | 0% | Demo Script |

## PROJECT RISK, ISSUE & CHANGE REGISTER

| ID | Type | Date | Description | Likelihood | Impact | Priority | Owner | Mitigation / Action | Status | Due Date | Outcome |
|---|---|---|---|---|---|---|---|---|---|---|---|
| R01 | Risk | 08 Oct 2026 | Written campus permission to collect staff or student data arrives late or is refused, so no real usability or time-on-task data can be collected. | Medium | Medium | Medium | Gedion Basson | Develop, test and demonstrate with synthetic data only; state this in the dissertation; keep the questionnaire ready to run within a day if permission arrives. | Open | 27 Oct 2026 | |
| R02 | Risk | 08 Oct 2026 | Scope creep: analytics, bulk exports or audit logs are started before the three core features work. | Medium | High | High | Gedion Basson | Stretch goals only after the core works; deferred requirements recorded in Chapter 3. | Closed | 22 Oct 2026 | Core features delivered; stretch goals deferred to future work. |
| R03 | Risk | 08 Oct 2026 | A Friday milestone slips because members also have other assessments. | High | High | High | Gedion Basson | Owner tells the group lead in the same week; cut back to the core features; daily plan in the weekly progress report. | In Progress | 29 Oct 2026 | Week 2 and Week 3 submitted on time. |
| R04 | Risk | 09 Oct 2026 | Uneven Git contributions between members reduce individual marks (rubric criteria 6 and 10). | Medium | High | High | Gedion Basson | Each member commits their own work on a name/task branch and merges through a pull request; contribution forms reference specific commits. | Open | 28 Oct 2026 | |
| R05 | Risk | 12 Oct 2026 | JSON file storage rewrites the whole file on each change and has no keys or constraints, so data could be corrupted or become inconsistent. | Medium | High | High | Moegammad Green | Replace with an SQLite database with foreign keys, CHECK constraints and transactions. | Resolved | 22 Oct 2026 | Migrated to SQLite (PR #5, commit 7080e7f). |
| R06 | Risk | 12 Oct 2026 | Harmful or unexpected files uploaded as evidence. | Low | High | Medium | Moegammad Green | Allow only PDF and image types, store under random names outside the public folder, serve only to the owner and staff. | Resolved | 12 Oct 2026 | Implemented in actions/logbook.ts and api/evidence route. |
| R07 | Risk | 23 Oct 2026 | The live demonstration fails (laptop, network or data problem). | Medium | High | High | Moegammad Green | Run locally with the SQLite seed data; npm run db:reset before the demo; screen-recorded walkthrough and screenshots as backup; second laptop ready. | Open | 29 Oct 2026 | |
| R08 | Risk | 23 Oct 2026 | Turnitin similarity of 15% or more. | Low | High | Medium | Dylan Shepard | Write in own words, cite every source in Harvard style, check the full draft early. | Open | 28 Oct 2026 | |
| I01 | Issue | 22 Oct 2026 | Chapters 1 and 3 and the SRS described mentor accounts, a separate lecturer dashboard and PostgreSQL, which do not match the system that was built. | High | Medium | High | Gedion Basson | Rewrite the documents to match the build and record deferred requirements. | Resolved | 23 Oct 2026 | Documents updated (PR #6). |
| I02 | Issue | 22 Oct 2026 | Security defect: the session cookie stores the user id without a signature, so a user who edits the cookie can impersonate another account. | High | High | High | Gedion Basson | Sign the session cookie or use a random session token stored in the database; add a test case and retest. | Resolved | 25 Oct 2026 | Fixed on 23 Oct 2026: signed session cookie (commit 5268c2e); DEF-01 closed after retest TC-121. |
| I03 | Issue | 22 Oct 2026 | Defect: when a mentor link expires before the mentor responds, the entry stays Pending verification and the student cannot resubmit it. | Medium | Medium | Medium | Gedion Basson | Allow resubmission of a pending entry that has no active link; add a test case and retest. | Resolved | 26 Oct 2026 | Fixed on 23 Oct 2026 (commit 5268c2e); DEF-02 closed after retest TC-137. |
| I04 | Issue | 22 Oct 2026 | The Next.js build reports that version 16.4.0 is affected by a known security vulnerability. | High | Medium | Medium | Moegammad Green | Upgrade Next.js on a separate branch and rerun all tests before merging. | Open | 27 Oct 2026 | |
| I05 | Issue | 23 Oct 2026 | Two Chapter 2 references are incomplete (Hains-Wesson et al. 2026; Sonia Online n.d.). | High | Medium | Medium | Dylan Shepard | Find the original sources and complete the Harvard references. | Open | 26 Oct 2026 | |
| I06 | Issue | 23 Oct 2026 | Workplace mentors cannot open evidence files from the verification link, because evidence downloads require sign-in. | High | Medium | Medium | Moegammad Green | Recorded as a limitation and future work; mentors verify from the entry description. | Accepted | 23 Oct 2026 | Documented in Chapter 3 (deferred requirements). |

| ID | Type | Date | Description | Likelihood | Impact | Priority | Owner | Mitigation / Action | Status | Due Date | Outcome |
|---|---|---|---|---|---|---|---|---|---|---|---|
| C01 | Change | 07 Oct 2026 | Scope narrowed to three core features (logbook, token verification, coordinator dashboard) after supervisor feedback; analytics, exports and audit logs moved to stretch goals. | Low | Medium | Medium | Gedion Basson | Revise the proposal and work plan. | Closed | 07 Oct 2026 | Revised proposal accepted (commit 5465df5). |
| C02 | Change | 07 Oct 2026 | Week 1-7 plan replaced with one-week sprints ending on each Friday submission, finishing on 29 October. | Low | Medium | Medium | Gedion Basson | Update the methodology and work plan. | Closed | 07 Oct 2026 | Sprint plan in Chapter 3. |
| C03 | Change | 12 Oct 2026 | Workplace mentors verify through a one-time link instead of having accounts and a mentor dashboard. | Low | Medium | Medium | Moegammad Green | Implement token verification; update requirements and design. | Closed | 23 Oct 2026 | Implemented; Chapters 1, 3 and 4 updated. |
| C04 | Change | 12 Oct 2026 | Lecturers use the coordinator dashboard instead of a separate lecturer dashboard. | Low | Low | Low | Moegammad Green | Route lecturers to /coordinator; update requirements. | Closed | 23 Oct 2026 | Implemented; lecturer demo account added. |
| C05 | Change | 22 Oct 2026 | Data storage changed from JSON files to SQLite. | Low | High | Medium | Gedion Basson | Migrate the repository layer only; rerun all tests. | Closed | 22 Oct 2026 | PR #5; 28 automated checks passed. |
| C06 | Change | 22 Oct 2026 | Written mentor feedback on rejected entries deferred to future work. | Low | Medium | Medium | Gedion Basson | Record in the deferred requirements and Chapter 6 recommendations. | Accepted | 23 Oct 2026 | Documented in Chapter 3. |